
Quantum Navigation

Release 1.1.0

University of Liverpool

Mar 25, 2025

CONTENTS:

1	About	3
1.1	Project Structure	3
1.2	Software Usage	5
1.3	Technical Overview	8
1.4	Trajectory Generation	10
1.5	Gravity Modelling	11
1.6	Sensor Processing	13
1.7	Fusion Methods	14
1.8	User Configuration	16
1.9	Software Components	24
1.10	System Requirements	146
1.11	Example Usage	149
1.12	Frequently Asked Questions	152
1.13	Potential Future Work	153
1.14	Software Licence	154
2	Indices and tables	155
	Python Module Index	157
	Index	159



ABOUT

A Python based navigation toolbox designed to be used for experimenting with multi-source navigation. This library provides the necessary services to conduct in-depth realistic simulations, fully configurable by the user. These can extend from employing simple numerical methods to track movement along a basic pre-generated racetrack, to using kalman filters with tightly-coupled GPS fusion to track a custom vehicle along a bespoke trajectory. The implementation and object association is easy to understand and supports third-party additions. In summary, the toolbox presents the following key features:

- Trajectory generation with vehicle constraints
- Configurable accelerometers, gyroscopes and altimeters
- External clock allowing for controlled timing errors
- Selection of Inertial Navigation (INS) Fusion Methods
- GPS satellite simulation with pseudo range generation
- GPS fusion (fixed gain, loosely-coupled, tightly-coupled)
- Inertial based quantum sensors and fusion methods
- Gravity gradient based quantum sensors and fusion methods
- Generating and plotting interactive results in web browser
- Handling of various input and output file types for data
- Support for platforms with limited hardware resources

1.1 Project Structure

This project contains a handful of files, directories and sub-directories; each with their own purpose. This section outlines the role of all project folders and files. Each are grouped and outlined below:

1.1.1 Project Folders

/src/

All source code files that the toolbox are composed of. This directory is identified as a “Python Directory” and houses all the developed code. This folder contains the project `qnav` and its series of sub-packages.

/test/

All pytest test functions and classes. This can be executed to re-perform all automated tests used to evaluate the correctness and robustness of the project.

/docs/

A collection of scripts and configuration files used for auto-generating the user documentation (in HTML offline webpages, LaTeX and PDF formats). This folder mostly consists of ReStructuredText (.rst) files which describe the information to be included within the documentation.

/config/

A series of sub-configuration profiles that can be imported by main configuration profiles (such as `default_settings.ini`). The purpose for these is to reduce the complexity of the main configuration script by partitioning sets of frequently used variables into sub-profiles. This folder contains sub-profiles for various aspects such as vehicle types, hardware models and waypoint trajectories.

/examples/

A collection of helpful example scripts providing useful sample code that can be executed to demonstrate the usage of various features contained within the toolbox. These examples including: generating trajectories, loading and altering configuration files and using various databases.

/output/

The default output directory produced by the toolbox. Here output results (data and report figures) are saved to auto-generated folders named by their time and date of creation. Check this folder after running a simulation to see the results produced.

1.1.2 Project Files

default_settings.ini

The default configuration settings automatically used by the project if a configuration file of choice is not presented. When making custom configuration files it is recommended to create a copy of `default_settings.ini` to use as a fully commented template.

README.md

A short description about the purpose of the project and how to use it. Only minor details are covered to provide a quick-start guide. This file is automatically detected and previewed by some software.

pyproject.toml

A standardised configuration file for the project, namely specifying build system requirements. Included to solve build-tool dependency problems and install necessary build tools in a virtual environment. Also required for configuring various development tools.

requirements.txt

A list of all the required Python packages along with the minimum version number. This can be used for automatically installing all requirements via applications such as [PIP](#).

LICENSE

Standard MIT license included with the project

1.2 Software Usage

The navigation software can be used in many ways. Firstly, and most commonly, it can simply be executed via command line with a selected configuration file. The toolbox can also be executed through Integrated Development Environments (IDEs) such as PyCharm, Spyder or IDLE. These are particularly useful for viewing the output values in detail and/or performing additional processing steps such as calculating or plotting statistics. Lastly, the toolbox can be integrated and employed by other Python projects. The most common ways to use the software are discussed in this section.

Important

During development it was noted that the interface and layouts for the various IDEs used evolved with time. There is also small variations between platforms such as Microsoft Windows and MacOS. The names and locations of the buttons, menus and settings discussed in this section could have changed slightly.

1.2.1 Command Line Usage



For standalone usage, the software simulation can be simply executed from the command line when within the project's root directory. This usage has the advantage of not requiring the software to be installed or invoked via Python programming. To run a simulation directly, *employing the default configuration settings*, the following single command can be used. This launch command is identical for Windows, Linux and MacOS systems.

Listing 1: Execute PNT Toolbox with default settings

```
python3 run.py
```

If multiple versions of Python are installed on the host machine it is always best to explicitly state which version is to be used to avoid any compatibility issues. To use Python 3.12 replace **'python3'** with **'python3.12'**.

On execution, the software will extract its settings and user preferences from a provided configuration file. Configuration files are used to describe all input parameters including the simulated environment, hardware/sensor activity and vehicle limitations/restrictions. A requested configuration can be state by supplying its name (and location) as the first argument to the run script. If a configuration file is not provided, the default configuration `default_settings.ini` will be used automatically.

An example of using a custom created configuration file named `custom_config.ini` (that is located in the project's root directory) has been generated below.

Listing 2: Execute PNT Toolbox with custom settings

```
python3 run.py custom_settings.ini
```

The expected structure and format of configuration files is discussed in detail under [User Configuration](#) (discussed next).

1.2.2 Anaconda/Spyder Usage



To run the software through [Anaconda/Spyder](#) simply open Spyder and select “**New Project...**” under the “**Project**” dropdown menu at the top of the page. From there, create a new project with an appropriate name such as ‘Nav Toolbox’. After this step, copy or move the entire contents of the toolbox’s root directory to the location of the created Spyder project. Once complete, the files should appear on the left hand-side in Spyder. To launch the software **right-click** the script `run.py` and select “**run**”. The software should now be running with the default settings.

Note

It is best to create a new project rather than attempting to open the toolbox as an existing one, since creating a new project results in Spyder producing new hidden auxiliary and settings files (based on your environment rather than trusting the included ones).

If you wish to select a different configuration file to use rather than the default settings, this can be done so by providing the location of a configuration file as the first command line argument. To do this in Spyder simply select “**Configuration per file...**” from the “**Run**” dropdown menu at the top. Ensure the execution file `run.py` is selected under the dropdown box titled “**Select a run configuration:**”. Then under “**General settings**” enable “**Command line options:**” and enter the name/location of the configuration file to use. To launch the toolbox with these settings click the “**run**” button located at the bottom of the menu.

For an example, let's assume we wish to use a configuration file called `custom_settings.ini` which is located in the same directory as `default_settings.ini`. To do this, we would enter `custom_settings.ini` into the “**Command line options:**” box. The toolbox will then automatically find the configuration file to use.

1.2.3 PyCharm Usage



To use the software with JetBrain's PyCharm simply create a new project. If not prompted at the home screen, this can be achieved by selected “**New Project...**” from the “**File**” dropdown menu at the top of the screen. From there ensure the project is being created in a sensible location and given an appropriate name. During the new project's creation you will be asked to select the “**Python Interpreter**” to use. It is recommended to select “**New Environment Using:**” one of the followings:

1. **Virtualenv** - Isolated virtual environment (highly recommended).
2. **Pipenv** - Creates and manages a virtualenv with Pipfiles.
3. **Conda** - Anaconda Python distribution (requires Conda to be installed).

Note

More information of configuring a Python interpreter in PyCharm is available [here](#)!

With the project created copy the Quantum Navigation files to the its location. Once complete, the files should appear within the left hand-side panel. To launch the software **right-click** the script `run.py` inside the root folder and select “**Run 'run_simulation'**”. The software should now be running with the default settings.

If you wish to select a different configuration file to use rather than the default settings, this can be done so by providing the location of a configuration file as the first command line argument. To do this in PyCharm simply **right-click** the script and select the **More Run/Debug** menu, then select “**Modify Configurations...**”. From this menu enter the name/location of the configuration file to use in the box named “**Script Parameters**” and then click “**Apply**”. Each time the `run.py` is executed these settings will now be used.

Note these settings may reset or change between sessions or when executing other scripts.

1.2.4 Eclipse/PyDev Usage



The project can also be used in PyDev (an Eclipse plug-in for Python development). To use the Quantum Navigation Toolbox in PyDev, simply create a new project. This can be done by going to the “**File**” Dropdown menu and selecting “**New**”, “**Project**”, “**PyDev**”, then “**PyDev project**”. From the PyDev Project creation window ensure the new project is given a suitable name and location. Ensure “**Python**” is selected as the “**Project type**” and that a compatible version of Python is chosen (i.e. Python 3.8 or above).

Note

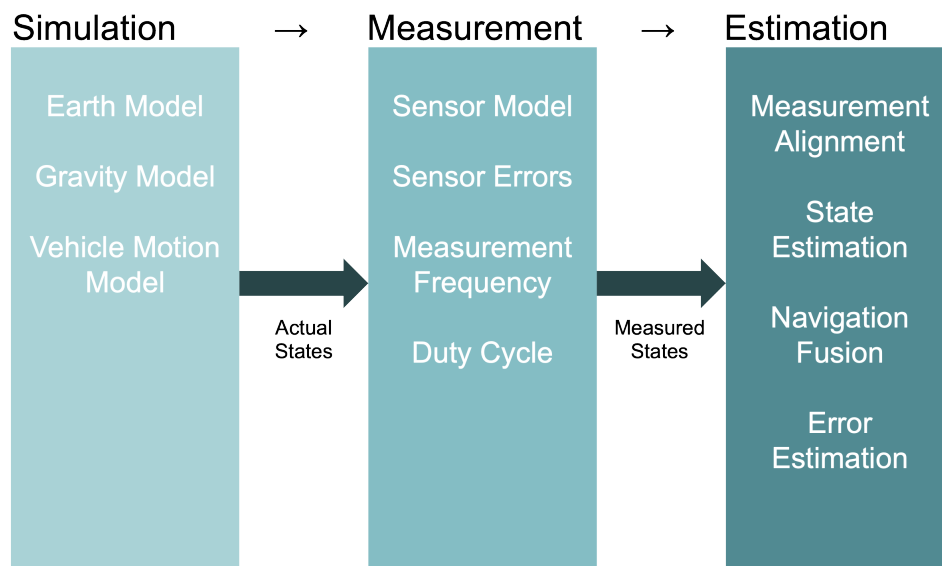
More information of creating and importing Python projects in PyDev is available [here](#)!

With the project created copy the toolbox files to the its location. Once complete, the files should appear within the left hand-side panel. To launch the software within PyDev **open the script** `run.py` inside the root folder. With the script `run.py` being displayed in the editor, go to the “**Run**” dropdown menu and select “**Run As**” followed by “**Python Run**”. Alternatively the shortcut key **F9** can be used to run the script currently open. The software should now be running with the default settings.

If you wish to select a different configuration file to use rather than the default settings, this can be done so by providing the location of a configuration file as the first command line argument. To do this in PyDev firstly open the main run file `run.py` so it is being displayed in the editor. Then go to the “**Run**” dropdown menu and select “**Run Configurations...**” located under the “Run As”. From there ensure the current script is selected on the **left-hand side** under “**Python Run**”. Finally, under the “**Arguments**” tab enter the name/location of the configuration file to use in the box titled “**Program arguments:**” then click “**apply**” followed by “**Run**”. If needed, alternative steps for setting runtime arguments are available [here](#)!

1.3 Technical Overview

This section discusses the technical aspects of the project; detailing the internal dataflow. This is namely used to describe the key stages during typical execution of the software. Moreover, this section also outlines how the project correctly enforces separations between the simulated “real” world, extracted measurements and the estimated states.



1.3.1 Trajectory Generation

Firstly, during a typical run-cycle, the software will firstly calculate the ground truth trajectory. This is performed using a two-step approach. During the first step a trajectory path is constructed to reach the given base points while complying with the selected vehicle model. This is generated at a frequency subject to the vehicles speed. During the next step the trajectory properties are calculated at each step (being the position, velocity, acceleration, attitude and angle rate records). These produced values can then serve as a look-up table with interpolation, using the simulation time in seconds as the key.

1.3.2 Estimation Initialisation

Secondly, with the ground truth trajectory generated, the initial estimation states can be declared. In the simplest usage these can be read directly from the first record of the ground truth trajectory (at 0 seconds). But realistically, initial estimation errors will be applied on top of these. Other differences, such as expected state errors and a non-identical gravity model (purposely unlike the one used to generate the ground truth) can be set here.

1.3.3 Sensor Configuration

Thirdly, using the initial estimated states, the sensors and their corresponding fusion methods can now be configured and paired. Individual sensors, such as the accelerometer and gyroscopes, can be configured with custom error profiles, axis orientations and random number generation (used for generating pseudo-random noise and errors). These will then assigned to one or more fusion methods. For instance, the accelerometer and gyroscopes could be allocated to a configured Kalman Filter for INS fusion. In addition, the same sensors could be shared and also assigned to other fusion methods, such as quantum sensors.

1.3.4 Simulation Loop

With everything configured, the main simulation loop can now be executed with all over the above. At a minimum, a simulation loop requires a ground truth trajectory, initial estimated state and a series of sensor-fusion pairs. Optionally, a custom clock model can be included to invoke purposeful timing errors. Custom display and result collectors can also be passed to obtain information with configured levels of filtering.

During the first step of the simulation, a scheduler is initialised for handling the sensors and fusion, calling the next due sensor or fusion method on time. When a sensor is ready for a measurement, the simulation time is move up to that time, with clock errors applied. The sensor is then given the ground truth record for that time, which is then uses to produce a noisy measurement (which is held/cached internally). When a fusion method is ready, depending on its trigger mode, it collects the noisy measurements from its sensors and processes then before updating the estimated state.

This approach allows sensors and fusion methods to be dynamically included in simulations, while enforcing complete separation between ground truth (solely seen by sensors) and the estimated state (solely updated by fusion methods with noisy measurements).

1.3.5 Capturing Results

After the simulation loop is complete, the captured results (either from the default or given results collector) will be returned. Results are firstly written to disk using the projects own binary file format (optimised for sequentially reading and writing of records). After this, results will then be exported in any of the requested file formats (such as csv, binary, numpy, MATLAB save) with various levels of compression (both lossy and lossless). Separate files are made for both the ground truth (optional) and estimation state records.

1.3.6 Displaying Results

If enabled, after results have been written to disk, their data will be displayed in a series of interactive graphs and plots comparing the ground truth and estimation history against time. The supported figure types are as listed:

- Summary Table Plot
- 2D Open Street Map Plot
- 2D Lat-Lon Plot
- 3D Lat-Lon-Alt Plot
- Position Axes Plot
- Velocity Axes Plot
- Acceleration Axes Plot
- Attitude Axes Plot
- Angle Rate Axes Plot

1.3.7 Recreation

After the simulation, the same results can be recreated by setting the same parameters and random seeds. To aid with this, a complete configuration file containing all parameters used is dumped into the generated results folder.

1.4 Trajectory Generation

Ground truth records are represented as a trajectory, a series of waypoint records generated at fixed frequency that support interpolation look-up. The project allows a trajectory is constructed of a vehicle model, a described path, and a gravity model.

1.4.1 Vehicle Model

A vehicle model describes the limitations of the vehicle platform, stating its expected speed, moving capabilities, acceleration rates, maneuvering restrictions and expected behaviour. These essentially define how trajectories are generated with realistic behaviours. For example, the path generated by a land vehicle and large ship vessel will be very different.

Important

A vehicle model is required to restrict the simulated maneuvering of the vehicle platform to within realistic bounds. Ensure one is configured correctly for the requested trajectory.

1.4.2 Path Generation

Trajectory paths are configured either as a arithmetical shape or series of base points that will form a shape. From these, a series waypoint positions are generated (using constraints set by the vehicle model). This operates by simulating controlled movement to reach each base point, or get as close to as possible. The result from this is a table of positions at fixed frequency. These can be generated using one of the three methods:

INI Configuration

Properties in the INI configuration file can describe the the latitude, longitude and altitude base points for a trajectory to cover. Moreover, a sub-configuration file can be referenced, allowing for a complex trajectory to be shared. Indepth details for INI configuration is covered by it own section.

CSV Configuration

Similar to the above, a comma-separate-values (CSV) file can be used to describe the base points. For this a CSV file containing either three or four columns can be used. If three columns are used, the records for each line will assume to be the latitude, longitude and altitude (given in decimal degrees, decimal degrees and metres respectively). Alternatively, if four columns are used, then the first will be the arrival time at each base point in seconds. If this is not provided, the vehicle model's average speed will be used to determine this.

Racetrack Configuration

A Racetrack is the name referred to for a looping arithmetical trajectory. The advantage to using one of these instead of the above base point methods is that a trajectory path of specified duration and lap time can be quickly generated at any location, which is useful for rapid prototyping.

1.4.3 Gravity Model

A gravity model is required for correctly calculating the gravitational and coriolis effects on the trajectory values. The selected model can range from something as simple as the Somigliana formula, to something as complex as a large scale database. Typically, a far more accurate model is used to generate the ground truth than what is used during simulation estimation.

Note

More information on *gravity models* is given under its own section.

1.5 Gravity Modelling

The project include a small collection of gravity models. Each is equipped with their own methods for calculating various values, such as the vector acceleration, vertical gravity gradient and expected anomaly or disturbance values for given positions.

Gravity models are grouped into two sets, being functions and correction maps. Gravity functions serve as a simple and quick approximation for the gravity at a position. These can be applied for any location and are used as base values. Extending from these are the gravity correction maps, which as suggested apply a correction on top of a selected base gravity function. These provide correction records, but can only be applied in areas they cover. If data for the requested position is unavailable, no correction to the value produced by the base function will be applied.

Note

For simple usage, only a basic gravity function will be needed. However, for applying realistic gravity behaviour, which is required for some forms of sensing, a high resolution correction map will also be needed.

1.5.1 Gravity Functions

Fixed Constant Value

Essentially not a function, serving a fixed value of 9.81 used for vertical acceleration at any position. This is primarily used for debugging.

Simple Uniform Sphere

Approximates the Earth as uniform sphere with height extrapolation. A slight improvement over the Fixed Constant model, but is again primarily used for debugging.

Somigliana Formula

The standardised “Normal” gravity function with height extrapolation (named after Carlo Somigliana (1860–1955). Also known as the International Gravity Formula 1980. This is far more accurate than the previous models, and also produces east component values in calculated gravity vectors.

NIMA’s WGS-84 Gravity

Calculations provided by National Imagery and Mapping Agency (NIMA) in “Department of Defense World Geodetic System 1984, Its Definition and Relationship with Local Geodetic Systems.”. This shares many similarities with the Somigliana model. This was namely included due to it being used in software such as MATLAB as standard.

1.5.2 Gravity Correction Maps

EGM Models

Application of 7 Earth Gravitation Models defining geoid undulations in raster form. These essentially provide the geoid height correction on top of the WGS-84 reference ellipsoid. Supported EGM models include:

- EGM-1984 (in 30 and 15 arc-minute resolutions)
- EGM-1996 (in 15 and 5 arc-minute resolutions)
- EGM-2008 (in 5, 2.5 and 1 arc-minute resolutions)

Note

While EGM-2020 has been technically released, it is currently unavailable for public access.

Geosat-44

the Geosat-44 gravity model provided data collected in 1999 from satellite altimeter measurements, which contain precise geoid and gravity anomaly profiles constructed from the average of 44 repeat cycles. In total, this dataset contains 987,755 ascending records and 991,313 descending records between the latitude bounds +/- 72 degrees north/south. This model is represented as scatter points, which are triangulated and used in interpolation.

Marine Gravity Map

The marine gravity map provides gravity anomaly values for acquired from satellite altimeter data in 2013. It is based on previous data from CryoSat-2 and Jason-1 satellite projects. The resulting model is a 9,600 x 21,600 raster grid, between the latitude bounds +/- 80 degrees north, with interpolation performed for land space.

GGM Plus

The Global Gravity Map Plus (GGM Plus) model is the result of a joint research initiative by Curtin University and TU Munich University in 2013. It provides records of gravitational acceleration and distributable at a 7.2 arc-second (approximately 200 metres) spatial resolution, for all land and near-coastal areas. The resulting model consists of 3,062,677,383 points, partitioned into 5 x 5 degree tiles, covering latitude bounds +/- 60 degrees north/south.

SRTM2Gravity

The SRTM2Gravity model extends from GGM Plus by performing the conversion of Shuttle Radar Topography Mission-based digital elevation data to implied gravity effects. It provides records for full-scale and residual gravity at 3 arc-second (approximately 90 metres) spatial resolution, for all land and near-coastal areas. The resulting model is partitioned into 1 x 1 degree tiles, covering latitude bounds +/- 60 degrees north/south.

Irish Sea Model

A bespoke marine gravity anomaly map covering an area a north western region Irish sea. Support for this model was included by request.

1.6 Sensor Processing

To acquire information from the ground truth and produce measurement of control noise and errors, the project presents a handful of sensors. Each of these implement the base sensor class and feature methods for generating measurements, updating internal states and indicating when their next measurement is due. These sensors are categorised below:

1.6.1 Classic Inertia

Classes representing conventional inertial based sensors are the accelerometer and gyroscope. These are implemented as generic sensors, as they are reused internally by other sensors within the toolbox. These are responsible for obtaining acceleration and angle rate measurements respectively.

1. *Generic Accelerometer* (Accelerometer)
2. *Generic Gyroscope* (Gyroscope)

1.6.2 Altitude

For obtaining altitude measurements, an altimeter sensor has been implemented. This simple obtains the measured vertical position, following configured noise profiles.

1. *Generic Altimeter* (Altimeter)

1.6.3 GPS/GNSS

For simulating GPS measurements two classes have been implemented. The first class is satellite, which represents a satellite in orbit. While not specifically a sensor class, multiple satellite instances are used by the GPS sensor class, which is responsible for producing measurements. Measurement information include clock times, pseudo range information and estimated position and velocity data.

1. *GPS Satellite* (Satellite)
2. *GPS Sensor* (GPSSensor)

1.6.4 Quantum Inertia

Two cold-atom quantum based inertia sensors have been implemented. The first one is a basic concept, simply extending the use of the generic altimeter by simulating fairly accurate average inertia measurements over a duty cycle. The second one improves on this by simulating and processing the movement of atoms to gain measurements.

1. *Concept Quantum IMU* (ConceptQuantumImu)
2. *Realistic Quantum IMU* (QuantumIMU)

1.6.5 Quantum Gravity

One cold-atom gravity gradient sensor has been implemented. This simulates a dual-interferometer, acquiring measurement signals from the releasing of two vertically stacked clouds of atoms that share a phase. A series of signals collected across multiple duty-cycles can be acquired to form an ellipse, used to estimate the vertical gravity gradient.

1. *Quantum Gravity Gradiometer* (GravityGradiometer)

1.7 Fusion Methods

To utilise measurements acquired from the sensors described in the previous section, a collection of fusion methods have been implemented. These are grouped and categorised as before.

1.7.1 Classic Inertia

Numerical integration methods for the INS include: Simple Numerical, 4th Order Runge-Kutta, Adams Bashforth and Kalman Filter Integration. For certain cases, Kalman Filter Integration will be required to leverage its covariance matrices. For example, some GPS and Quantum Sensor fusion methods will require access to these.

1. *Simple Numerical* (NumericalINS)
2. *Runge Kutta* (RungeKuttaINS)
3. *Adams Bashforth* (AdamsBashforthINS)
4. *Kalman Filter* (KalmanINS)

1.7.2 Altitude

For processing altimeter measurements two fusion methods have been implemented. The first is simple fixed-gain, which updates the current estimated altitude by using a weighted average between the measurement and current state. The second is alpha-beta fusion, which extends on the previous by also correcting the vertical velocity in a similar way.

1. *Fixed-Gained Fusion* (FixedGainAltimeter)
2. *Alpha-Beta Fusion* (AlphaBetaAltimeter)

1.7.3 GPS/GNSS

For processing GPS measurements three fusion methods are available. Firstly, fixed gain simple updates the estimated position and velocity by using a fixed weighted gain for the measurements produced by the satellites. Secondly, Loosely coupled fusion improves on this by using a Kalman filter to integrate INS parameters with the received GPS position and velocity measurements. Lastly, Tightly coupled fusion further extends this by operating on the raw GPS measurements from the receiver, such as the satellite pseudo-ranges and Doppler observables.

1. *GPS Fixed Gain* (GpsFixedGainFusion)
2. *GPS Loosely Coupled* (GpsLooseFusion)
3. *GPS Tightly Coupled* (GpsTightFusion)

1.7.4 Quantum Inertia

For processing cold-atom quantum inertia measurements three fusion methods are implemented. The first method is called Concept Fusion, which mostly serves as a base class. From the collected measurements this simple calculates the believed sensor biases and corrects the estimated state. The second method is a particle filter method, which applies a particle filter approach for estimating the acceleration and angle rate biases independently. The third method extends on this further by also accounting for any sensor misalignment.

1. *Concept Quantum Fusion* (ConceptQuantumFusion)
2. *Particle Filter Quantum Fusion* (QuantumPFFusion)
3. *Particle Filter and Misaligned Quantum Fusion* (QuantumPFMFusion)

1.7.5 Quantum Gravity

Currently, there is only one fusion method for processing the gravity gradient sensor, by performing gravity gradient map matching. This employs a particle filter approach, exploring a search space for best fitting position, velocity and attitude candidates.

1. *Quantum Gravity Gradient Fusion* (GravityGradientPF)

1.8 User Configuration

Configuration is primarily performed by providing the location of a configuration file to use as the first command line argument. Examples of how to do this is cover under *Software Usage*. This section discusses the various sections contained with a standard configuration files, along with their purpose and key variables.

In total, there is over 400 customisable variables that can be changed. Except for a few cases, each variable has its own default value which is used if it is not included in the current configuration file. In the event a configuration file is not provided, the default configuration `default_settings.ini` will be used automatically. It is recommended that a copy of `default_settings.ini` is made and renamed. This copy can serve as a fully commented template configuration file to use and modify.

Note

Configuration files used by the project are formatted as “initialisation” files (.ini format). Because .ini files are a popular and commonly used method for software configuration, many editor programs support syntax highlighting for them. To avoid making mistakes or typos, it is recommended to use an editor that supports initialisation file format.

Important

- Both = and : can be used for assigning values.
- The symbols # and ; are used to state comments.
- String values do not require quotes such as " or '.
- In-line comments (comments after variables) are **not** supported.
- For numerical values anything after a string character is ignored.
- File paths can be given in either Windows or POSIX (Linux) formats.
- Valid boolean values are true, false, yes, no, on, off, 1 or 0.
- Mathematics expressions such as +, -, *, /, //, ^ and e are supported.
- To reduce the likely-hood of errors and typo, variable names are **case insensitive**.

1.8.1 Configuration Sections

A configuration file is composed of 15 sections in total: **Output, Database, Waypoints, Racetrack, Vehicle, Gravity, Geoid, Measurement, Estimation, Altimeter, GPS, Quantum, Holonomics, Random and Misc**. The use of each of these section is discussed below along with a description for the most common variables used:

Output Settings

The **Output** section holds all variables used for controlling the output of the simulation. This includes settings for stating what is to be displayed during runtime, what files are to be produced, where files are to be saved, any down-sampling or compression that should be applied.

Table 1: Key Output Section Variables

Variable Name	Description	Type
saveResults	Should summary results be saved to disk?	Yes <i>or</i> No
saveFigures	Should figures be generated and saved to disk?	Yes <i>or</i> No
showFigures	Should figures be generated in the web-browser?	Yes <i>or</i> No
copyConfig	Should a copy of the configuration file be created?	Yes <i>or</i> No
resultsDownSampleRate	A factor to down-sample the collection of results by.	Integer
outputDownSampleRate	A factor to down-sample the exporting of results data by.	Integer
resultsPath	The location where all results are to be saved.	Filepath
appendDateFolder	Should the date be appended to the output?	Yes <i>or</i> No

Note

Toolbox supports the following save file formats:

- csv - Comma Separated Values
- csv_gz - Comma Separated Values with G-Zip compression
- numpy - Numpy binary array file (generated with np.save())
- numpy_gz - Numpy array file with G-Zip compression
- binary - Standard float64 big-endian binary file
- mat - MATLAB v7.2 save file without compression
- mat_comp - MATLAB v7.2 save file with compression

Database Settings

The **Database** section controls the location where datasets are read from and where they will be written to.

Table 2: Key Database Section Variables

Variable Name	Description	Type
geoidDatabase	Directory for the geoid dataset files.	Filepath
gpsDatabase	Directory for the GPS Rinx dataset files.	Filepath
geosatGravityDatabase	Directory for the Geosat-44 gravity model files.	Filepath
marineGravityDatabase	Directory for the Marine gravity model files.	Filepath
ggmPlusGravityDatabase	Directory for the GGM Plus gravity model files.	Filepath
srtm2GravityDatabase	Directory for the SRTM2Gravity gravity model files.	Filepath
irishSeaGravityDatabase	Directory for the Irish Sea gravity model files.	Filepath

Waypoints Settings

The **Waypoints** section controls how the true trajectory is generated and used. This is where information of the effective trajectory path is stated. This also controls the timings and frequency of the data.

Table 3: Key Waypoint Section Variables

Variable Name	Description	Type
waypointFrequency	The waypoint/simulated world's update frequency to use.	Float
waypointInterpolation	The interpolation method to use for ground truth look-up.	cubic, pchip <i>or</i> akima
waypointMode	The mode for reading base waypoints.	config, csv <i>or</i> racetrack
waypointConfig	A waypoint sub-configuration file.	Yes <i>or</i> No
waypointCSV	A waypoint CVS file.	Yes <i>or</i> No
useVirtualMemory	Should disk space be used to hold waypoint data?	Yes <i>or</i> No

Racetrack Settings

The **Racetrack** section controls how arithmetical looping trajectories will be generated, if the waypoint method is set to racetrack. This can be useful for quickly experimenting with different settings or profiles, without having to present a planned route to use. The produced track is a looping trajectory which will be followed until the simulations maximum time is reached.

Table 4: Key Waypoint Section Variables

Variable Name	Description	Type
circuitTime	The time (in seconds) to complete a single lap of the circuit.	Float
distanceX	The distance span in the X (North) direction (in metres).	Float
distanceY	The distance span in the Y (East) direction (in metres).	Float
distanceZ	The distance span in the Z (Down) direction (in metres).	Float
numVariationsY	The number of loops in the Y axis with respect to the X axis.	Float
numVariationsZ	The number of loops in the Z axis with respect to the X axis.	Float

Vehicle Settings

The **Vehicle** section lists all the values used for stating the limitations of the vehicle (model) that is to be use. This values effect the shape of the requested trajectory, the manoeuvring limitations and the movement noise of the vehicle.

Table 5: Key Vehicle Section Variables

Variable Name	Description	Type
vehicleConfig	The location of the vehicle sub-profile to use.	Filepath

Gravity Settings

The **Gravity** section controls all gravity related calculations, specifically the gravity functions and gravity correction maps used in both ground truth and estimation.

Table 6: Key Gravity Section Variables

Variable Name	Description	Type
autoDownload	Should missing gravity data be downloaded automatically?	Yes <i>or</i> No
trueGravityFunction	The gravity function to use for the ground truth.	See Below
trueGravityCorrection	The optional gravity correction map to use for the ground truth.	See Below
estimatedGravityFunction	The gravity function to use for the simulation estimation.	See Below
estimatedGravityCorrection	The optional gravity correction map to use for the simulation estimation.	See Below

Note

Toolbox supports the following gravity function options:

- **fixed** - Simple fixed constant value
- **uniform** - Uniform sphere with height interpolation
- **somigliana** - Somigliana formula for theoretical gravity
- **wgs84** - NIMA's WGS-84 ellipsoid gravity model (used by MATLAB).

Note

Toolbox supports the following gravity correction map options:

- **none** - Do not use a map, simple use the function alone.
- **geoid** - Use the EGM model (defined under Geoid section).
- **geosat** - Use the geosat-44 gravity correction map.
- **marine** - Use the marine gravity correction map.
- **ggmPlusAcc** - Use GGMPlus gravity acceleration map.
- **ggmPlusDist** - Use GGMPlus gravity disturbance map.
- **srtm2gravityFS** - Use SRTM2Gravity full-scale gravity map.
- **srtm2gravityRes** - Use SRTM2Gravity residual gravity map.
- **irishSea** - Use bespoke Irish sea gravity anomaly map.

Geoid Settings

The **Geoid** section controls all the selection of geoid model for the ground truth and simulation estimation,

Table 7: Key Geoid Section Variables

Variable Name	Description	Type
trueGeoidModel	The geoid model to apply when calculating the ground truth.	See Below
estimatedGeoidModel	The geoid model to apply during estimation simulations.	See Below

Note

Toolbox supports the following geoid model options:

- none - Do not use a geoid model
- egm84-30 or egm84-15 - For the EGM 1984 model (at 30 or 15 minute resolutions).
- egm96-15 or egm96-5 - For the EGM 1996 model (at 15 or 5 minute resolutions).
- egm2008-5, egm2008-2_5 or egm2008-1 - For the EGM 2008 model (at 5, 2.5 or 1 minute resolutions).

Measurement Settings

The **Measurement** section contains mostly hardware configurations for the sensors to use. This includes stating measurement frequencies and related errors.

Table 8: Key Measurement Section Variables

Variable Name	Description	Type
imuMeasurementFreq	The virtual hardware measurement frequency in Hertz.	Float
altimeterMeasurementFreq	The altimeter measurement frequency in Hertz (if enabled).	Float
gpsMeasurementFreq	The GPS measurement frequency in Hertz (if enabled).	Float
accelerometerConfig	The accelerometer profile to use for measurements.	Filepath
gyroscopeConfig	The accelerometer profile to use for measurements.	Filepath
altimeterConfig	The altimeter profile to use for measurements.	Filepath
clockConfig	The clock profile to use for measurements.	Filepath

Estimation Settings

The **Estimation** section contains values relating to estimation methods that will be used for predicting the vehicle's current position, orientation, velocity and other factors.

Table 9: Key Estimation Section Variables

Variable Name	Description	Type
integrationMethod	The navigation integration method to use.	Integration ID
accelerometerConfig	The accelerometer profile to use for estimating errors.	Filepath
gyroscopeConfig	The gyroscope profile to use for estimating errors.	Filepath
altimeterConfig	The altimeter profile to use for estimating errors.	Filepath
clockConfig	The clock profile to use for estimating errors.	Filepath
gpsFusionMethod	The GPS fusion method to use for processing GPS data.	GPS Method ID
useVirtualMemory	Should disk space be used to hold estimation results?	Yes <i>or</i> No

Note

The following INS integration methods are supported:

Integration Methods:

Numerical, Kalman, Runge_Kutta, Adams_Bashforth

Altimeter Settings

The **Altimeter** section controls all altimeter related features. This include the altimeter frequency, error profile and fusion method.

Table 10: Key Altimeter Section Variables

Variable Name	Description	Type
altimeterEnabled	Should the altimeter be used in the simulation?	Yes <i>or</i> No
altimeterFusion	Should the altimeter be used in the simulation?	See Below
altimeterGainAmount	The fixed gain amount to use for fixed-gain filtering.	Float
altimeterAlphaAmount	The alpha value to use for alpha-beta filtering.	Float
altimeterBetaAmount	The beta value to use for alpha-beta filtering.	Float

Note

The following altimeter fusion methods are supported:

Integration Methods:

fixed_gain or alpha_beta

GPS Settings

The **GPS** section controls all GPS related features. This includes if GPS is to be used in a simulation, the GPS satellite data to use and areas where GPS signals are available.

Table 11: Key GPS Section Variables

Variable Name	Description	Type
gpsEnabled	Should the GPS be used in the simulation?	Yes <i>or</i> No
gpsFusionMethod	The GPS fusion method to use	See Below
gpsTimeYear	The 4-digit year to use for the satellite information.	Integer
gpsTimeMonth	The 2-digit month to use for the satellite information.	Integer
gpsTimeYear	The 2-digit day to use for the satellite information.	Integer
gpsAutoDownload	Can GPS information be automatically downloaded?	Yes <i>or</i> No

Note

The following GPS fusion methods are supported:

GPS Fusion Methods:

Fixed, Loose, Tight

Quantum Settings

The **Quantum** section controls all Quantum Sensor related features. This covers both the cold-atom quantum inertia and gravity gradient sensors and fusion methods.

Table 12: Key Quantum Section Variables

Variable Name	Description	Type
quantumImuEnabled	Should the quantum IMU be used in the simulation?	Yes <i>or</i> No
quantumImuSensor	The type of quantum inertia sensor to use.	concept <i>or</i> realistic
quantumImuFusion	The fusion method to use for the quantum inertia sensor.	basic, particle_filter <i>or</i> misaligned
quantumImuConfig	The quantum IMU sub-configuration file.	Filepath
quantumGravityEnabled	Should the quantum gravity map-matcher be used?	Yes <i>or</i> No
quantumGravitySensor	The type of quantum gravity sensor to use.	dual_interferometer
quantumGravityFusion	The fusion method to use for the quantum gravity map-matching.	particle_filter
quantumGravityConfig	The quantum gravity map-matching sub-configuration file.	Filepath

Holonomic Settings

The **Holonomics** section controls the optional use of holonomic constraints for preventing numerical sensitivities impacting simulation accuracy. Essentially these are included to combat unwanted numerical rounding errors in simulations.

Table 13: Key Holonomics Section Variables

Variable Name	Description	Type
useHolonomics	Should holonomic constraints be used?	Yes <i>or</i> No
correctionFreq	The update rate at which corrections are applied (in Hz).	Float
positionGain	Holonomic gain for position.	Float
positionAxis	Holonomic axis percentages for position.	CSV (3-values)
velocityGain	Holonomic gain for velocity.	Float
velocityAxis	Holonomic axis percentages for position.	CSV (3-values)
accelerationGain	Holonomic gain for acceleration.	Float
accelerationAxis	Holonomic axis percentages for acceleration.	CSV (3-values)
attitudeGain	Holonomic gain for attitude.	Float
attitudeAxis	Holonomic axis percentages for attitude.	CSV (3-values)
angleRateGain	Holonomic gain for angle rates.	Float
angleRateAxis	Holonomic axis percentages for angle rates.	CSV (3-values)

Random Settings

The **Random** section controls the generation of all random numbers within waypoint generation, object initialisation and during simulations. Essentially this section allows random aspects such as noise, drift, sensor errors etc. to be managed via random seeds. If the same random seed is reused, the randomness will always be the same.

Table 14: Key Random Section Variables

Variable Name	Description	Type
initialRandomSeed	The random seed to use during initialisation.	Integer
autoGenerateInitialSeed	Should the value be autogenerated?	Yes <i>or</i> No
waypointRandomSeed	The random seed to use during waypoint creation.	Integer
autoGenerateWaypointSeed	Should the value be autogenerated?	Yes <i>or</i> No
simulationRandomSeed	The random seed to use during simulations.	Integer
autoGenerateSimulationSeed	Should the value be autogenerated?	Yes <i>or</i> No

Misc Settings

The **Misc** section contains the remaining miscellaneous settings that do not fit into any of the above sections. These namely control the extraction process of values from configuration files and whether any automatic behaviour is to be displayed.

Table 15: Key Misc Section Variables

Variable Name	Description	Type
allowConfigOverloading	Should sub-config values overload those in the main config?	Yes <i>or</i> No
displayFallbackWarnings	Should warnings about missing variables be displayed?	Yes <i>or</i> No

1.8.2 Sub-configuration Files

To reduce the complexity of the configuration file and to allow for a more modular design, some sections will reference sub-configuration files which contain variables to use. For example, the variable `vehicleConfig` points to the vehicle sub-configuration file to use, allowing pre-created profiles to be re-used. For instance, `vehicleConfig` can be used to load values belonging to different vehicle models. This has many advantages including allowing the changing of multiple parameters by only changing a single value.

When a sub-configuration file is loaded, the values extracted from it will be overwritten by pre-existing values in the main configuration file. For instance, if `vehicleName` is set to “**Cargo Plane**” in the main configuration file, but is set to “**Commercial Plane**” in the vehicle sub-configuration file, “**Cargo Plane**” will be used.

Note

This overwriting priority behaviour can be altered by changing the value of `allowConfigOverloading` (set to false by default) located in the `Misc` section.

1.9 Software Components

The functionality of the produced software has been logically categorised into Python sub-packages. This allows means for grouping related elements together and organising the project into manageable components (rather than large amount of files and resources). The modules belonging to each sub-package of the project have been outlined below.

1.9.1 Earth

Modules relating to core calculations involving the earth:

`qnav.util.transformations`

`transformations.py`

summary

This module includes all general transformation calculations that are required for the Navigation Simulation Study. The output from the presented functions has been compared with other implementations such as Python’s NavPy and MATLAB’s mapping toolbox.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of this module.

`qnav.util.transformations.deg2rad(theta_deg: float) → float`

A wrapper function for converting from degrees to radians.

Parameters

`theta_deg (float)` – An input value in degrees.

Returns

The converted input value in radians.

Return type

float

`qnav.util.transformations.rad2deg(theta_rad: float) → float`

A wrapper function for converting from radians to degrees.

Parameters

theta_rad (*float*) – An input value in radians.

Returns

The converted input value in degrees.

Return type

float

`qnav.util.transformations.deg2dec(degrees: float, minutes: float, seconds: float) → float`

Converts a degrees, minutes, seconds coordinate to decimal format.

Parameters

- **degrees** (*float*) – The degrees of the given coordinate.
- **minutes** (*float*) – The minutes of the given coordinate.
- **seconds** (*float*) – The seconds of the given coordinate.

Returns

The decimal equivalent of the given coordinate.

Return type

float

`qnav.util.transformations.dec2deg(theta_deg: float) → tuple[float, float, float]`

Converts a decimal coordinate to degrees, minutes, seconds format.

Parameters

theta_deg (*float or NumPy Array*) – The decimal format of the given coordinate.

Returns

The degrees, minutes and seconds of the given coordinate.

Return type

float or NumPy Array

`qnav.util.transformations.radius(lat: float) → float`

This function calculates the radius (in metres) from the centre of the Earth for a given latitude position, using WGS84 parameters.

Parameters

lat (*float*) – The latitude of the points of interest.

Returns

The calculated radius in metres for the given position.

Return type

float

`qnav.util.transformations.radius_vec(lat: ndarray) → ndarray`

The vectorised version of the `radius()` function. This calculates the radius (in metres) from the centre of the Earth for a large series of given latitude positions, using WGS84 parameters.

Parameters

lat (*numpy.ndarray*) – The latitude of the points of interest.

Returns

The calculated radius in metres for the given position.

Return type

numpy.ndarray

`qnav.util.transformations.rotation_rate(lat: float) → ndarray`

Returns the rotation rate of earth at given latitude position. Calculates the rotation rate vector of the earth the given latitude.

Parameters

lat (*float*) – The latitude position (in decimal degrees).

Returns

The rotation vector in degrees/second.

Return type

np.ndarray

`qnav.util.transformations.rotation_rate_vec(lat: ndarray) → ndarray`

Returns the rotation rate of earth at given latitude position. Calculates the rotation rate vector of the earth the given latitude.

Parameters

lat (*np.ndarray*) – The latitude position (in decimal degrees).

Returns

The rotation vector in degrees/second.

Return type

np.ndarray

`qnav.util.transformations.radius84(lat: float) → float`

This function calculates the radius (in metres) from the centre of the Earth for a given latitude position, using WGS84 parameters.

Parameters

lat (*float*) – The latitude of the points of interest.

Returns

The calculated radius in metres for the given position.

Return type

float

`qnav.util.transformations.radius84_vec(lat: ndarray) → ndarray`

The vectorised version of the [radius84\(\)](#) function. This calculates the radius (in metres) from the centre of the Earth for a large series of given latitude positions, using WGS84 parameters.

Parameters

lat (*numpy.ndarray*) – The latitude of the points of interest.

Returns

The calculated radius in metres for the given position.

Return type

numpy.ndarray

`qnav.util.transformations.rotate_3d(psi: float, theta: float, phi: float) → ndarray`

This function produces a three-dimensional rotation matrix corresponding to the Euler angles psi (Heading), theta (Pitch), phi (Roll/Bank).

Parameters

- **psi** (*float*) – The heading angle in radians.
- **theta** (*float*) – The pitch angle in radians.
- **phi** (*float*) – The roll/bank angle in radians.

Returns

A matrix corresponding to three rotations in the above order.

Return type

numpy.ndarray (3-by-3 elements)

qnav.util.transformations.**rotate_3d_vec**(psi: ndarray, theta: ndarray, phi: ndarray) → ndarray

This function is the vectorised version of the [rotate_3d\(\)](#) function. This produces multiple three-dimensional rotation matrices corresponding to the Euler angles psi (Heading), theta (Pitch), phi (Roll/Bank).

Parameters

- **psi** (*numpy.ndarray*) – The heading angles in radians.
- **theta** (*numpy.ndarray*) – The pitch angles in radians.
- **phi** (*numpy.ndarray*) – The roll/bank angles in radians.

Returns

The matrices corresponding to three rotations in the above order.

Return type

numpy.ndarray (n-3-by-3 elements)

qnav.util.transformations.**lla2ned**(lla: Buffer | _SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float |
complex | str | bytes | _NestedSequence[bool | int | float | complex | str |
bytes], ref_lla: Buffer | _SupportsArray[dtype[Any]] |
_NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float |
complex | str | bytes | _NestedSequence[bool | int | float | complex | str |
bytes]) → ndarray

This function converts a reference Latitude-Longitude-Altitude (LLA) position from a given centre LLA point and translates its location to North-East-Down (NED) coordinates.

Parameters

- **lla** (*numpy.ndarray (3-elements)*) – The centre location (array containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.
- **ref_lla** (*numpy.ndarray (3-elements)*) – The reference location (array containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

An array containing the calculated North, East and Down coordinates from the centre position, were all values are given in metres.

Return type

numpy.ndarray (3-elements)

qnav.util.transformations.**lla2ned_vec**(lla: ndarray, ref_lla: ndarray) → ndarray

This function is the vectorised version of the [lla2ned\(\)](#) function. This allows means for converting reference Latitude-Longitude-Altitude (LLA) positions from given centre LLA points and translates their location to North-East-Down (NED) coordinates.

Parameters

- **lla** (*numpy.ndarray (n-by-3 elements)*) – The centre locations (array rows containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.
- **ref_lla** (*numpy.ndarray (n-by-3 elements)*) – The reference location (array rows containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

An array containing the calculated North, East and Down coordinates from the centre position, where all values are given in metres.

Return type

numpy.ndarray (n-by-3 elements)

`qnav.util.transformations.ned2lla(ned: ndarray, ref_lla: ndarray) → ndarray`

This function converts a reference North-East-Down (NED) position from a given centre position and translates its location to Latitude-Longitude-Altitude (LLA) coordinates.

Parameters

- **ned** (*numpy.ndarray (3-elements)*) – The centre location (array containing [north, east, down]) with all values being given in metres.
- **ref_lla** (*numpy.ndarray (3-elements)*) – The reference location (array containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

An array containing the calculated Latitude, Longitude and Altitude coordinates from the centre position, with lat-lon is given in degrees and alt given in metres.

Return type

numpy.ndarray (3-elements)

`qnav.util.transformations.ned2lla_vec(ned: ndarray, ref_lla: ndarray) → ndarray`

This function is the vectorised version of the `ned2lla()` function. This converts reference North-East-Down (NED) positions from a given centre positions and translates their location to Latitude-Longitude-Altitude (LLA) coordinates.

Parameters

- **ned** (*numpy.ndarray (n-by-3 elements)*) – The centre locations (array rows containing [north, east, down]) with all values being given in metres.
- **ref_lla** (*numpy.ndarray (n-by-3 elements)*) – The reference location (array rows containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

Array rows containing the calculated Latitude, Longitude and Altitude coordinates from the centre positions, with lat-lon is given in degrees and alt given in metres.

Return type

numpy.ndarray (n-by-3 elements)

`qnav.util.transformations.lla2ecef(lla: ndarray) → ndarray`

This function converts a reference Latitude-Longitude-Altitude (LLA) position to Earth-Centred Earth-Fixed coordinates.

Parameters

- **lla** (*numpy.ndarray (3-elements)*) – The reference location (array containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

An array containing the calculated X, Y and Z coordinates from the centre of the Earth, with all values being given in metres.

Return type

numpy.ndarray (3-elements)

`qnav.util.transformations.lla2ecef_vec(lla: ndarray) → ndarray`

This function is the vectorised version of the `lla2ecef()` function. This converts a series of reference Latitude-Longitude-Altitude (LLA) positions to Earth-Centred Earth-Fixed coordinates.

Parameters

lla (*numpy.ndarray (n-by-3 elements)*) – The reference locations (array rows containing [lat, lon, alt]) with lat-lon given in degrees and alt given in metres.

Returns

An array containing the calculated X, Y and Z coordinates from the centre of the Earth, with all values being given in metres.

Return type

numpy.ndarray (n-by-3 elements)

`qnav.util.transformations.euler_to_quaternion(angles: ndarray) → ndarray`

Converts given attitude angles to quaternion format. Given an array representing euler angles, this function converts them to quaternion format. On succession, an array containing the a, b, c and d quaternion coefficients (i.e. where $a + bi + cj + dk$) will be returned.

Parameters

angles (*numpy.ndarray (3-elements)*) – Euler angles given in degrees

Returns

The a, b, c, and d quaternion coefficients in columns.

Return type

numpy.ndarray (4-elements)

`qnav.util.transformations.quaternion_to_euler(quat: ndarray) → ndarray`

Converts given quaternions to Euler angle format. Given an array representing coefficients for quaternions (i.e. values a, b, c and d where $a + bi + cj + dk$), this function converts them to euler angle format. On succession, an array containing the Euler angles will be returned.

Parameters

quat (*numpy.ndarray (4-elements)*) – The a, b, c, and d quaternion coefficients in columns.

Returns

The converted Euler angles given in degrees.

Return type

numpy.ndarray (3-elements)

`qnav.util.transformations.euler_to_quaternion_vec(angles: ndarray) → ndarray`

Converts given attitude angles to quaternion format. Given an array representing euler angles, this function converts them to quaternion format. On succession, an array containing the a, b, c and d quaternion coefficients (i.e. where $a + bi + cj + dk$) will be returned.

Parameters

angles (*numpy.ndarray (n-by-3 elements)*) – Euler angles given in degrees

Returns

The a, b, c, and d quaternion coefficients in columns.

Return type

numpy.ndarray (n-by-4 elements)

qnav.util.transformations.**quaternion_to_euler_vec**(*quat: ndarray*) → ndarray

Converts given quaternions to Euler angle format. Given an array representing coefficients for quaternions (i.e. values a, b, c and d where $a + bi + cj + dk$), this function converts them to euler angle format. On succession, an array containing the Euler angles will be returned.

Parameters

quat (*numpy.ndarray (n-by-4 elements)*) – The a, b, c, and d quaternion coefficients in columns.

Returns

The converted Euler angles given in degrees.

Return type

numpy.ndarray (n-by-3 elements)

qnav.util.transformations.**haversine**(*lat1: float, lon1: float, alt1: float, lat2: float, lon2: float, alt2: float*) → float

Returns the estimated distance in metres between two sets of latitude-longitude-altitude positions. This function uses the Haversine formula to calculate the distance between latitude-longitude points, then employs pythagoras to estimate the actual distance by considering change in altitude between positions.

Parameters

- **lat1** (*numpy.ndarray (n-elements)*) – The latitude position(s) belonging to the first set.
- **lon1** (*numpy.ndarray (n-elements)*) – The longitude position(s) belonging to the first set.
- **alt1** (*numpy.ndarray (n-elements)*) – The altitude position(s) belonging to the first set.
- **lat2** (*numpy.ndarray (n-elements)*) – The latitude position(s) belonging to the second set.
- **lon2** (*numpy.ndarray (n-elements)*) – The longitude position(s) belonging to the second set.
- **alt2** (*numpy.ndarray (n-elements)*) – The altitude position(s) belonging to the second set.

Returns

The distance between the given positions in metres.

Return type

numpy.array (n-elements)

```

qnav.util.transformations.haversine_vec(lat1: Buffer | _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes], lon1: Buffer |
    _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes], alt1: Buffer | _SupportsArray[dtype[Any]]
    | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes], lat2: Buffer | _SupportsArray[dtype[Any]]
    | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes], lon2: Buffer |
    _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes], alt2: Buffer | _SupportsArray[dtype[Any]]
    | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
    float | complex | str | bytes | _NestedSequence[bool | int | float |
    complex | str | bytes]) → ndarray

```

Returns the estimated distance in metres between two sets of latitude-longitude-altitude positions. This function uses the Haversine formula to calculate the distance between latitude-longitude points, then employs pythagoras to estimate the actual distance by considering change in altitude between positions.

Parameters

- **lat1** (*numpy.ndarray* (*n-elements*)) – The latitude position(s) belonging to the first set.
- **lon1** (*numpy.ndarray* (*n-elements*)) – The longitude position(s) belonging to the first set.
- **alt1** (*numpy.ndarray* (*n-elements*)) – The altitude position(s) belonging to the first set.
- **lat2** (*numpy.ndarray* (*n-elements*)) – The latitude position(s) belonging to the second set.
- **lon2** (*numpy.ndarray* (*n-elements*)) – The longitude position(s) belonging to the second set.
- **alt2** (*numpy.ndarray* (*n-elements*)) – The altitude position(s) belonging to the second set.

Returns

The distance between the given positions in metres.

Return type

numpy.array (*n-elements*)

```

qnav.util.transformations.get_time_pos_vel(time: datetime) → tuple[ndarray, ndarray]

```

This function calculates the position and velocity matrices for a given timestamp.

Parameters

time (*Python datetime instance*) – A datetime for the requested time.

Returns

The position and velocity matrices.

Return type

tuple (mat_pos, mat_vel)

```
qnav.util.transformations.ecef2eci(time: datetime, r_ecef: ndarray, v_ecef: ndarray) → tuple[ndarray, ndarray]
```

This function converts a position and velocity from the ECI frame to the ECEF frame.

Parameters

- **time** (*Python datetime instance*) – A datetime for the requested time.
- **r_ecef** (*numpy.ndarray (3-elements)*) – Position in ECI (given in metres).
- **v_ecef** – Velocity in ECI (given in metres/second).

Returns

The position and velocity vectors in ECEF. Position is given in metres and velocity is given in metres/second.

Return type

tuple (r_eci, v_eci)

```
qnav.util.transformations.eci2ecef(time: datetime, r_eci: ndarray, v_eci: ndarray) → tuple[ndarray, ndarray]
```

This function converts a position and velocity from the ECEF frame to the ECI frame.

Parameters

- **time** (*Python datetime instance*) – A datetime for the requested time.
- **r_eci** (*numpy.ndarray (3-elements)*) – Position in ECEF (given in metres).
- **v_eci** (*numpy.ndarray (3-elements)*) – Velocity in ECEF (given in metres/second).

Returns

The position and velocity vectors in ECI. Position is given in metres and velocity is given in metres/second.

Return type

tuple (r_ecef, v_ecef)

```
qnav.util.transformations.ecef2lla(xyz: ndarray) → ndarray
```

Converts the given XYZ Earth Centred Earth Fixed coordinates to longitude latitude altitude coordinates.

Parameters

xyz (*numpy.ndarray (3-elements)*) – A position in ECEF (m-m-m)

Returns

The given position converted to LLA (deg-deg-m).

Return type

numpy.ndarray (3-elements)

```
qnav.util.transformations.ecef2lla_vec(xyz: ndarray) → ndarray
```

This function is the vectorised version of the [ecef2lla\(\)](#) function. This converts the given XYZ Earth Centred Earth Fixed coordinates to longitude latitude altitude coordinates.

Parameters

xyz (*numpy.ndarray (n-by-3 elements)*) – A series of positions in ECEF (m-m-m)

Returns

The given positions converted to LLA (deg-deg-m).

Return type

numpy.ndarray (n-by-3 elements)

qnav.util.transformations.**ecef2ned_transform**(ref_lla: ndarray) → ndarray

This function generates a transformation matrix for ECEF to North-East-Down (NED) for an input Latitude-Longitude-Altitude (LLA) reference.

Parameters

ref_lla (numpy.ndarray (3-elements)) – The reference position in LLA, with lat-lon given in degrees and alt given in metres.

Returns

The calculated 3-by-3 transformation matrix.

Return type

NumPy Matrix (3-by-3 elements)

qnav.util.transformations.**ecef2ned_transform_vec**(ref_lla: ndarray) → ndarray

This function is the vectorised version of the [ecef2ned_transform\(\)](#) function. This generates transformation matrices for ECEF to North-East-Down (NED) for input Latitude-Longitude-Altitude (LLA) references.

Parameters

ref_lla (numpy.ndarray (n-by-3 elements)) – The reference position in LLAs, with lat-lon given in degrees and alt given in metres.

Returns

The calculated n-by-3-by-3 transformation matrix.

Return type

numpy.ndarray (n-3-3 elements)

qnav.util.transformations.**ecef2ned**(xyz: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes], ref_lla: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes]) → ndarray

This function takes an ECEF position and a reference Latitude-Longitude-Altitude (LLA) position and returns the relative position of the former from the latter in North-East-Down (NED) reference frame.

Parameters

- **xyz** (numpy.ndarray (3-elements)) – Position in ECEF, given in metres.
- **ref_lla** (numpy.ndarray (3-elements)) – The reference LLA position, with lat-lon given in degrees and alt given in metres.

Returns

The given position converted to NED, given in metres.

Return type

numpy.ndarray (3-elements)

qnav.util.transformations.**ned2ecef**(ned: ndarray, ref_lla: ndarray) → ndarray

This function takes an origin Latitude-Longitude-Altitude (LLA) position and a relative position in North-East-Down (NED) from this origin and returns the latter position in ECEF.

Parameters

- **ned** (numpy.ndarray (3-elements)) – Position in NED, given in metres.

- **ref_lla** (*numpy.ndarray (3-elements)*) – The reference LLA position, with lat-lon given in degrees and alt given in metres.

Returns

The given position converted to ECEF, given in metres.

Return type

numpy.ndarray (3-elements)

`qnav.util.transformations.ned2ecef_vec(ned: ndarray, ref_lla: ndarray) → ndarray`

This function takes a North-East-Down (NED) position along with its relative Latitude-Longitude-Altitude (LLA) reference position and returns the latter position in ECEF.

Parameters

- **ned** (*numpy.ndarray (n-by-3 elements)*) – Positions in NED, given in metres.
- **ref_lla** (*numpy.ndarray (n-by-3 elements)*) – The reference LLA positions, with lat-lon given in degrees and alt given in metres.

Returns

The given position converted to ECEF, given in metres.

Return type

numpy.ndarray (n-by-3 elements)

`qnav.util.transformations.vned2vecef(vel_ned: ndarray, pos_lla: ndarray) → ndarray`

This function takes an origin Latitude-Longitude-Altitude (LLA) position and a relative velocity in North-East-Down (NED) components from this origin and returns the velocity in ECEF format.

Parameters

- **vel_ned** (*numpy.ndarray (3-elements)*) – Velocity in NED, given in metres/second.
- **pos_lla** (*numpy.ndarray (3-elements)*) – The reference LLA position, with lat-lon given in degrees and alt given in metres.

Returns

The given velocity vector converted to ECEF, given in metres/sec.

Return type

NumPy Matrix (3-by-3 elements)

`qnav.util.transformations.utc2gps(time: datetime) → dict`

This function converts a provided time given in Coordinated Universal Time (UTC) to a Python dictionary containing the corresponding week number, day number and second.

Parameters

time (*Python Datetime instance*) – A requested time in Coordinated Universal Time (UTC) format.

Returns

The provided time as a Python dictionary that can be used for GPS functions.

Return type

dict

`qnav.util.transformations.ned2azel(ned: ndarray) → tuple[float, float]`

This function takes an origin North-East-Down (NED) position and converts it into an azimuth and elevation value.

Parameters

ned (*numpy.ndarray (3-elements)*) – The centre location (array containing [north, east, down]) with all values being given in metres.

Returns

A tuple containing two elements, the corresponding azimuth and elevation in radians.

Return type

tuple[float, float]

`qnav.util.transformations.get_transport_rate(position_lla: ndarray, velocity_ned: ndarray) → ndarray`

This function computes the transport rate (in Local NED axes) for given position and velocity vectors.

Parameters

- **position_lla** (*numpy.ndarray (3-elements)*) – The current position vector stating the latitude, longitude and altitude (degrees-degrees-metres).
- **velocity_ned** (*numpy.ndarray (3-elements)*) – The current velocity vector stating the velocity North, East and Down (given in metres/second).

Returns

The transport rate for the given position and velocity.

Return type

NumPy Matrix (3-by-3 elements)

`qnav.util.transformations.get_transport_rate_vec(position_lla: ndarray, velocity_ned: ndarray) → ndarray`

This function is the vectorised version of the [get_transport_rate\(\)](#) function. This computes the transport rate (in Local NED axes) for a series of given positions and velocity vectors.

Parameters

- **position_lla** (*numpy.ndarray (n-by-3 elements)*) – The current position vectors stating the latitude, longitude and altitude (degrees-degrees-metres).
- **velocity_ned** (*numpy.ndarray (n-by-3 elements)*) – The current velocity vectors stating the velocity North, East and Down (given in metres/second).

Returns

The transport rate for the given positions and velocities.

Return type

NumPy Matrix (n-3-3 elements)

`qnav.util.transformations.vel_ecef2vel_ned(vel_ecef: ndarray, lla: ndarray) → ndarray`

A function to take ECEF velocity and an LLA location and convert it into the NED velocity.

Parameters

- **vel_ecef** (*numpy.ndarray (3-elements)*) – The velocity given in Earth-Centred Earth-Fixed format.
- **lla** – The current latitude, longitude and altitude (in deg-deg-m).

Returns

The given Earth-Centred Earth-Fixed (ECEF) velocity converted into local North-East-Down (NED), subject to the given reference position.

Return type

numpy.ndarray (3-elements)

`qnav.util.transformations.cross_prod_xy(vec1: ndarray, vec2: ndarray) → ndarray`

An optimised method for computing the cross-product of two 1-by-3 arrays. In most cases this method is far quicker than the Numpy cross function.

Parameters

- **vec1** (*numpy.ndarray (3 elements)*) – The first 1-by-3 tuple/array.
- **vec2** (*numpy.ndarray (3 elements)*) – The first 1-by-3 tuple/array.

Returns

The cross product of the two tuples.

Return type

numpy.ndarray (3 elements)

qnav.earth.dted

dted.py

summary

A module for managing Digital Terrain Elevation Data (DTED). This is responsible for the reading and dynamic loading of DTED files for extracting and interpolating elevation from a locally stored Digital Elevation Map (DEM). This allows the extraction of the terrain height above the WSG84 ellipsoid, excluding man-made features. This is namely used for waypoint/trajectory generation of man-made features.

authors

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of this module.

class `qnav.earth.dted.DtedTile(origin_lat: int, origin_lon: int, dted_dir: Path, dted_version: int = 2, interp_method: str = 'linear', is_empty_on_error: bool = True)`

Holds and handles the read content for a single DTED file. Manages the data contained in a DTED tile (1-degree by 1-degree region). This allows local interpolation and other relevant calculations. Instances of this are used by `DtedHandler` for dynamic loading and management of terrain information.

get_elevation(*lat: float, lon: float, method: str = None*) → *float*

Interpolates the terrain elevation for the requested position. Given a latitude and longitude, the interpolated terrain elevation will be returned (in metres). Optionally, an overriding interpolation method to use can be provided.

Parameters

- **lat** (*float*) – The latitude coordinate of the reference position.
- **lon** (*float*) – The longitude coordinate of the reference position.
- **method** (*str*) – An alternative interpolation method to use. Supported values are: 'linear', 'nearest', 'slinear', 'cubic', 'quintic' and 'pchip'.

Returns

The interpolated terrain elevation (in metres)

Return type

float


```
get_elevation_vec(lat_points: Buffer | _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes
    | _NestedSequence[bool | int | float | complex | str | bytes], lon_points: Buffer |
    _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool |
    int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str |
    bytes], method: str = None) → ndarray
```

Interpolates the terrain elevation for the requested positions. Given latitude and longitude coordinates, the interpolated terrain elevations will be return (in metres). Optionally, an overriding interpolation method to use can be provided.

Parameters

- **lat_points** (*npt.ArrayLike*) – The latitude coordinates of the reference positions.
- **lon_points** (*npt.ArrayLike*) – The longitude coordinates of the reference positions.
- **method** (*str*) – An alternative interpolation method to use. Supported values are: 'linear', 'nearest', 'slinear', 'cubic', 'quintic' and 'pchip'.

Returns

The interpolated terrain elevations (in metres)

Return type

npt.ArrayLike

```
is_position_covered(lat: float, lon: float) → bool
```

A method for checking if a requested point is covered by the tile. Performs a check to see if the position is encompassed by the tile, allowing its terrain heights to be interpolated.

Parameters

- **lat** (*float*) – The latitude coordinate of the reference position.
- **lon** (*float*) – The longitude coordinate of the reference position.

Returns

True if the point is covered by the tile.

Return type

bool

```
is_position_covered_vec(lat: Buffer | _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex |
    str | bytes | _NestedSequence[bool | int | float | complex | str | bytes], lon:
    Buffer | _SupportsArray[dtype[Any]] |
    _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex |
    str | bytes | _NestedSequence[bool | int | float | complex | str | bytes]) → ndarray
```

A vectorised method for checking if points are covered by the tile. Performs a check to see which positions are encompassed by the tile, supporting their terrain heights to be interpolated.

Parameters

- **lat** (*npt.ArrayLike*) – The latitude coordinates of the reference positions.
- **lon** (*npt.ArrayLike*) – The longitude coordinates of the reference positions.

Returns

A boolean array indicating which points are covered by the tile. True is used for points inside the tile.

Return type

np.ndarray.

distance_to_centre(*lat: float, lon: float*) → float

Returns approximation to distance from centre of tile (in metres). Calculates the approximate to distance from centre of tile for the given position. Namely used to identify the most distant tile.

Parameters

- **lat** (*float*) – The latitude coordinate of the reference position.
- **lon** (*float*) – The longitude coordinate of the reference position.

Returns

The distance from the centre of the tile (in metres).

Return type

float

class qnav.earth.dted.DtedHandler(*dted_dir: Path, dted_version: int = 2, max_tiles: int = None, method: str = None*)

Manages the dynamic loading and handling of regions of DTED data. This class is use dynamically load DTED data for extracting elevation. This automatically determines the required DTED files and loads into memory. Constraints on the maximum amount of data to load can be set to conserve resources.

get_elevation(*lat: float, lon: float, method: str = None*) → float

Interpolates the terrain elevation for the requested position. Performed interpolation of the terrain elevation for the given latitude and longitude location. Optionally an overriding interpolation method can be used.

Parameters

- **lat** (*float*) – The latitude coordinates of the reference positions.
- **lon** (*float*) – The longitude coordinates of the reference positions.
- **method** (*str*) – The overriding interpolation method to use. Supported methods are: 'linear', 'nearest', 'slinear', 'cubic', 'quintic' and 'pchip'.

Returns

The interpolated terrain elevation (in metres).

Return type

float

get_elevation_vec(*lat_points: ndarray, lon_points: ndarray, method: str = None*) → ndarray

Interpolates the terrain elevation at given positions. Performed vectorised interpolation of the terrain elevation for the given latitude and longitude locations. Optionally an overriding interpolation method can be used.

Parameters

- **lat_points** (*np.ndarray*) – The latitude coordinates of the reference positions.
- **lon_points** (*np.ndarray*) – The longitude coordinates of the reference positions.
- **method** (*str*) – The overriding interpolation method to use. Supported methods are: 'linear', 'nearest', 'slinear', 'cubic', 'quintic' and 'pchip'.

Returns

The interpolated terrain elevation (in metres).

Return type

np.ndarray

set_max_tiles(*value: int*)

A setter method for the maximum number of tiles. Can use the value None to remove limit on number of tiles to hold.

Parameters

value (*int*) – The maximum number of DTED tiles to hold in the cache.

clear_cache()

Clears all cached DTED from memory. Simply removes all DTED tiles from the cache.

qnav.earth.dted.get_dted_file_path(*dted_dir: Path, origin_lat: int, origin_lon: int, dted_version: int*) → *Path*

Give a latitude, longitude and optional DTED version number, this function generates the file path of the DTED file that would contain information for the given position.

Parameters

- **dted_dir** (*Path*) – The parent database_dir of the DTED files.
- **origin_lat** (*int*) – The integer latitude value (south-most coordinate).
- **origin_lon** (*int*) – The integer longitude value (west-most coordinate).
- **dted_version** (*int*) – The version of the DTED data.

:return the location of a corresponding DTED file. :rtype: Path

qnav.earth.dted.read_dted_file(*file_dir: Path*) → dict

Extracts elevation information from a provided DTED file (in metres). When provided with the location of a DTED file (of any level), this function will return an elevation matrix and latitude-longitude spacing of each point. This function reads DTED files of format version 1.1.

Parameters

file_dir (*Path*) – The location of the DTED file to be read.

Returns

A dictionary containing the contents of the DTED file. This contains the extracted origin coordinates, step sizes, coordinate ticks and elevation matrix.

Return type

dict

qnav.earth.dted.get_template_dted_data(*origin_lat: int, origin_lon: int, num_lat_points: int = 2, num_lon_points: int = 2*) → dict

Generates a template DTED data dictionary as if read from a file. Simulates the output of `read_dted_file()` by generating identical output structure with flat elevation. This is to be used as substitution for missing/unavailable DTED files.

Parameters

- **origin_lat** (*int*) – The integer latitude value (south-most coordinate).
- **origin_lon** (*int*) – The integer longitude value (west-most coordinate).
- **num_lat_points** (*int*) – The number of latitude points (rows) to generate.
- **num_lon_points** (*int*) – The number of longitude points (columns) to generate.

Returns

A dictionary containing contents of a DTED file. This contains the imitated origin coordinates, step sizes, coordinate ticks and elevation matrix that would be read.

Return type

dict

qnav.earth.geoid**geoid.py****summary**

A module for interpolating geoid heights from presented geoid PGM files. This module provides classes and functions used to extract geoid information from given PGM encoded geoid files. The most common use for this module is for interpolating the geoid height at requested positions.

authors

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of this module.

class qnav.earth.geoid.GeoidModel

Abstract base class for geoid representations. Defines the method that implemented geoid representations must supply.

abstract get_height(*lat: float, lon: float*) → float

Returns geoid height for single latitude and longitude position.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.

Returns

The geoid height for the corresponding location (in metres).

Return type

float

abstract get_height_vec(*lat: ndarray, lon: ndarray*) → ndarray

Returns geoid height for given latitude and longitude positions.

Parameters

- **lat** (*float*) – The latitude coordinates in decimal degrees.
- **lon** (*float*) – The longitude coordinates in decimal degrees.

Returns

The geoid height for the corresponding locations (in metres).

Return type

float

abstract get_dov(*lat: float, lon: float*) → tuple[float, float]

Returns the Deflection of the Vertical (DoV) for requested position. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

abstract get_dov_vec(*lat: ndarray, lon: ndarray*) → tuple[ndarray, ndarray]

Returns the Deflection of the Vertical (DoV) for requested positions. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinates in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinates in decimal degrees.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

class qnav.earth.geoid.**GeoidPGM**(*pgm_file: Path*)

A class for handling and managing reading the Geoid height information from a selected PGM (Pixel Gray Map) file and providing functionality for extracting required information from it. Methods for reading or interpolating the Geoid height at given coordinates, as well as visualising the loaded data are all contained within this class.

get_height(*lat: float, lon: float*) → float

Interpolates geoid height for given latitude and longitude. For a single coordinate point, interpolates and returns the geoid elevation height in metres.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.

Returns

The geoid height for the corresponding location (in metres).

Return type

float

get_height_vec(*lat: ndarray, lon: ndarray*) → ndarray

Interpolates geoid height for given latitude and longitude points. For a single coordinate point, interpolates and returns the geoid elevation height in metres.

Parameters

- **lat** (*float*) – The latitude coordinates in decimal degrees.
- **lon** (*float*) – The longitude coordinates in decimal degrees.

Returns

The geoid height for the corresponding locations (in metres).

Return type

float

get_dov(*lat: float, lon: float, step_size: float = 100*) → tuple[float, float]

Returns the Deflection of the Vertical (DoV) for requested position. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.
- **step_size** (*float*) – The step size for calculating the gradient.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

get_dov_vec(*lat: ndarray, lon: ndarray, step_size: float = 100*) → tuple[ndarray, ndarray]

Returns the Deflection of the Vertical (DoV) for requested position. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinates in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinates in decimal degrees.
- **step_size** (*float*) – The step size for calculating the gradient.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

property resolution: tuple[int, int]

The north and east resolutions of the grid data. Can be used for determining the accuracy of the model.

Returns

The north and east steps of the grid data.

Return type

tuple[int, int]

class qnav.earth.geoid.**GeoidDummy**

get_height(*lat: float, lon: float*) → float

Returns geoid height for single latitude and longitude position.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.

Returns

The geoid height for the corresponding location (in metres).

Return type

float

get_height_vec(*lat: ndarray, lon: ndarray*) → ndarray

Returns geoid height for given latitude and longitude positions.

Parameters

- **lat** (*float*) – The latitude coordinates in decimal degrees.
- **lon** (*float*) – The longitude coordinates in decimal degrees.

Returns

The geoid height for the corresponding locations (in metres).

Return type

float

get_dov(*lat: float, lon: float*) → tuple[float, float]

Returns the Deflection of the Vertical (DoV) for requested position. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*float*) – A latitude coordinate in decimal degrees.
- **lon** (*float*) – A longitude coordinate in decimal degrees.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

get_dov_vec(*lat: ndarray, lon: ndarray*) → tuple[ndarray, ndarray]

Returns the Deflection of the Vertical (DoV) for requested positions. Calculates and returns the DoV gradients for the north and east components (in metres) for requested location.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinates in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinates in decimal degrees.

Returns

Tuple containing the north and east components respectively.

Return type

tuple[float, float]

qnav.earth.geoid.read_pgm_data(*pgm_file: Path, pgm_offset: float = -108, pgm_scale: float = 0.003*) → ndarray

Reads and returns the numerical contents of given PGM file. For a given PGM file, this function will read it and extract the Geoid height information (relative to the Earth's Ellipsoid) from it. The returned matrix's origin point's coordinates will be [90, -180] with the last point's coordinates being [-90, 180].

Parameters

- **pgm_file** (*str or Path*) – The location of the PGM file containing Geoid data.
- **pgm_offset** (*float*) – (Optional) The offset values for the values within the PGM file as PGM values normally start from 0 (default = -108).
- **pgm_scale** (*float*) – (Optional) The scale for the values within the PGM file as PGM values can only be integers (default = 0.003).

Returns

A matrix containing the Geoid height (relative to the Earth's Ellipsoid) for various locations along the earth.

Return type

np.ndarray

`qnav.earth.geoid.read_pgm_header(pgm_file: Path) → dict`

Returns values present in the header of a given PGM file. PGM files can contain comments which describe information such as the offset and scale values for the Geoid data they contain. For scenarios where the offset and scale are contained within these comments, this function will find and extract their values.

Parameters

pgm_file (*Path*) – The location of the PGM file containing Geoid data.

Returns

A dictionary containing the found offset and scale values (or None/Null if the values could not be found within the comments).

Return type

dict

qnav.earth.utm

`qnav.earth.utm.utm2lla(x: float, y: float, zone: int) → tuple[float, float]`

This function converts from Universal Transverse Mercator (UTM) to Latitude-Longitude (LLA) coordinates. This function takes the X (eastern) and Y (northern) offset from a given UTM integer zone (1 to 60) and return a tuple containing the corresponding latitude-longitude coordinates, respectively.

Parameters

- **x** (*float*) – The X/Eastern offset from the given UTM zone (in metres).
- **y** (*float*) – The Y/Northern offset from the given UTM zone (in metres).
- **zone** (*int*) – The integer UTM zone (in range 1 to 60).

Returns

The given UTM coordinate converted to Latitude and Longitude coordinates (contained within a tuple).

Return type

tuple

1.9.2 Estimation

Modules related to state estimation:

qnav.estimated.state**state.py****summary**

Supported base estimation state classes used to hold estimated data. This module contains classes that handle the estimated state of the platform during simulations. These are shared between the employed fusion methods. Extensions of the base class are required for the handling of additional data (such as error states and kalman matrices).

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.estimated.state.EstimatedState(ground_truth: ~qnav.waypoints.trajectory.GroundTruth,
                                           gravity_model: ~qnav.gravity.base.GravityModel =
                                           <qnav.gravity.simple.SimpleUniform object>)
```

The base estimated state holding only general state information. Simply maintains the estimated time, position, velocity, acceleration, attitude and angle rates of the platform. Additionally, this also holds a shared gravity model for calculating the expected gravity dynamics.

property timestamp: float

Returns the current estimated duration of the simulation (in seconds).

Returns

The estimated time (in seconds).

Return type

float

property position: ndarray

Returns the estimated position (in degrees-degrees-metres). This is the estimated position represented by an array of latitude, longitude and altitude values respectively.

Returns

The estimated position (in degrees-degrees-metres).

Return type

np.ndarray

property velocity: ndarray

Returns the estimated velocity (in meters/second). This is the estimated velocity in body axis represented as an array of along, across and down velocities respectively.

Returns

The estimated velocity (in meters/second).

Return type

np.ndarray

property acceleration: ndarray

Returns the estimated acceleration (in meters/second²). This is the estimated acceleration in body axis represented as an array of along, across and down accelerations respectively.

Returns

The estimated acceleration (in meters/second²).

Return type

np.ndarray

property attitude: ndarray

Returns the estimated attitude (in degrees). This is the estimated attitude in NED axis represented as an array of heading, pitch and yaw angles respectively.

Returns

The estimated attitude (in degrees).

Return type

np.ndarray

property angle_rates: ndarray

Returns the estimated angle rates (in degrees/second). This is the estimated angle rates in body axis represented as an array of P, Q and R euler angles respectively.

Returns

The estimated angle rates (in degrees/second).

Return type

np.ndarray

property gravity_model: *GravityModel*

Returns the shared gravity model. This is the gravity model instance used for estimating the gravitational effect during estimation and fusion. For consistency, this is held in the estimated state so it is shared.

Returns

The estimation gravity model.

Return type

GravityModel

update_estimates (*timestamp: float = None, position: ndarray = None, velocity: ndarray = None, acceleration: ndarray = None, attitude: ndarray = None, angle_rates: ndarray = None*)

Overwrites selected current estimated states. This is namely used when estimation fusion is applied and the internal states must be updated. To prevent accidental overwriting of estimated states or the setting of invalid data, all changes must be submitted through this method.

Parameters

- **timestamp** (*float*) – The updated timestamp marking the estimated current duration of the simulation (in seconds).
- **position** (*numpy.ndarray (3-elements)*) – The updated position vector containing the latitude (in degrees), longitude (in degrees) and altitude (in metres) respectively.
- **velocity** (*numpy.ndarray (3-elements)*) – The updated velocity vector for the North, East and Down components respectively (in m/s).
- **acceleration** (*numpy.ndarray (3-elements)*) – The updated acceleration vector for the North, East and Down components respectively (in m/s²).
- **attitude** (*numpy.ndarray (3-elements)*) – The updated attitude vector for the Heading, Pitch and Roll components respectively (in degrees).
- **angle_rates** (*numpy.ndarray (3-elements)*) – The updated angle rate vector for the P, Q and R body rates respectively (in deg/s).

as_dict() → dict[str, float | ndarray]

Represents the state and its content as a dictionary. Namely used when a simple representation is needed.

Returns

States in a python dictionary.

Return type

dict[str, float | np.ndarray]

as_numpy() → ndarray

Represents the state and its content as a numpy array. Namely used when a simple representation is needed.

Returns

States in concatenated numpy array.

Return type

numpy.ndarray

clone()

Generates a cloned instance of itself.

Returns

Deep clone of this instance.

Return type

State

```
class qnav.estimation.state.KalmanEstimatedState(ground_truth:
    ~qnav.waypoints.trajectory.GroundTruth, h:
    ~numpy.ndarray, r: ~numpy.ndarray, f:
    ~numpy.ndarray, q: ~numpy.ndarray, state_errors:
    ~numpy.ndarray = array([[1.e-09, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 1.e-09, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 1.e-09,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 1.e-09, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 1.e-09, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 1.e-09,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 1.e-09, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 1.e-09, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 1.e-09,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 1.e-09, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 1.e-09,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 1.e-09, 0.e+00,
    0.e+00, 0.e+00, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 1.e-09, 0.e+00, 0.e+00],
    [0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 1.e-09, 0.e+00], [0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00, 0.e+00,
    0.e+00, 0.e+00, 0.e+00, 0.e+00, 1.e-09]]),
    gravity_model: ~qnav.gravity.base.GravityModel =
    <qnav.gravity.simple.SimpleUniform object>)
```

Extended estimation state featuring Kalman Filter properties. This is an extension for the standard estimated state so that Kalman states can also be represented. These additions include shared Kalman Filter matrices and state errors.

property state_vector

Returns estimated states in form of state vector. A commonly used representation of estimated states in Kalman Filters.

Returns

Kalman State Vector

Return type

numpy.ndarray (15-elements)

update_estimates(*timestamp: float = None, position: ndarray = None, velocity: ndarray = None, acceleration: ndarray = None, attitude: ndarray = None, angle_rates: ndarray = None, state_errors: ndarray = None*)

Overwrites selected current estimated states. This is namely used when estimation fusion is applied and the internal states must be updated. To prevent accidental overwriting of estimated states or the setting of invalid data, all changes must be submitted through this method.

Parameters

- **timestamp** (*float*) – The updated timestamp marking the estimated current duration of the simulation (in seconds).
- **position** (*numpy.ndarray (3-elements)*) – The updated position vector containing the latitude (in degrees), longitude (in degrees) and altitude (in metres) respectively.
- **velocity** (*numpy.ndarray (3-elements)*) – The updated velocity vector for the North, East and Down components respectively (in m/s).
- **acceleration** (*numpy.ndarray (3-elements)*) – The updated acceleration vector for the North, East and Down components respectively (in m/s²).
- **attitude** (*numpy.ndarray (3-elements)*) – The updated attitude vector for the Heading, Pitch and Roll components respectively (in degrees).
- **angle_rates** (*numpy.ndarray (3-elements)*) – The updated angle rate vector for the P, Q and R body rates respectively (in deg/s).
- **state_errors** (*numpy.ndarray (15-by-15 elements)*) – The update Kalman state errors.

1.9.3 Fusion

Modules related to standard sensor fusion methods:

qnav.fusion.ins

ins.py

summary

Fusion method for supplying Inertial Navigation. Provides a selection of standard sensor fusion methods for means of basic internal navigation. Each of these takes measurements from accelerometer and gyroscope sensors (IMU).

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.fusion.ins.NumericalINS(accelerometer: Accelerometer, gyroscope: Gyroscope,  
                                estimated_acc_axis: SensorAxis = None, estimated_gyro_axis:  
                                SensorAxis = None)
```

A fairly basic simple numerical solution for Internal Navigation (INS). This INS solution is based on simple numerical integration. While its estimation calculations are relatively primitive, it serves as a base model that other solutions can extend from. The simple implementation also provides straight-forward means for testing calculations.

perform_fusion(*estimated_state*: [EstimatedState](#)) → None

Performs fusion using latest accelerometer and gyroscope measurements. Using the latest basic inertial measurements, updates the full current estimated state using a simple numerical model.

Parameters

estimated_state ([EstimatedState](#)) – The current estimated state.

```
class qnav.fusion.ins.RungeKutta(accelerometer: Accelerometer, gyroscope: Gyroscope,  
                                estimated_acc_axis: SensorAxis = None, estimated_gyro_axis:  
                                SensorAxis = None)
```

An extension of the numerical INS that employs Runge-Kutta methods. Processes the measurement vector and updates the estimation states using Runge Kutta (4th order) integration.

Does include:

- Simple Accelerometer Measurement Model (basic - independent measurement errors, giving drift term)
- 3 x Individual Accelerator Bias values (fixed)
- 3 x Individual Accelerator Scaling errors (fixed)
- 3D Accelerometer non-orthogonality errors (fixed) for cross-coupling
- Simple (not position dependent) Gravity Compensation
- Simple Gyroscope Measurement Model (basic - independent measurement errors, giving drift term)
- 3 x Individual Gyroscope Bias values (fixed)
- 3 x Individual Gyroscope Scaling errors (fixed)
- 3D Gyroscope non-orthogonality errors (fixed) for cross-coupling
- Sensor bias drift for accelerometers and gyroscopes
- Kalman Filtering of measurement signals
- WGS'84 coordinates (ellipsoidal rotating Earth)
- Effect of Coriolis effect due to Earth's rotation

Does NOT currently include:

- Schuler correction loop.
- Physics-based accelerometer sensor model
- Physics-based gyroscope sensor model
- Lever-arm effects from rotations not around origin/centre of the IMU

perform_fusion(*estimated_state*: [EstimatedState](#)) → None

Performs fusion using latest accelerometer and gyroscope measurements. Using the latest basic inertial measurements, updates the full current estimated state using a simple numerical model.

Parameters

estimated_state ([EstimatedState](#)) – The current estimated state.

```
class qnav.fusion.ins.AdamsBashforth(accelerometer: Accelerometer, gyroscope: Gyroscope,
                                     estimated_acc_axis: SensorAxis = None, estimated_gyro_axis:
                                     SensorAxis = None)
```

An extension of the numerical INS that employs Adams-bashforth method. Processes the measurement vector and updates the estimation states using Adams-Bashforth integration.

Does include:

- Simple Accelerometer Measurement Model (basic - independent measurement errors, giving drift term)
- 3 x Individual Accelerator Bias values (fixed)
- 3 x Individual Accelerator Scaling errors (fixed)
- 3D Accelerometer non-orthogonality errors (fixed) for cross-coupling
- Simple (not position dependent) Gravity Compensation
- Simple Gyroscope Measurement Model (basic - independent measurement errors, giving drift term)
- 3 x Individual Gyroscope Bias values (fixed)
- 3 x Individual Gyroscope Scaling errors (fixed)
- 3D Gyroscope non-orthogonality errors (fixed) for cross-coupling
- Sensor bias drift for accelerometers and gyroscopes
- Kalman Filtering of measurement signals
- WGS'84 coordinates (ellipsoidal rotating Earth)
- Effect of Coriolis effect due to Earth's rotation

Does NOT currently include:

- Schuler correction loop.
- Physics-based accelerometer sensor model
- Physics-based gyroscope sensor model
- Lever-arm effects from rotations not around origin/centre of the IMU

```
perform_fusion(estimated_state: EstimatedState) → None
```

Performs fusion using latest accelerometer and gyroscope measurements. Using the latest basic inertial measurements, updates the full current estimated state using a simple numerical model.

Parameters

estimated_state (EstimatedState) – The current estimated state.

qnav.fusion.kalman

kalman.py

summary

Kalman based fusion method for supplying Inertial Navigation. Provides Kalman Filter fusion for the Inertial Navigation system (INS).

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
 Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.fusion.kalman.KalmanINS(accelerometer: Accelerometer, gyroscope: Gyroscope,  
                                   estimated_acc_axis: SensorAxis = None, estimated_gyro_axis:  
                                   SensorAxis = None)
```

Kalman Filter Inertial Navigation System (INS). This class represents a Kalman filter that uses linear quadratic estimations to update the estimated states and estimations of measurement errors. This solution is known to be more accurate than the other more basic INS methods provided in the project.

```
perform_fusion(kalman_state: KalmanEstimatedState) → None
```

Performs fusion using latest accelerometer and gyroscope measurements. Updates the estimated states by performing Kalman Filtering.

Parameters

kalman_state (EstimatedState) – The current estimated state.

```
qnav.fusion.kalman.state_vector2ned(state_vector: ndarray, ref_lla: ndarray, gravity_model:  
                                     GravityModel) → ndarray
```

This function converts the state vector array and measurements into local North-East-Down (NED) coordinates from a given reference location, suitable for kalman filtering.

Parameters

- **state_vector** (*numpy.ndarray (15-elements)*) – The state vector array holding the position, velocity, acceleration, attitude and angle rate variables.
- **ref_lla** (*numpy.ndarray (3-elements)*) – The reference latitude-longitude-altitude vector (given in degrees-degrees-metres) identifying the centre point.
- **gravity_model** (GravityModel) – The model for estimating downwards acceleration.

Returns

The given state vector array and measurements converted to local NED/Earth axes for Kalman filtering.

Return type

numpy.ndarray (15-elements)

```
qnav.fusion.kalman.ned2state_vector(state_vector: ndarray, ref_lla: ndarray, gravity_model:  
                                     GravityModel) → ndarray
```

This function converts the state vector array back to Latitude-longitude- altitude (LLA) coordinates (in degrees-degrees-metres). Please note that the state error matrix remains in Local North-East-Down (NED) coordinates.

Parameters

- **state_vector** (*numpy.ndarray (15-elements)*) – The state vector array holding the position, velocity, acceleration, attitude and angle rate variables.
- **ref_lla** (*numpy.ndarray (3-elements)*) – The reference latitude-longitude-altitude vector (given in degrees-degrees-metres) identifying the centre point.
- **gravity_model** (GravityModel) – The model for estimating downwards acceleration.

Returns

The given state vector array converted to LLA.

Return type

numpy.ndarray (15-elements)

qnav.fusion.altimeter**altitude.py****summary**

Supplied fusion method for altitude measurements. This module contains methods that can be used for acquiring altimeter measurements during simulation and fusing them with the current estimated altitude. Additional methods may be added in future updates.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

class qnav.fusion.altimeter.**FixedGainAltimeter**(*altimeter*: [Altimeter](#), *gain_amount*: *float* = 0.5)

Fuses altimeter altitude measurements using fixed gain fusion. Updates the estimated altitude by using a weighted average of the measurement altitude and current estimated altitude. This gain amount can be set and updated at any time.

perform_fusion(*estimated_state*: [EstimatedState](#)) → None

Performs the fixed gained fusion with last altimeter measurement. Simply fuses the last altimeter measurement with previous measurement using a fixed-gain (weighted average) update.

Parameters

estimated_state ([EstimatedState](#)) – The current estimated state.

property fixed_gain: **float**

Property getter method for fixed gain amount.

Returns

The current gain amount for the altimeter measurements.

Return type

float

class qnav.fusion.altimeter.**AlphaBetaAltimeter**(*altimeter*: [Altimeter](#), *alpha*: *float* = 0.9, *beta*: *float* = None)

Fuses altimeter altitude measurements using alpha-beta filtering. Updates the estimated altitude by using an alpha-beta filter, where alpha relates to the expected accuracy of the altimeter and beta relates to the precision of the vertical velocity estimate. This method updates both estimated altitude and the vertical velocity.

perform_fusion(*estimated_state*: [EstimatedState](#)) → None

Performs alpha-beta fusion with latest altimeter measurement. Updates both the estimated altitude and the vertical velocity using the latest measurement from the altimeter. From this both the vertical position and vertical velocity are updated.

Parameters

estimated_state ([EstimatedState](#)) – The current estimated state.

property alpha: **float**

Property getter method for the alpha value.

Returns

The current alpha value to use for filtering.

Return type

float

property beta: float

Property getter method for the beta value.

Returns

The current beta value to use for filtering.

Return type

float

1.9.4 GPS

Modules relating to GPS sensor and fusion methods:

qnav.gps.ephemeris

ephemeris.py

summary

Collection of functions for extracting GPS/GNSS information. This module is responsible for the util satellite functionality within the toolbox. This includes functions for downloading GPS information, parsing RINEX files, extracting satellite information and estimating missing values.

authors

Michael Wright - mjwright@liverpool.ac.uk

Dr. Niall Moroney - niall.moroney17@imperial.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gps.ephemeris.SatelliteType(*value, names=<not given>, *values, module=None, qualname=None, type=None, start=1, boundary=None*)

Character identifiers for the different satellite types. These characters identify the record types corresponding to the satellite systems supported inside a Rinex File (as of version 4).

qnav.gps.ephemeris.get_rinex_path(*gps_time: datetime, directory: Path*) → Path

Obtain the location of a local RINEX file identified by GPS date and time. This function will return the local filepath of a RINEX file corresponding to a given (python) datetime. The file of the returned path may or may not exist (requiring downloading if missing).

Parameters

- **gps_time** (*datetime.datetime*) – Python datetime object for the time required for the ephemeris.
- **directory** (*Path*) – Location for the files to be downloaded to.

Returns

The path of the corresponding RINEX file.

Return type

Path

qnav.gps.ephemeris.download_rinex_file(*gps_time: datetime, database_dir: Path*)

This function downloads and extracts an ephemeris file for a specified day from “cddis.gsfc.nasa.gov” and saves it under the provided directory.

Parameters

- **gps_time** – The Python datetime object for the time required for the ephemeris data.
- **database_dir** (*Path*) – The location for the files to be downloaded to.

Type

datetime.datetime

`qnav.gps.ephemeris.download_rinex_files`(*from_time: datetime, to_time: datetime, database_dir: Path, inc_last: bool = True*)

This function downloads and extracts multiple ephemeris files. Given a date range, this function will download all the ephemeris files covered by the range.

Parameters

- **from_time** (*datetime.datetime*) – The first date for the download range. This is the date for the first GPS file to be downloaded.
- **to_time** (*datetime.datetime*) – The last date for the download range. This is the end date for the GPS files to download.
- **database_dir** (*Path*) – The location for the files to be downloaded to.
- **inc_last** (*bool*) – Should the last date of the range be included?

`qnav.gps.ephemeris.read_rinex_file`(*ephemeris_file: Path*) → dict[str, dict]

Parses a downloaded ephemeris file and returns its data. Reads a given ephemeris file, skipping the header contents and returning the contained entry records in dictionaries. Each is keyed by the ID of the satellite.

Parameters

ephemeris_file (*Path*) – The local ephemeris file to read.

Returns

A dictionary of extracted satellite parameters.

Return type

dict[str, dict]

`qnav.gps.ephemeris.rinex_to_satellite_params`(*rinex_data: dict, *satellite_types: SatelliteType*) → list[*SatelliteParams*]

Extracts given rinex data and converts into satellite parameters class. A helper function for extracting the usable satellite parameter data from given Rinex data, allowing for satellite instances to be created. Optional filtering can also be supplied to limit returns to specific satellite types.

Parameters

- **rinex_data** (*dict[str, dict]*) – The data records read from a Rinex file.
- **satellite_types** (*SatelliteType*) – (Optional) The satellite types to filter by. By default, all types are selected and no filtering is performed.

Returns

The satellite parameters for each rinex record.

Return typelist[*SatelliteParams*]

qnav.gps.fusion**fusion.py****summary**

A collection of GPS/GNSS methods for fusing GPS measurements with INS. This module contains all the GPS fusion methods for merging GPS position and velocity updates with the INS current estimation. Simpler methods will simply merge the estimates, whereas more sophisticated methods will adapt expected errors to compensate for INS noise/drift.

authors

Kallum O'Hara - sgkohara@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

```
class qnav.gps.fusion.GpsFusion(gps_sensor: GpsSensor, estimated_state: EstimatedState,  
                               estimated_clock_bias: float = 0.0, estimated_clock_drift: float = 0.0,  
                               burn_in_iterations: int = 3)
```

The abstract base class for fusion methods. This class outlines the structure each fusion method must extend from. This essentially serves as a template class containing methods and properties that must be incorporated by each GPS fusion method.

solve() → None

Performs solving of GPS measurements to estimated values. Essentially this translates the GPS satellite measurements into internal local estimates.

```
class qnav.gps.fusion.GpsFixedGainFusion(gps_sensor: GpsSensor, estimated_state: EstimatedState,  
                                         gain_amount: float = 0.1)
```

Performs simple fixed-gain GPS fusion. Updates the estimated state's position and velocity using simple fixed gained fusion. Can be used with any estimated state class.

perform_fusion(estimated_state: EstimatedState) → None

Performs GPS correction using simple fixed-gained fusion.

Parameters

estimated_state ([EstimatedState](#)) – The current estimated state.

```
class qnav.gps.fusion.GpsLooseFusion(gps_sensor: GpsSensor, estimated_state: KalmanEstimatedState,  
                                     pos_cov_mat_ned: ndarray, vel_cov_mat_ned: ndarray)
```

Performs loosely-coupled based GPS fusion. A fusion method that uses loosely-coupled fusion for updating the INS with given GPS estimates. To further improve the accuracy of the navigation solution, the error states are fed back to the INS. Must be used with a Kalman estimated state class.

perform_fusion(kalman_state: KalmanEstimatedState) → None

Applies loose-coupled fusion using GPS estimates and INS solution. This method updates the given INS instance's estimates by fusing them with the given GPS estimates. The INS states are updated internally.

Parameters

kalman_state ([KalmanEstimatedState](#)) – The current estimated state.

```
class qnav.gps.fusion.GpsTightFusion(gps_sensor: GpsSensor, estimated_state: KalmanEstimatedState,  
                                     tight_state_vector: ndarray, tight_state_errors: ndarray)
```

Tightly-coupled based GPS fusion. A fusion method that uses tightly-coupled fusion for updating the INS with given GPS estimates. This method uses a centralized Kalman filter that integrates the estimated Pseudo Ranges

and Doppler shift from the GNSS receiver, along with the estimated position, velocity and attitude from the INS. Must be used with a Kalman estimated state class.

perform_fusion(*estimated_state*: [KalmanEstimatedState](#)) → None

Applies tightly-coupled fusion using GPS estimates and INS solution. This method updates the given INS instance's estimates by fusing them with the given GPS estimates. The INS states are updated internally.

Parameters

estimated_state ([KalmanEstimatedState](#)) – The current estimated state.

qnav.gps.fusion.calculate_cov_mat(*satellites*: [Collection\[Satellite\]](#), *ant_pos_lla*: *ndarray*, *sigma_position*: *float* = 50, *sigma_velocity*: *float* = 0.3) → tuple[*ndarray*, *ndarray*]

Produces the covariance matrices for loosely-coupled fusion. This method produces and returns the covariance matrices for position and velocity based on the current position. These are namely required for initialising a loosely-coupled GPS fusion instance (if matrices are not produced manually).

Parameters

- **satellites** ([Collection\[Satellite\]](#)) – The GPS satellites involved in measurements.
- **ant_pos_lla** (*numpy.array* (3-elements)) – The true antenna position (latitude, longitude and altitude given in deg-deg-metres).
- **sigma_position** (*float*) – The pseudo-range position error (in metres).
- **sigma_velocity** (*float*) – The pseudo-range velocity error (in metres/sec).

Returns

A tuple containing the constructed position and velocity covariance matrices, respectively.

Return type

tuple[*np.ndarray*, *np.ndarray*]

qnav.gps.satellite

satellite.py

summary

All GPS satellite and solver functionalities. This module is responsible for simulating up-to-date satellites and supplying a solver to produce measurements (with configured errors and noise). During a simulation these classes will be used for acquiring GPS information.

authors

Dr. Niall Moroney - niall.moroney17@imperial.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

0.9.0 - Awaiting further implementations.

```
qnav.gps.satellite.satellite_param_fields = ('type', 'id', 'year', 'month', 'day',
'hour', 'minute', 'second', 'clock_bias', 'drift_freq_bias', 'drift_rate_message_time',
't_0e', 'prn', 'delta_n', 'i_dot', 'omega_dot', 'omega_0', 'c_us', 'c_uc', 'c_is',
'c_ic', 'c_rs', 'c_rc', 'sqrt_a', 'e', 'i_0', 'w', 'm_bar_0')
```

The field names for satellite parameters.

```
class qnav.gps.satellite.SatelliteParams(type, id, year, month, day, hour, minute, second, clock_bias,
drift_freq_bias, drift_rate_message_time, t_0e, prn, delta_n,
i_dot, omega_dot, omega_0, c_us, c_uc, c_is, c_ic, c_rs, c_rc,
sqrt_a, e, i_0, w, m_bar_0)
```

A collection of satellite parameters needed to initialise a Satellite instance.

c_ic

Alias for field number 20

c_is

Alias for field number 19

c_rc

Alias for field number 22

c_rs

Alias for field number 21

c_uc

Alias for field number 18

c_us

Alias for field number 17

clock_bias

Alias for field number 8

day

Alias for field number 4

delta_n

Alias for field number 13

drift_freq_bias

Alias for field number 9

drift_rate_message_time

Alias for field number 10

e

Alias for field number 24

hour

Alias for field number 5

i_0

Alias for field number 25

i_dot

Alias for field number 14

id

Alias for field number 1

m_bar_0

Alias for field number 27

minute

Alias for field number 6

month

Alias for field number 3

omega_0

Alias for field number 16

omega_dot

Alias for field number 15

prn

Alias for field number 12

second

Alias for field number 7

sqrt_a

Alias for field number 23

t_0e

Alias for field number 11

type

Alias for field number 0

w

Alias for field number 26

year

Alias for field number 2

```
class qnav.gps.satellite.Satellite(params: SatelliteParams, gps_time: datetime = None,
                                   pseudo_range_noise: float = 0, pseudo_range_rate_noise: float = 0,
                                   rand_seed: int = None)
```

Represents a GPS satellite and its current states/properties. This class is responsible for representing information about a GPS satellite. Methods are supplied for updating these states and allowing for measurement information to be acquired. During simulations a collection of satellite instances will be used for generating a GPS signal that will be used to improve navigation accuracy.

update(*dt: float*)

Updates the position, velocity and errors given the time difference. This method updates the state of the satellite by a given amount of time (in seconds). The updated position and velocity of the satellite are calculated along with the new error states for generating noisy measurements.

Parameters

dt (*float*) – The time difference since the last update was requested.

```
get_location_azimuth_elevation(ant_pos_lla: ndarray) → tuple[float, float]
```

Get the relative location of the satellite from the current position, in azimuth and elevation (deg-deg).

Parameters

ant_pos_lla (*numpy.array (3-elements)*) – The true antenna position (latitude, longitude and altitude given in deg-deg-metres).

Returns

Returns the corresponding azimuth and elevation (both in degrees).

Return type

tuple[float, float]

```
get_measurements(ant_pos_lla: ndarray, ant_att_deg: ndarray, ant_vel_body_axes: ndarray) → dict
```

Calculates and returns the noisy pseudo ranges, position, velocity and time information for the satellite.

Parameters

- **ant_pos_lla** (*numpy.array (3-elements)*) – The true antenna position (latitude, longitude and altitude given in deg-deg-metres).
- **ant_att_deg** (*numpy.array (3-elements)*) – The true antenna attitude (heading, pitch and roll in degrees).
- **ant_vel_body_axes** (*numpy.array (3-elements)*) – The true antenna velocity (in body axes in metres/second).

Returns

The relevant measurement information in a dict.

Return type

dict

property sat_id: str

Returns the full satellites ID from given parameters.

Returns**Return type**

str

property sat_time: datetime

The satellite's current clock time.

Returns

Current satellite time.

Return type

datetime.datetime

property position: ndarray

The satellite's ECEF position since last update.

Returns

Current ECEF position.

Return type

numpy.ndarray

property velocity: ndarray

The satellite's ECEF velocity since last update.

Returns

Current ECEF velocity.

Return type

numpy.ndarray

property pseudo_range_noise: float

Returns the satellite's initial pseudo range noise.

Returns

Pseudo range noise.

Return type

float

property pseudo_range_rate_noise: float

Returns the satellite's initial pseudo range rate noise.

Returns

Pseudo range rate noise.

Return type

float

qnav.gps.sensor

gps/sensor.py

summary

GPS sensor activities used for collecting measurements from satellites. This module provides sensor classes that are responsible for obtaining and returning GPS measurements from selected satellites. These measurements can then be passed to the appropriate GPS fusion method.

authors

Kallum O'Hara - sgkohara@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

```
class qnav.gps.sensor.GpsSensor(frequency: float, satellites: Collection[Satellite], gps_zones:
    FlaggedPolygonGroup = None, start_time: float = 0.0, rand_seed: int =
    None)
```

A basic sensor for collecting measurements from GPS satellites. This sensor can be used to collect measurements on demand from presented GPS satellites at requested frequency.

take_measurement(*est_time: float, ground_truth: GroundTruth*)

Obtains updated measurement from GPS satellites. Obtains measurements from all GPS satellites and updates the last measurement. Performed prior to GPS fusion.

Parameters

- **est_time** (*float*) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** (*GroundTruth*) – The corresponding ground truth data.

property last_measurement: dict

Returns the list of latest GPS satellite measurements.

Returns

Latest GPS satellite measurements.

Return type

dict

update(*time_sec: float = None*)

Updates the state of the satellites for the given amount of seconds. This method is used to update each of the underlying satellites in the given collection.

Parameters

time_sec (*float*) – The estimated current simulation time.

qnav.gps.zones**zones.py****summary**

Handles the representation and method connected with zones/areas. This module contains a family of classes used for representing and querying zones. This has been designed for reuse, with various child area-polygon classes and their associated group handlers.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class `qnav.gps.zones.Polygon`(*lat_points: ndarray, lon_points: ndarray, priority: int = 0*)

Represents a simple latitude-longitude closed polygon. Can be used for defining custom 2D zones on the globe.

is_point_inside(*lat_point: float, lon_point: float*) → bool

Determines if a given position is inside the polygon. Given the latitude and longitude coordinates of a single point in decimal degrees, this function will return True if the point is inside the polygon.

Parameters

- **lat_point** (*float*) – The latitude coordinate of the point of interest.
- **lon_point** (*float*) – The longitude coordinate of the point of interest.

Returns

Returns True if given point is inside the polygon.

Return type

bool

is_points_inside(*lat_points: ndarray, lon_points: ndarray*) → ndarray

Determines if a series of positions are inside the polygon. Given the latitude and longitude coordinates of a multiple points in decimal degrees, this function will return True for each point that is inside the polygon.

Parameters

- **lat_points** (*float*) – The latitude coordinates of the points of interest.
- **lon_points** (*float*) – The longitude coordinates of the points of interest.

Returns

Returns True for each given point inside the polygon.

Return type

bool

property lat_points: ndarray

Returns the list of latitude coordinate points of the polygon. This property returns a copy, preventing accidental editing.

Returns

The latitude coordinate points of the polygon.

Return type

np.ndarray

property lon_points: ndarray

Returns the list of longitude coordinate points of the polygon. This property returns a copy, preventing accidental editing.

Returns

The longitude coordinate points of the polygon.

Return type

np.ndarray

property priority: int

Returns the priority (or layer) of the polygon. This is used for determining the outcome when multiple polygons overlapping one another are involved. When this occurs the highest priority polygon is used.

Returns

The priority (or layer) of the polygon.

Return type

int

class qnav.gps.zones.FlaggedPolygon(*lat_points: ndarray, lon_points: ndarray, is_active_area: bool, priority: int = 0*)

Represents a latitude-longitude closed polygon with binary active flag. These polygons are used for determining “active” and “inactive” areas. For example, areas where specific sensors, such as where GPS or quantum sensors can and cannot be used.

property is_active_area: bool

Returns True if the polygon is active area. This can be used to determine if points which are found inside the polygon are to be considered inside an active area.

Returns

Whether the polygon is an active area.

Return type

bool

class qnav.gps.zones.FlaggedPolygonGroup(*polygons: List[FlaggedPolygon], is_active_by_default: bool = True*)

A handler for a group of flagged polygons. Allows the querying of multiple polygons, in priority order, with a default flag.

get_point_flag(*lat_point: float, lon_point: float*) → bool

Returns the boolean flag for given point. Searches through given polygons in priority order and returns the flag for the highest priority polygon that the point is inside of. If the point is not in any polygons, the default flag will be used.

Parameters

- **lat_point** (*float*) – The latitude coordinate of the point of interest.
- **lon_point** (*float*) – The longitude coordinate of the point of interest.

Returns

Boolean flag of the highest priority polygon the point is inside of (or the default flag for the group).

Return type

bool

property polygons: List[*FlaggedPolygon*]

Returns list of flagged polygons used by group.

Returns

list of flagged polygons used by group

Return type

List[*FlaggedPolygon*]

property num_polygons: int

Returns the number of polygons covered by the group.

Returns

The number of polygons covered by the group.

Return type

int

get_point_flags(*lat_points: ndarray, lon_points: ndarray*) → ndarray

Returns the boolean flags for multiple given points. Searches through given polygons in priority order and returns the flags of the highest priority polygons that each of the point is inside of. For points that do not fall inside any polygon's the default flag is used.

Parameters

- **lat_points** (*np.ndarray*) – The latitude coordinates of the points of interest.
- **lon_points** (*np.ndarray*) – The longitude coordinates of the points of interest.

Returns

Boolean flags of the highest priority polygon each point is inside of (or the default flag for the group).

Return type

np.ndarray

qnav.gps.zones.is_inside_polygon(*x_point: float, y_point: float, polygon_x: ndarray, polygon_y: ndarray*) → bool

Determines if a given position is inside given the polygon. Tests by generating horizontal lines and assessing the number of polygon lines that are crossed. Using this count, a point can be determine to be inside the closed polygon or not.

Parameters

- **x_point** (*float*) – The x coordinate of the point of interest.
- **y_point** (*float*) – The y coordinate of the point of interest.
- **polygon_x** (*np.ndarray*) – The x-coordinates of the polygon. Assumed to be closed.
- **polygon_y** (*np.ndarray*) – The y-coordinates of the polygon. Assumed to be closed.

Returns

Returns True if the given position is inside the polygon.

Return type

bool

qnav.gps.zones.is_inside_polygon_vec(*x_points: ndarray, y_points: ndarray, polygon_x: ndarray, polygon_y: ndarray*) → ndarray

Determines if a given position is inside given the polygon. Tests by generating horizontal lines and assessing the number of polygon lines that are crossed. Using this count, points can be determine to be inside the closed polygon or not.

Parameters

- **x_points** (*float*) – The x coordinates of the point of interest.
- **y_points** (*float*) – The y coordinates of the point of interest.
- **polygon_x** (*np.ndarray*) – The x-coordinates of the polygon. Assumed to be closed.
- **polygon_y** (*np.ndarray*) – The y-coordinates of the polygon. Assumed to be closed.

Returns

Returns True if the given position is inside the polygon.

Return type

bool

1.9.5 Gravity

Modules relating to gravitational calculations:

qnav.gravity.base

base.py

summary

Gravity model template class interfaces that supported gravity models must extend from. The purpose of this is to define the functions, input arguments and outputs that gravity models must incorporate to be used in the toolbox. This also simplifies the ability to add new gravity models future.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.base.GravityModel

The abstract class that gravity models are to follow. This defines the interface that all gravity models in the project must follow, outlining the necessary functions they must supply for calculating various gravity components at requested positions. For a gravity model to be used by the toolbox, it must correctly extend from this abstract base class.

In short, each gravity model must supply the following methods:

- **__str__** - The descriptive name for the gravity model.
- **calc_gravity_z** - Calculate downwards gravitational acceleration.
- **calc_gravity_xyz** - Calculate gravitational acceleration vector.
- **calc_gravity_grad** - Calculate gravitational gradient tensor.

In addition to this, there are also optional methods they can supply:

- **calc_gravity_z_vec** - Vectorised downwards gravitational acceleration.
- **calc_gravity_xyz_vec** - Vectorised gravitational acceleration vector.
- **calc_gravity_grad_vec** - Vectorised gravitational gradient tensor.

These are used to efficiently calculate gravity for a large range of positions. To enforce compatibility, if not implemented, calling these methods will execute the scalar corresponding methods in a loop.

abstract calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_xyz(*lat: float, lon: float, alt: float*) → ndarray

Calculates and returns the gravitational acceleration vector. Computes the gravity acceleration X/North, Y/East and Z/Down components (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s²).

Return type

numpy.ndarray (3-elements)

calc_vertical_grad(*lat: float, lon: float, alt: float, step: float = 100*) → float

Calculates the vertical gradient of gravity. Computes the vertical change in gravity (dg/dr) of the Z/Down component (in m/s²). This is primary used for gravity map-matching related calculations.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.
- **step** (*float*) – The step size (in metres) for calculating the gradient. To be used when the gradient cannot be computed directly.

Returns

The vertical gravity gradient (in m/s²).

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

calc_gravity_xyz_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration for the X/North, Y/East and Z/Down axes (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s²).

Return type

numpy.ndarray (N-by-3 elements)

calc_vertical_grad_vec(*lat: ndarray, lon: ndarray, alt: ndarray, step: float = 10*) → ndarray

In a vectorised batch, calculates the vertical gradient of gravity. Computes the vertical change in gravity (dg/dr) of the Z/Down component (in m/s²). This is primary used for gravity map-matching related calculations.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.ndarray*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

- **step** (*float*) – The step size (in metres) for calculating the gradient. To be used when the gradient cannot be computed directly.

Returns

The vertical gravity gradient (in m/s^2).

Return type

float

property base_model: Self | None

Returns the base gravity model the model extends from. Allows access to the base function/model the instance is using (if any).

Returns

Either the base gravity model or None.

Return type

GravityModel | None

`qnav.gravity.base.calculate_curvature(lat: float | ndarray) → float | ndarray`

Calculates the mean curvature of the ellipsoid J at given position. Using equations from Physical Geodesy 2006, this function calculates J for given positions.

Parameters

lat (*float* | *np.ndarray*) – The latitude of the point(s) of interest (in decimal degrees).

Returns

The mean curvature of the ellipsoid.

Return type

float | *np.ndarray*

qnav.gravity.simple

simple.py

summary

Implementation of simple gravity modelling classes and functions. This class contains very simple gravity modelling functions that are namely used for testing, replacing gravity with values that are fixed or vary very little. Primarily used for cancelling out any impact of gravity during experiments.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.simple.FixedValue

Sets gravity equal to fixed constant regardless of location. Using this model, gravity will be fixed and the same constant value will be returned.

calc_gravity_z(*lat: float = None, lon: float = None, alt: float = None*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s^2) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.

- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray = None, alt: ndarray = None*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

class qnav.gravity.simple.**SimpleUniform**

Calculates gravity using uniformly across the Earth. Produces values using weighted average of the gravity between the poles and equator, meaning only latitude affected the calculations produced. Changes in longitude or altitude will have no impact.

calc_gravity_z(*lat: float, lon: float = None, alt: float = None*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray = None, alt: ndarray = None*) → float

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s^2).

Return type

np.array (N-elements)

qnav.gravity.somigliana**somigliana.py****summary**

Implementation of the Somigliana using WGS-84 parameters. This class contains gravity functions related to the Somigliana formula (with height correction).

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.somigliana.Somigliana

Gravity function implementation of the Somigliana formula. This allows the calculation of the vertical gravity, gravitational acceleration and gravity gradients using the Somigliana formula (with altitude interpolation).

calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the gravitational acceleration along the Z/Down axis (in m/s^2) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_xyz(*lat: float, lon: float, alt: float*) → ndarray

Calculates and returns the gravitational acceleration for the X/North, Y/East and Z/Down axes (in m/s^2) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s^2).

Return type

numpy.ndarray (3-elements)

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s^2) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s^2).

Return type

np.array (N-elements)

calc_gravity_xyz_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration for the X/North, Y/East and Z/Down axes (in m/s^2) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s^2).

Return type

numpy.ndarray (N-by-3 elements)

qnav.gravity.nima**nima.py****summary**

National Imagery and Mapping Agency (NIMA) gravity functions. This module contains all the gravity calculation methods published by NIMA, specifically those stated in “Department of Defense World Geodetic System 1984: Its Definition, and Relationship with Local Geodetic Systems, TR8350.2, Third Ed.” Department of Defense, Washington, DC: 1997.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.nima.WGS84Gravity

Implementation of the WGS84 representation of Earth gravity. This model uses the equations published by NIMA. This is known to be a relatively accurate close approximation model for heights up to 20,000 meters above the WGS84 reference Ellipsoid.

calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_xyz(*lat: float, lon: float, alt: float*) → ndarray

Calculates and returns the gravitational acceleration for the X/North, Y/East and Z/Down axes (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s²).

Return type

numpy.ndarray (3-elements)

qnav.gravity.map**map.py****summary**

Module provides template implementation for gravity correction maps. Due to their high resolution and spatial complexity, methods from the original gravity model base class have had to be adjusted. The largest feature provided by this module is the method for correctly approximating the vertical gravity gradient.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.map.**CircularGrid**(*num_rings: int = 100, num_angles: int = 50, max_radius: float = 0.25, l_threshold: float = 1.0*)

Definition for a circular reference grid from a centre position. Circular grid points used for involving the surrounding data for accurately computing the vertical gravity gradient.

generate_grid(*lat: float, lon: float, alt: float*) → dict[str, ndarray]

Generates a circular grid of query positions surrounding a point. On succession, a dictionary containing the positional and angular information of points surrounding a position of interest are returned. This also includes commonly calculated values step size values.

Parameters

- **lat** (*float*) – The latitude position of the point (in decimal degrees).
- **lon** (*float*) – The longitude position of the point (in decimal degrees).
- **alt** (*float*) – The altitude position of the point (in metres).

Returns

A dictionary holding the calculated positions (lla), NED offsets (ned), azimuth angles (alpha), distances from centre

(psi) and step sizes (delta_psi and delta_alpha).

Return type

dict[str, np.ndarray]

qnav.gravity.map.**set_default_grid**(*num_rings: int, num_angles: int, max_radius: float, l_threshold: float*)

Overwrites the settings to use for the default circular reference grid. This is the grid that will be used by default for calculating the vertical gravity gradient.

Parameters

- **num_rings** (*int*) – Number of radial distance steps.
- **num_angles** (*int*) – Number of azimuth angle steps.
- **max_radius** (*float*) – Maximum radial distance (in degrees).

- **l_threshold** (*float*) – Threshold for internal points (in metres).

class qnav.gravity.map.**GravityMap**(*base_model*: GravityModel, *geoid_model*: GeoidModel | None, *map_dir*: Path | None, *auto_download*: bool = True)

An extension to the abstract base class GravityModel that provides re-worked methods for handling high-resolution lookup maps.

abstract **get_anomaly**(*lat*: float, *lon*: float) → float

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

abstract **get_disturbance**(*lat*: float, *lon*: float) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

abstract **get_anomaly_vec**(*lat*: ndarray, *lon*: ndarray) → ndarray

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

abstract get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

calc_gravity_xyz(*lat: float, lon: float, alt: float*) → ndarray

Calculates and returns the gravitational acceleration vector. Computes the gravity acceleration X/North, Y/East and Z/Down components (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s²).

Return type

numpy.ndarray (3-elements)

calc_gravity_xyz_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration for the X/North, Y/East and Z/Down axes (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The local gravity vector for the XYZ components (in m/s²).

Return type

numpy.ndarray (N-by-3 elements)

calc_vertical_grad(*lat: float, lon: float, alt: float, step: float = 100, circular_grid: CircularGrid = None*)
→ float

Calculates the vertical gradient of gravity. Computes the vertical change in gravity (dg/dr) of the Z/Down component (in m/s²). This is primary used for gravity map-matching related calculations.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.
- **step** (*float*) – The step size (in metres) for calculating the gradient. To be used when the gradient cannot be computed directly.

Returns

The vertical gravity gradient (in m/s²).

Return type

float

calc_vertical_grad_vec(*lat: ndarray, lon: ndarray, alt: ndarray, step: float = 100, circular_grid: CircularGrid = None*) → ndarray

In a vectorised batch, calculates the vertical gradient of gravity. Computes the vertical change in gravity (dg/dr) of the Z/Down component (in m/s²). This is primary used for gravity map-matching related calculations.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.ndarray*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.
- **step** (*float*) – The step size (in metres) for calculating the gradient. To be used when the gradient cannot be computed directly.

Returns

The vertical gravity gradient (in m/s²).

Return type

float

property base_model: Self | None

Returns the base gravity model the model extends from. Allows access to the base function/model the instance is using.

Returns

The base gravity model.

Return type

GravityModel | None

qnav.gravity.egm**egm.py****summary**

A module for supplying Earth Gravitational Models (EGMs). This provides implementations for EGM gravity models, using a standard ‘base’ gravity function and a geoid model for height and surface deflection correction.

authors

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of this module.

class `qnav.gravity.egm.GeoidCorrection`(*base_model*: [GravityModel](#), *geoid_model*: [GeoidModel](#))

Gravity Map using Geoid Model for applying acceleration corrections. Using a given geoid model to applying corrections to the gravity accelerations, using the height offsets and deflections of vertical.

calc_gravity_z(*lat*: *float*, *lon*: *float*, *alt*: *float*) → *float*

Calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat*: *ndarray*, *lon*: *ndarray*, *alt*: *ndarray*) → *ndarray*

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

get_anomaly(*lat: float, lon: float*) → float

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

get_anomaly_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

qnav.gravity.geosat**geosat.py****summary**

Geosat-44 reading, interpolation and gravity functions. This module contains all the gravity calculation functionalities related to the geosat-44 dataset. The Geosat44 database contains records of time-offsets, positions, geoid heights, gravity anomalies and calculation uncertainties.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.geosat.**Geosat44**(*base_model*: [GravityModel](#), *geoid_model*: [GeoidModel](#) | *None*, *map_dir*: *Path*, *auto_download*: *bool* = *True*, *auto_load*: *bool* = *True*)

Gravity Map implementation for the Geosat-44 dataset. Provides corrections for a given base gravity function, using the non-gridded geosat data values. Performs linear interpolation by producing a Delaunay triangulation mesh with fixed zero value for extrapolation.

calc_gravity_z(*lat*: *float*, *lon*: *float*, *alt*: *float*) → *float*

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat*: *ndarray*, *lon*: *ndarray*, *alt*: *ndarray*) → *ndarray*

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s^2).

Return type

np.array (N-elements)

get_map_value(*lat: float | ndarray, lon: float | ndarray*) → *float | ndarray*

Interpolates the gravity anomaly for requested position(s). Looks-up and returns the gravity anomaly value for requested latitude and longitude positions. Supports both scalar and array input.

Parameters

- **lat** (*float | np.ndarray*) – The latitude coordinate(s) in decimal degrees.
- **lon** (*float | np.ndarray*) – The longitude coordinate(s) in decimal degrees.

Returns

The interpolated gravity anomaly value(s) in m/s^2 .

Return type

float | np.ndarray

get_anomaly(*lat: float, lon: float*) → *float*

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s^2).

Return type

float

get_anomaly_vec(*lat: ndarray, lon: ndarray*) → *ndarray*

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s^2).

Return type

float

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

load_data()

Loads the Geosat data and generates internal interpolator for look-up. Called on initialisation to load the required data to allow for correction interpolation. Due to spacing between geosat records, linear interpolation is used for stability.

download_data(*overwrite: bool = False*)

Downloads the required Geosat-44 files from the source. If allowed and files are not present, will download the required geosat files to the set directory. Optionally, files can be overwritten.

Parameters

overwrite (*bool*) – Should downloading replace existing files. By default this is set to false and only missing files are downloaded.

qnav.gravity.geosat.read_geosat_file(*file_path: Path*) → dict[str, ndarray]

Reads and extracts the content of a given geosat file. Parses a given binary (.bin) geosat file and returns its records in a Python dictionary. The records include:

- **time_offset**: The time since the beginning of each pass (seconds)
- **latitude**: The latitude position of measurement (degrees)
- **longitude**: The longitude position of measurement

(degrees) • geoid: The geoid height from the ellipse (metres) • anomaly: The calculated gravity anomaly (m/s^2)
• uncertainty: The measurement uncertainty (m/s^2)

Parameters

file_path (*Path*) – The path for the binary geosat file to read.

Returns

A dictionary containing the read data records.

Return type

`dict[str, np.ndarray]`

`qnav.gravity.geosat.create_interp_grid(geosat_data: dict[str, np.ndarray], interp_method: str = 'linear', resolution: float = 0.1) → tuple[np.ndarray, np.ndarray, np.ndarray]`

Generates interpolated grid points for geosat data. For given geosat data, a set of grid points are generated using the requested interpolation method and resolution. This is provided to produced uniformly gridded maps for faster look-up and interpolation.

Parameters

- **geosat_data** (`dict[str, np.ndarray]`) – The contents read from the geosat file(s).
- **interp_method** (*str*) – (Optional) The interpolation method to use. Supported methods include 'nearest', 'linear' and 'cubic'. By default, the 'linear' method is used.
- **resolution** – (Optional) The requested step-size for the interpolation in decimal degrees. By default 0.1 is used.

Returns

A tuple containing the produced latitude, longitude and gravity anomaly values, respectively.

Return type

`tuple[np.ndarray, np.ndarray, np.ndarray]`

qnav.gravity.marine

marine.py

summary

Marine Gravity Map reading, interpolation and gravity functions. This module contains all the gravity calculation functionalities related to the marine gravity dataset. The Marine Gravity Map database contains records of positions and free-air gravity anomalies, uncertainties in anomalies and vertical gravity gradients. Data was last updated August 2022.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class `qnav.gravity.marine.MarineGravity(base_model: GravityModel, geoid_model: GeoidModel | None, map_dir: Path, auto_download: bool = True, auto_load: bool = True)`

Gravity Map implementation for the Marine Gravity dataset. Provides corrections for a given base gravity function, using the marine gravity map anomaly values.

calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s^2) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

get_map_value(*lat: float | ndarray, lon: float | ndarray*) → float | ndarray

Interpolates the gravity anomaly for requested position(s). Looks-up and returns the gravity anomaly value for requested latitude and longitude positions. Supports both scalar and array input.

Parameters

- **lat** (*float | np.ndarray*) – The latitude coordinate(s) in decimal degrees.
- **lon** (*float | np.ndarray*) – The longitude coordinate(s) in decimal degrees.

Returns

The interpolated gravity anomaly value(s) in m/s².

Return type

float | np.ndarray

get_anomaly(*lat: float, lon: float*) → float

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.

- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s^2).

Return type

float

get_anomaly_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s^2).

Return type

float

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

load_data()

Loads the Marine Gravity data and generates interpolator for look-up. Called on initialisation to load the required data to allow for correction interpolation.

download_data(overwrite: bool = False)

Downloads the required marine gravity files from the source. If allowed and files are not present, will download the required geosat files to the set directory. Optionally, files can be overwritten.

Parameters

overwrite (bool) – Should downloading replace existing files. By default this is set to false and only missing files are downloaded.

qnav.gravity.marine.read_marine_file(file_path: Path) → dict[str, ndarray]

This function extracts the data from a Marine gravity dataset. Marine gravity files are encoded in HDF5 scientific data format. Once successfully read, the gravity anomaly matrix, along with the latitude and longitude positions will be returned. The gravity values are in metres/seconds².

Parameters

file_path (Path) – The location of the Marine gravity dataset to read.

Returns

A dictionary containing the marine gravity map data, consisting of gravity anomaly, latitude and longitude values.

Return type

dict[str, np.ndarray]

qnav.gravity.ggm_plus**ggm_plus.py****summary**

Global Gravity Map Plus (GGMPlus) gravity map implementations. This module provides support for the GGMPlus gravity models. Specifically, two implementations for the GGPlus gravity acceleration and gravity disturbance corrections.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

class qnav.gravity.ggm_plus.GGMPlusAcc(base_model: GravityModel, geoid_model: GeoidModel | None, map_dir: Path, auto_download: bool = True, margin_size: int = 1)

Global Gravity Map Plus for Acceleration Correction. An implementation of GGMPlus, using the provided base gravity acceleration values. This overwrites the base values zero-altitude gravity estimation, but uses its altitude extrapolation.

calc_gravity_z(lat: float, lon: float, alt: float) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

get_anomaly(*lat: float, lon: float*) → float

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

get_anomaly_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s^2).

Return type

float

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s^2) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s^2).

Return type

float

get_map_value(*lat: float, lon: float*) → float

Interpolates the gravity acceleration for requested position(. Looks-up and returns the gravity anomaly value for requested latitude and longitude position.

Parameters

- **lat** (*float*) – The latitude coordinate in decimal degrees.
- **lon** (*float*) – The longitude coordinate in decimal degrees.

Returns

The interpolated gravity map value in m/s^2 .

Return type

float

get_map_values(*lat: ndarray, lon: ndarray*) → ndarray

Looks-up the correction values using the map. This will dynamically load data to cover the range, keeping the maximum number of loaded tiles under the margin value, to conserve memory constraints.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinate in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinate in decimal degrees.

Returns

The corresponding values interpolated from the map.

Return type

np.ndarray

download_data(*origin_lat: int, origin_lon: int, overwrite: bool = False*)

Downloads the required marine gravity files from the source. If allowed and files are not present, will download the required geosat files to the set directory. Optionally, files can be overwritten.

Parameters

- **origin_lat** (*int*) – Origin latitude of tile to download.
- **origin_lon** (*int*) – Origin longitude of tile to download.
- **overwrite** (*bool*) – Should downloading replace existing files. By default this is set to false and only missing files are downloaded.

```
class qnav.gravity.ggm_plus.GGMPlusDist(base_model: GravityModel, geoid_model: GeoidModel | None,  
                                         map_dir: Path, auto_download: bool = True, margin_size: int =  
                                         1)
```

Global Gravity Map Plus for Gravity Disturbance Correction. An implementation of GGMPlus, using the acquired gravity disturbance values values.

get_map_value(*lat: float, lon: float*) → float

Interpolates the gravity acceleration for requested position(. Looks-up and returns the gravity anomaly value for requested latitude and longitude position.

Parameters

- **lat** (*float*) – The latitude coordinate in decimal degrees.
- **lon** (*float*) – The longitude coordinate in decimal degrees.

Returns

The interpolated gravity map value in m/s^2.

Return type

float

get_map_values(*lat: ndarray, lon: ndarray*) → ndarray

Looks-up the correction values using the map. This will dynamically load data to cover the range, keeping the maximum number of loaded tiles under the margin value, to conserve memory constraints.

Parameters

- **lat** (*np.ndarray*) – The latitude coordinate in decimal degrees.
- **lon** (*np.ndarray*) – The longitude coordinate in decimal degrees.

Returns

The corresponding values interpolated from the map.

Return type

np.ndarray

calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

ReturnsThe gravity disturbance value (in m/s²).**Return type**

float

qnav.gravity.ggm_plus.read_ggm_file(*file_path: Path*) → ndarray

Reads the data from a given GGM_plus file. Once read, the returned 2500-by-2500 matrix data will be in *metres/second*². No latitude or longitude coordinates are contained within any GGM_plus file.

Parameters**file_path** (*Path or str*) – The location of the GGM_plus file to read.**Returns**The gravitational acceleration matrix read from the file, measured in *metres/second*²**Return type**

NumPy Matrix (2500-by-2500 elements)

qnav.gravity.ggm_plus.get_empty_ggm_tile() → ndarray

Produces an empty tile of dimensions of GGM Plus data file. Used when data is unavailable and cannot be downloaded.

Returns

Empty GGM Plus tile data for missing files.

Return type

NumPy Matrix (2500-by-2500 elements)

qnav.gravity.srtm2gravity**srtm2gravity.py****summary**

SRTM2Gravity gravity map implementations. This module provides support for the SRTM2Gravity gravity models. Specifically, the full-scale and residual gravity maps.

authorMichael Wright - mjwright@liverpool.ac.uk**version**

1.0.0 - First implementation of this module.

```
class qnav.gravity.srtm2gravity.SRTM2GravityFull(base_model: GravityModel, geoid_model:  
GeoidModel | None, map_dir: Path, auto_download:  
bool = True, margin_size: int = 1)
```

Implementation of SRTM2Gravity Full-Scale gravity. Provides a gravity correction map that uses the SRTM2Gravity Full-Scale dataset to provide gravity disturbance values.

```
class qnav.gravity.srtm2gravity.SRTM2GravityRes(base_model: GravityModel, geoid_model:  
GeoidModel | None, map_dir: Path, auto_download:  
bool = True, margin_size: int = 1)
```

```
qnav.gravity.srtm2gravity.read_srtm_file(file_path: Path) → ndarray
```

Reads the data from a given SRTM2Gravity file. Once read, the returned 1200-by-1200 matrix data will be in *metres/second*². No latitude or longitude coordinates are contained within any SRTM2Gravity file.

Parameters

file_path (*Path* or *str*) – The location of the GGM_plus file to read.

Returns

The gravity disturbance matrix read from the file, measured in *metres/second*²

Return type

NumPy Matrix (1200-by-1200 elements)

```
qnav.gravity.srtm2gravity.get_empty_srtm_tile() → ndarray
```

Produces an empty tile of dimensions of GGM Plus data file. Used when data is unavailable and cannot be downloaded.

Returns

Empty GGM Plus tile data for missing files.

Return type

NumPy Matrix (1200-by-1200 elements)

qnav.gravity.irish

```
class qnav.gravity.irish.IrishSeaGravity(base_model: GravityModel, geoid_model: GeoidModel |  
None, map_dir: Path | None)
```

load_data()

Loads the Marine Gravity data and generates interpolator for look-up. Called on initialisation to load the required data to allow for correction interpolation.

```
get_map_value(lat: float | ndarray, lon: float | ndarray) → float | ndarray
```

Interpolates the gravity anomaly for requested position(s). Looks-up and returns the gravity anomaly value for requested latitude and longitude positions. Supports both scalar and array input.

Parameters

- **lat** (*float* | *np.ndarray*) – The latitude coordinate(s) in decimal degrees.
- **lon** (*float* | *np.ndarray*) – The longitude coordinate(s) in decimal degrees.

Returns

The interpolated gravity anomaly value(s) in m/s².

Return type

float | np.ndarray

calc_gravity_z(*lat: float, lon: float, alt: float*) → float

Calculates and returns the total gravitational acceleration. Computes the gravity along the Z/Down axis (in m/s²) for the given location and altitude offset.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*float*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location.

Return type

float

calc_gravity_z_vec(*lat: ndarray, lon: ndarray, alt: ndarray*) → ndarray

In a vectorised batch, calculates and returns the gravitational acceleration along the Z/Down axis (in m/s²) for the given location and altitude offset. Ideal when calculating gravity for a large number of positions.

Parameters

- **lat** (*np.array*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*np.array*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.
- **alt** (*np.array*) – The altitude offset for the position of interest. The elevation from the WGS-84 reference ellipsoid in metres.

Returns

The downwards gravitational acceleration for each location (in m/s²).

Return type

np.array (N-elements)

get_anomaly(*lat: float, lon: float*) → float

Calculates the gravity anomaly for given position. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

get_anomaly_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity anomaly for multiple given positions. Retrieves the gravity anomaly value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity anomaly value (in m/s²).

Return type

float

get_disturbance(*lat: float, lon: float*) → float

Calculates the gravity disturbance for given position. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

get_disturbance_vec(*lat: ndarray, lon: ndarray*) → ndarray

Calculates the gravity disturbance for multiple given positions. Retrieves the gravity disturbance value for the requested position (in m/s²) at the base surface. For gravity maps this value is interpolated from its dataset, following possible conversion.

Parameters

- **lat** (*float*) – The latitude coordinate for the position of interest. The latitude position value given in decimal degrees.
- **lon** (*float*) – The longitude coordinate for the position of interest. The longitude position value given in decimal degrees.

Returns

The gravity disturbance value (in m/s²).

Return type

float

qnav.gravity.irish.read_irish_gravity_file(*data_file: Path, null_value: float = -99999*) → ndarray

Extracts the data from a given Irish Sea Gravity Map data file, using values read from its corresponding header file. On succession, the read gravitational anomaly matrix will be returned (measured in m/s²). No axis information is contained within data files.

Parameters

- **data_file** (*str or Path*) – The location of the Irish Sea Gravity Map data file.
- **null_value** (*float*) – Data values less than or equal to this value will be converted to NaN values (default=-99999).

Returns

The extracted gravitational anomaly matrix (measured in m/s^2).

Return type

2D NumPy Array

`qnav.gravity.irish.read_irish_ers_file(header_file) → dict[str, any]`

This function will read a given Irish Sea Gravity Map header file (a file normally with the extension ‘.ers’) and extract the relevant information from it.

Parameters

header_file (*str or Path*) – The location of the header corresponding file.

Returns

A dictionary holding the extracted northings and eastings origin coordinates, data dimensions, and cell spacing information.

1.9.6 Input

Modules related to reading in input data:

`qnav.input.config`

`config.py`

summary

Obtains primary simulation components from configuration file. This module is used for initialising and obtaining the main components used in simulations and for logging details to the console.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implemented version.

`qnav.input.config.get_configuration(config_file: Path = None) → ConfigHandler`

Initialises and returns a `qnav.input.config.ConfigHandler` instance using the contents of the given file. On succession, the file’s variables, along with those in any sub-configurations, will be parsed and loaded. If not file is given, a default settings file will be used.

Parameters

config_file (*Path*) – A path for a configuration (.ini) file to use.

Returns

An initialised `ConfigHandler` instance.

Return type

ConfigHandler

`qnav.input.config.get_trajectory(config: ConfigHandler) → Trajectory`

Extracts values and produces ground truth trajectories. When called, this function reads the required variables from the given configuration and uses them to generate a trajectory. On succession, this is return ready for use in a simulation.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

The trajectory ready for use in a simulation.

Return type

Trajectory

`qnav.input.config.get_estimation(config: ConfigHandler, trajectory: Trajectory) → EstimatedState`

Extracts values and produces the initial estimation state. When called, this function reads the required variables from the given configuration and uses them to initialise the starting estimation information. This uses the first record from the previously generated trajectory and induces any requested errors. On succession, an initial estimated state is return.

Parameters

- **config** ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.
- **trajectory** ([Trajectory](#)) – The trajectory to be used during the simulation. Required to obtain the true state at the first time step.

Returns

The initial estimated state to use in the simulation.

Return type

EstimatedState

`qnav.input.config.get_platform_hardware(config: ConfigHandler, estimated_state: EstimatedState) → list[SensorFusion]`

Extracts values and produces the virtual platform hardware instances. When called, this function reads the required variables from the given configuration and uses them initialize and return the requested Sensor-Fusion instance to use during the simulation. On succession, a list of SensorFusion classes to use in a simulation are returned.

Parameters

- **config** ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.
- **estimated_state** ([EstimatedState](#)) – The initial estimates before start of simulation. Required as some fusion methods need an initial estimate to start with.

Returns

The list of SensorFusion instances to use in a simulation.

Return type

[list](#)[[SensorFusion](#)]

`qnav.input.config.get_platform_clock(config: ConfigHandler) → Clock`

Extracts values and produces a virtual platform clock instances. When called, this function reads the required variables from the given configuration and uses them to initialise and return the platform clock to use.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

The Clock instance to use in a simulation.

Return type

Clock

`qnav.input.config.get_collector(config: ConfigHandler) → ResultsCapture`

Extracts values and produces the results collection instance to use. When called, this function reads the required variables from the given configuration and uses them to initialise and return a results collection instance, responsible for filtering and collecting simulation results.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

The ResultsCapture instance to use in a simulation.

Return type

ResultsCapture

`qnav.input.config.get_displayer(config: ConfigHandler) → Display`

Extracts values and produces the displaying instance to use. When called, this function reads the required variables from the given configuration and uses them to initialise and return a display instance, responsible for feeding back real-time information about the simulation. Can also be used for stopping the simulation prematurely when criteria is met.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

The Display instance to use in a simulation.

Return type

[Display](#)

`qnav.input.config.get_results_settings(config: ConfigHandler) → dict`

Extracts values and produces the results generation settings. When called, this function reads the required variables from the given configuration and uses them to define how results will be gathered after the simulation and exported.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

A dictionary containing settings for exporting results.

Return type

dict

`qnav.input.config.get_figure_settings(config: ConfigHandler) → dict`

Extracts values to use for results figure and report generation. When called, this function reads the required variables from the given configuration and returns them in a dictionary. These can then be used later for generating the relevant after-simulation graphics and report results.

Parameters

config ([ConfigHandler](#)) – The ConfigHandler instance to use for simulation settings.

Returns

Dictionary containing figure generation settings.

Return type

dict

`qnav.input.config.print_banner(text: str = None)`

Used to print formatted heading in the output console. Simply used to group list points together cleanly.

Parameters

text (*str*) – (Optional) The text to include in the heading. If not provided a dividing line will be used.

`qnav.input.config.print_line(field: str, value: any)`

Used to print formatted lines in the output console. Simply used to produce clean uniformly spaced list points.

Parameters

- **field** (*str*) – The text to place on the left-hand side.
- **value** (*any*) – The value to place on the right-hand side.

qnav.input.config_handler**config_handler.py****summary**

This class provides a wrapper for Python's ConfigHandler class. This supplies additional functionality such as automatically reading from sub-configuration files and providing platform independent means for extracting configuration values in usable formats.

authors

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of class.

class `qnav.input.config_handler.ConfigHandler`(*config_file: Path = PosixPath('default_settings.ini')*)

This class provides means for reading the user's configuration preferences from a given configuration file. Once read, values can be extracted within various formats (strings, booleans, floats, arrays, etc..). This class is essentially a wrapper for Python's ConfigHandler class, providing additional functionality such as the ability to split configuration across multiple files.

get_config_name() → *str*

This simple function returns the name of the configuration file being used. This function will automatically remove the project database_dir from the returned name (if the selected config is located within the project database_dir).

Returns

The short name of the selected configuration file that is being used.

Return type

str

load_configuration()

This function reads and records the content of the given configuration file (and expected sub-files). Expected sections/sensors are automatically searched and their sub-files are also included within the loading process. This function can be used to re-load/re-read the information from configuration files (if changes are made during runtime).

append_config(*config_file: str, from_section: str, to_section: str, to_overwrite: bool = False*)

This function will read and record the values of an external configuration files (i.e. a sub-configuration) and merge the read information with the current ConfigHandler instance.

Parameters

- **config_file** (*str*) – The sub-configuration file to read.

- **from_section** (*str*) – The section within the sub-configuration file to read and record.
- **to_section** (*str*) – The section within the current instance to append the read information to (can be a new section).
- **to_overwrite** (*bool*) – If set to true, previous existing values from the main configuration will be overwritten by those read from sub-configuration files (default=False).

expand_section(*section: str, path_property: str, sub_file_section: str = None, to_overwrite: bool = False*)

Expand section by importing the referenced sub-configuration path. Imports additional properties into section, for a given section and property referencing the sub-configuration file.

Parameters

- **section** (*str*) – The section containing the property with the sub-configuration path.
- **path_property** (*str*) – The property containing the sub- configuration path.
- **sub_file_section** (*str*) – (Optional) The section to import from the sub-configuration file.
- **to_overwrite** (*bool*) – Should this import overwrite existing settings?

quick_append_config(*from_section: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes], from_variable: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes], external_section: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes], to_overwrite: bool = False*)

This function simplifies merging multiple configuration file sections to the current configuration.

Parameters

- **from_section** (*numpy.typing.ArrayLike*) – The sections within the main configuration file that contain the variables to read from.
- **from_variable** (*numpy.typing.ArrayLike*) – The filepath variable names from the given section stating the external configuration files to read from.
- **external_section** (*numpy.typing.ArrayLike*) – The external sub-configuration file sections to merge with the main configuration.
- **to_overwrite** (*bool*) – If set to true, previous existing values from the main configuration will be overwritten by those read from sub-configuration files (default=False).

get_all_values() → ndarray

This function returns a matrix containing all the values read from the given configuration file(s). This function is namely for debugging purposes. Values are displayed as raw strings.

Returns

An N-by-3 matrix of values containing the section, variable name and value for each option read.

Return type

numpy.ndarray

get_sections()

This function returns a list of all the sections read from the given configuration file(s). This is useful for debugging and ensure required content is being correctly read.

Returns

A list containing all section names.

Return type

list

get_str(*section: str, name: str, fallback: str = None*) → str

This function will return the string value for a parameter in the selected configuration file, identified by its section and name. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*str*) – (Optional) The default value to use if not found.

Returns

Returns either the string found within the config file for the given section and name, or the default value if not found.

Return type

str

get_str_alpha(*section: str, name: str, fallback: str = None*) → str

This function will return the alphanumeric string value for a parameter in the selected configuration file, identified by its section and name. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*str*) – (Optional) The default value to use if not found.

Returns

Returns either the alphanumeric parts of the string found within the config file for the given section and name, or the default value if not found.

Return type

str

get_int(*section: str, name: str, fallback: int = None*) → int

This function will return the integer value for a parameter in the selected configuration file, identified by its section and name. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*int*) – (Optional) The default value to use if not found.

Returns

Returns either the integer value found within the config file for the given section and name, or the default value if not found.

Return type

int

get_float(*section: str, name: str, fallback: float = None*) → float

This function will return the float value for a parameter in the selected configuration file, identified by its section and name. This function returns a numpy 64-bit float instead of a normal float. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*float*) – (Optional) The default value to use if not found.

Returns

Returns either the float value found within the config file for the given section and name, or the default value if not found.

Return type

float

get_bool(*section: str, name: str, fallback: bool = None*) → bool

This function will return the boolean value for a parameter in the selected configuration file, identified by its section and name. Acceptable True values are ‘yes’, ‘true’, ‘on’ and ‘1’. Acceptable False values are ‘no’, ‘false’, ‘off’ and ‘0’. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*bool*) – (Optional) The default value to use if not found.

Returns

Returns either the boolean found within the config file for the given section and name, or the default value if not found.

Return type

bool

get_csv_numeric(*section: str, name: str, fallback: Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes] = None, sep: str = ','*) → Buffer | _SupportsArray[dtype[Any]] | _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int | float | complex | str | bytes | _NestedSequence[bool | int | float | complex | str | bytes] | None

This function will return an array of values that are separated by a common character (comma by default) from the selected configuration file, identified by its section and name. This function returns a numpy array. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*list*) – (Optional) The default value(s) to use if not found.
- **sep** – (Optional) The item separator character (default=',').

Type

str

Returns

A numpy array containing the written items.

Return type

list

get_csv_str(*section: str, name: str, fallback: list = None, sep: str = ','*) → list | None

This function will return an array of string values that are separated by a common character (comma by default) from the selected configuration file, identified by its section and name. This function returns a list of one or more character values. If the value is missing, an optional default value can be used instead.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*list | None*) – (Optional) The default value(s) to use if not found.
- **sep** – (Optional) The item separator character (default=',').

Type

str

Returns

A Python list containing the written items.

Return type

list | None

get_value_filepath(*section: str, name: str, fallback: str | Path = None, must_exist: bool = False*) → Path

This function will return a local filepath read for a parameter in the selected configuration file, identified by its section and name. This function will handle read paths in a platform independent manner, automatically deciding and converting when needed. This allows paths read from the configuration file to be written in either Linux/Unix or Windows/NT format without affecting behaviour on different platforms.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*str*) – (Optional) The default value to use if not found.
- **must_exist** – (Optional) Should the path exist on the filesystem? If enabled an error will raise if the given path is not found on the system (default=True).

Returns

A valid filepath for the current system to the given location/resources.

Return type

Path

get_value_filepath_str(*section: str, name: str, fallback: str = None*)

This function will return a local filepath read for a parameter in the selected configuration file, identified by its section and name. This function will handle read paths in a platform independent manner, automatically deciding and converting when needed. This allows paths read from the configuration file to be written in either Linux/Unix or Windows/NT format without affecting behaviour on different platforms.

Unlike 'get_value_filepath' this function does not enforce the existence of a requested file/database_dir.

Parameters

- **section** (*str*) – The section of the configuration file.
- **name** (*str*) – The name of the variable to read from the given section.
- **fallback** (*str*) – (Optional) The default value to use if not found.

Returns

A filepath for the current system to the given location/ resources. This resource does not have to exist.

Return type

Path

is_value_set(*section: list, value: list*) → bool

Returns true if there is an entry for (all of) the given sections and values. This function works for both single values and lists of values. For multiple values, both lists must be of the same length.

Parameters

- **section** (*list or str*) – A single or list of sections names.
- **value** – A single or list of value names.

Returns

True if an entry could be found in the provided configuration for all the given section-list pairs. False if one or more entries could not be found.

Return type

bool

overwrite_value(*section: str, name: str, new_value*)

This simple function can be used to overwrite or insert any value extracted from the given configuration file. This is only to be used for automatically generated values and is not an alternative to fallbacks.

Parameters

- **section** (*str*) – The section name of the variable to override
- **name** (*str*) – The name of the variable to read from the given section.
- **new_value** (*str, int, float*) – The new value to use for the given section and name.

copy_file(*dest: Path*)

Copies the configuration file to a destination. This method produces a copy of the configuration file at the requested destination.

Parameters

dest (*Path*) – The location to write the file to.

write_file(*dest: Path*)

Writes the configuration values to a file. This method writes the configuration values in their current state (including modifications) to a requested file. The produced file can then be re-used as a settings file to repeat experimentation.

Parameters

dest (*Path*) – The location to write the file to.

exception `qnav.input.config_handler.NavConfigWarning`

A custom warning type used to inform the user about invalid, unsuitable or missing values present in the configuration file. If these warnings generated can be safely ignored, this warning type can be suppressed to hide all

configuration file warnings for the session. Suppressing this warning type will not affect the displaying of other warnings.

exception `qnav.input.config_handler.NavConfigError`(*section_name: str, variable_name: str, error_type: str, error_desc: str*)

A custom exception type used for inform the user about erroneous values present in the provided configuration file that cannot be ignored. This includes syntax, value type and conflicting value errors. This exception type is primarily used for formatting the errors being displayed. This custom class also allows configuration based errors to be distinguished from serious errors such faults not detected until after delivery.

1.9.7 Measurement

Modules related to standard sensors for measurements:

qnav.measurement.sensor

sensor.py

summary

Defines the abstract base classes for sensor usage in simulations. This module contains the base classes for both sensors (used for taking GroundTruth records and producing measurements) and sensor fusion (methods for applying measurement information to the current estimated states).

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

class `qnav.measurement.sensor.Sensor`(*frequency: float, start_time: float = 0.0, rand_seed: int = None*)

Base class for sensors used in navigation. Defines the format of sensors used in the toolbox, specifically the methods, inputs and outputs expected for fusion and other classes.

abstract `take_measurement`(*est_time: float, ground_truth: GroundTruth*)

Produces the measurement captured by the sensor. Given the current estimated time and the ground truth, this method shall return the measurement captured at that time. Alternatively, a None value can be returned to signify a measurement is not ready.

Parameters

- **est_time** (*float*) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** (*GroundTruth*) – The corresponding ground truth data.

abstract property `last_measurement: any`

Returns the latest recorded measurement produced by sensor.

Returns

Latest measurement produced by sensor.

Return type

any

update(*time_sec: float = None*)

Called on demand to update the sensor's state. Typically called after measurements have been taken. When called this increases the next measurement time and propagates internal states, to the time difference.

Parameters

time_sec (*float*) – The current simulation time in seconds.

property last_update: float

Returns the time (in seconds) of the last measurement.

Returns

Time of last measurement (in seconds).

Return type

float

property next_update: float

Returns the due time (in seconds) of the next measurement.

Returns

Time of next measurement (in seconds).

Return type

float

property frequency: float

Returns the sensor's measurement frequency in Hz.

Returns

The frequency in Hz.

Return type

float

property time_step: float

Returns the sensor's measurement time step in seconds.

Returns

The measurement interval in seconds.

Return type

float

class qnav.measurement.sensor.**FusionTrigger**(*value, names=<not given>, *values, module=None, qualname=None, type=None, start=1, boundary=None*)

Flags used to indicate the triggering mode for fusion. These essentially define when fusion is triggered, subject to the sensor measurements that are available.

class qnav.measurement.sensor.**SensorFusion**(*trigger: FusionTrigger, *sensors: Sensor*)

Base class for a sensor fusion method used in navigation. Defines the format of data fusion used in the toolbox, specifically the methods, inputs and outputs expected.

property trigger: FusionTrigger

The selected trigger method for deciding when to apply fusion. This decides when fusion can be applied, subject to the available sensor measurements.

Returns

The selected fusion trigger method.

Return type

FusionTrigger

property sensors: Iterable[Sensor]

The provided sensors associated with the fusion method. This is a collection of all shared sensors whose measurements will be used by the fusion method.

Returns

The shared sensors used by the fusion method.

Return type

Iterable[*Sensor*]

abstract perform_fusion(*estimated_state*: *EstimatedState*) → None

Performs the fusion with sensors and updates the estimated state. When called at scheduled time during a simulation loop, this method performs the data fusion by extracting last measurements from sensors and using them with the fusion algorithm to update the given current estimated state.

Parameters

estimated_state (*EstimatedState*) – The current estimated states of the platform. This is what the fusion method will update.

qnav.measurement.platform**platform.py****summary**

Provides conversion to and from sensor reference frame. This module is responsible for representing the orientation and location of sensor's on a platform body, providing means for common conversion between the reference frames.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.measurement.platform.SensorAxis(sensor_angles: Buffer | _SupportsArray[dtype[Any]] |
                                           _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
                                           float | complex | str | bytes | _NestedSequence[bool | int |
                                           float | complex | str | bytes] = array([0., 0., 0.]), lever_arm:
                                           Buffer | _SupportsArray[dtype[Any]] |
                                           _NestedSequence[_SupportsArray[dtype[Any]]] | bool | int |
                                           float | complex | str | bytes | _NestedSequence[bool | int |
                                           float | complex | str | bytes] = array([0., 0., 0.]
```

Specifies the orientation and offset of sensors relative to platform. For a corresponding sensor, this class specifies the angle orientation and offset location of the sensor on the platform body. This supplies a consistent practice for local conversion between sensor and platform reference frames.

property sensor_angles_deg: ndarray

Returns the sensor orientation angles in degrees.

Returns

Sensor angles (in degrees).

Return type

np.ndarray (3-elements)

property sensor_angles_rad: ndarray

Returns the sensor orientation angles in radians.

Returns

Sensor angles (in radians).

Return type

np.ndarray (3-elements)

property lever_arm: ndarray

The lever arm position offset in metres.

Returns

Lever arm offset (in metres).

Return type

np.ndarray (3-elements)

property body2sensor_mat: ndarray

The rotation matrix for body to sensor conversion.

Returns

The rotation matrix for reference frame conversion.

Return type

np.ndarray (3-by-3 elements)

qnav.measurement.error_properties

error_properties.py

summary

Contains representations for sensor error profiles. Module contains classes that are used for profiling the common error types represented by various sensor types. Namely used to reduce repeated common class arguments and methods.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```

class qnav.measurement.error_properties.ErrorProperties(bias_error: float | Buffer |
    _SupportsArray[dtype[Any]] | _NestedSe-
    quence[_SupportsArray[dtype[Any]]] |
    bool | int | complex | str | bytes |
    _NestedSequence[bool | int | float | complex
    | str | bytes] = 0, bias_drift_rate: float |
    Buffer | _SupportsArray[dtype[Any]] |
    _NestedSe-
    quence[_SupportsArray[dtype[Any]]] |
    bool | int | complex | str | bytes |
    _NestedSequence[bool | int | float | complex
    | str | bytes] = 0, scale_error: float | Buffer
    | _SupportsArray[dtype[Any]] | _NestedSe-
    quence[_SupportsArray[dtype[Any]]] |
    bool | int | complex | str | bytes |
    _NestedSequence[bool | int | float | complex
    | str | bytes] = 0, non_orth_error: float |
    Buffer | _SupportsArray[dtype[Any]] |
    _NestedSe-
    quence[_SupportsArray[dtype[Any]]] |
    bool | int | complex | str | bytes |
    _NestedSequence[bool | int | float | complex
    | str | bytes] = 0, avg_meas_noise: float |
    Buffer | _SupportsArray[dtype[Any]] |
    _NestedSe-
    quence[_SupportsArray[dtype[Any]]] |
    bool | int | complex | str | bytes |
    _NestedSequence[bool | int | float | complex
    | str | bytes] = 0)

```

Defines common error properties used for 3D sensors. This data class contains the errors associated with 3D sensors; specifically, the bias error, bias drift rate, scaling errors, non-orthogonality and measurement noise properties. These are commonly used when defining sensors such as accelerometer and gyroscope, along with their quantum counterparts.

property bias_error: ndarray

The bias errors for each axis (in micro-g).

Returns

Sensor bias errors (in micro-g).

Return type

np.ndarray (3-elements)

property bias_drift_rate: ndarray

The bias drift rate for each axis (in micro-g/root sec).

Returns

Bias drift rate (in micro-g/root sec).

Return type

np.ndarray (3-elements)

property scale_error: ndarray

The scaling errors for each axis (in ppm).

Returns

Scaling errors (in ppm).

Return type

np.ndarray (3-elements)

property non_orth_error: ndarray

The non-orthogonal “cross-coupling” errors (in micro-g). Specifically the alignment errors for axis xy, xz, yx, yz, zx and zy respectively.

Returns

non-orthogonal errors (in micro-g).

Return type

np.ndarray (6-elements)

property avg_meas_noise: ndarray

The average measurement error for each axis (in micro-g.root Hz).

Returns

average measurement error (in micro-g.root Hz).

Return type

np.ndarray (3-elements)

property error_matrix: ndarray

A matrix containing the scale factor and cross-coupling errors for a nominally orthogonal sensor. This is a commonly used representation. For scale errors S and non-orthogonal errors M, this is formatted as: [[Sx, Mxy, Mxz],

[Myx, Sy, Myz], [Mzx, Mzy, Sz]]

Returns

Error matrix for scale factor and cross-coupling errors.

Return type

np.ndarray (3-by-3 elements)

qnav.measurement.accelerometer**accelerometer.py****summary**

Contains the virtual accelerometer classes for producing measurements. This module contains virtual accelerometer classes that can be used for generating accelerometer measurements, from a simulated environment, with a configured level of noise/errors.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.measurement.accelerometer.Accelerometer(frequency: float, error_profile:  
                                                    ~qnav.measurement.error_properties.ErrorProperties,  
                                                    sensor_axis:  
                                                    ~qnav.measurement.platform.SensorAxis =  
                                                    <qnav.measurement.platform.SensorAxis object>,  
                                                    start_time: float = 0.0, rand_seed: int = None)
```


Provides the functionality of an accelerometer in a simulated environment. This class essentially represents a virtual accelerometer converting waypoint accelerations to body rate axis and applying noise/errors. The output of this can then be passed to a selected INS fusion class.

take_measurement (*est_time: float, ground_truth: GroundTruth*)

Produces an accelerometer measurement from given truth data. This method converts the given waypoint data to body axes and applies the configured errors/noise. The output can then be fed to the selected INS solution.

Parameters

- **est_time** (*float*) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** (*GroundTruth*) – The corresponding ground truth data.

property last_measurement: *ndarray*

Returns the latest recorded acceleration produced by sensor. The measured acceleration of the accelerometer (North, East and Down components in metres/second).

Returns

Latest acceleration measurement produced by sensor.

Return type

np.ndarray (3-elements)

update (*time_sec: float = None*)

Called every iteration to update simulated sensor errors. As sensor errors propagate with time, it is important that this method is called prior to every measurement update.

Parameters

- **time_sec** (*float*) – The current simulation time in seconds.

property sensor_axis: *SensorAxis*

Returns the true axis for the accelerometer.

Returns

The sensor axis of the accelerometer.

Return type

SensorAxis

qnav.measurement.gyroscope

gyroscope.py

summary

Contains the virtual gyroscope classes for producing measurements. This module contains virtual gyroscope classes that can be used for generating gyroscope measurements, from a simulated environment, with a configured level of noise/errors.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.measurement.gyroscope.Gyroscope(frequency: float, error_profile:
    ~qnav.measurement.error_properties.ErrorProperties,
    sensor_axis: ~qnav.measurement.platform.SensorAxis =
    <qnav.measurement.platform.SensorAxis object>,
    start_time: float = 0.0, rand_seed: int = None)
```

Provides the functionality of a gyroscope in a simulated environment. This class essentially represents a virtual gyroscope converting waypoint angle rates to body rate axis and applying noise/errors. The output of this can then be passed to a selected INS solution.

take_measurement(*est_time*: float, *ground_truth*: [GroundTruth](#))

Produces a gyroscope measurement from given truth data. This method converts the given waypoint data to body axes and applies the configured errors/noise. The output can then be fed to the selected INS solution.

Parameters

- **est_time** (float) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** ([GroundTruth](#)) – The corresponding ground truth data.

property last_measurement: ndarray

Returns the latest recorded gyroscope produced by sensor. The measured angle rates of the gyroscope (P, Q and R angles in degrees/second).

Returns

Latest angle rates measurement produced by sensor.

Return type

np.ndarray (3-elements)

update(*time_sec*: float = None)

Called every iteration to update simulated sensor errors. As sensor errors propagate with time, it is important that this method is called prior to every measurement update.

Parameters

- **time_sec** (float) – The current simulation time in seconds.

property sensor_axis: [SensorAxis](#)

Returns the true axis for the gyroscope.

Returns

The sensor axis of the gyroscope.

Return type

[SensorAxis](#)

qnav.measurement.altimeter

altimeter.py

summary

Contains the virtual altimeter classes for producing measurements. This module contains virtual altimeter classes that can be used for generating altimeter measurements, from a simulated environment, with a configured level of noise/errors.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

class `qnav.measurement.altimeter.Altimeter`(*frequency: float, bias_error: float = 0.0, bias_drift_rate: float = 0.0, start_time: float = 0.0, rand_seed: int = None*)

Provides the functionality of an altimeter in a simulated environment. This class essentially represents a virtual altimeter that takes the altitude from the waypoint data and adds a configured noise/error. The output of this can then be passed to a selected INS solution.

take_measurement(*est_time: float, ground_truth: GroundTruth*)

Produces an altimeter measurement from given truth data. This method takes the true altitude (above sea level) and applies the configured errors/noise. The output can then be fed to the selected INS solution.

Parameters

- **est_time** (*float*) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** (*GroundTruth*) – The corresponding ground truth data.

property last_measurement: float

Returns the latest recorded altitude produced by the altimeter.

Returns

Latest altitude measurement produced.

Return type

float

update(*time_sec: float = None*)

Called on demand to update the altimeter's state. Typically called after measurements have been taken. When called this increases the next measurement time and propagates internal states, to the time difference.

Parameters

- **time_sec** (*float*) – The current simulation time in seconds.

qnav.measurement.clock

clock.py

summary

Contains the virtual clock/timer classes for estimating elapsed time. This module contains virtual clock classes that can be used for simulating recorded time (with errors and drift) within a simulated environment. This is namely to simulate clock errors within simulations.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

class `qnav.measurement.clock.Clock`(*clock_bias: float = 0, bias_drift_rate: float = 0, start_time: float = 0.0, rand_seed: int = None*)

Provides the functionality of a system clock in a simulated environment. Distinct from typical sensor hardware, the clock represents a virtual clock that simulated the recording of elapsed time with configured error drift.

update(*time_sec: float*)

Called every iteration to update clock measurement errors. As clock errors propagate with time, it is important that this method is called prior to every measurement update.

Parameters

time_sec (*float*) – The current simulation time in seconds.

get_time(*current_time: float*) → float

Returns the believed elapsed time since the start of the simulation. This is given in seconds (with considerations for rounding errors).

Parameters

current_time (*float*) – The true current time of the simulation.

Returns

The current time with clock errors applied (in seconds).

Return type

float

class qnav.measurement.clock.**DummyClock**(*start_time: float = 0.0*)

update(*time_sec: float*)

Called every iteration to update clock measurement errors. As clock errors propagate with time, it is important that this method is called prior to every measurement update.

Parameters

time_sec (*float*) – The current simulation time in seconds.

get_time(*current_time: float*) → float

Returns the believed elapsed time since the start of the simulation. This is given in seconds (with considerations for rounding errors).

Parameters

current_time (*float*) – The true current time of the simulation.

Returns

The current time with clock errors applied (in seconds).

Return type

float

1.9.8 Output

Modules related to exporting output data:

qnav.output.data

class qnav.output.data.**ResultsTable**(*file_path: Path, read_only: bool = False, auto_load: bool = True*)

Dynamic on-disk data container for results records. Provides a container that uses virtual-memory, storing the results on disk but still allowing fast querying and retrieval. This can be useful for a number of tasks, including conversion and report generation, without loading all data into memory.

property is_data_loaded: bool

States data has already been loaded.

Returns

Returns True if data is loaded.

Return type
bool

qnav.output.formats

class qnav.output.formats.**FileFormat**(*match_id: str, file_ext: str*)

Defines the fields for each supported file format. Included as a simpler way to represent the SaveFormat enum class.

class qnav.output.formats.**SaveFormat**(*value, names=<not given>, *values, module=None, qualname=None, type=None, start=1, boundary=None*)

A collection of the supported file formats for results table. Controllable flags for selecting the use of the supported file formats when exporting data. A useful way to efficiently export data in multiple formats, without employing multiple functions.

qnav.output.formats.**save_as_csv**(*file_path: Path, results: ResultsTable, down_sample: int = 1*)

Exports the given results table as a CSV file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the CSV data to.
- **results** (*ResultsTable*) – The results to convert to CSV.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

qnav.output.formats.**save_as_csv_comp**(*file_path: Path, results: ResultsTable, down_sample: int = 1*)

Exports the given results table as a compressed CSV file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the compressed CSV data to.
- **results** (*ResultsTable*) – The results to convert to compressed CSV.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

qnav.output.formats.**save_as_numpy**(*file_path: Path, results: ResultsTable, down_sample: int = 1*)

Exports the given results table as a numpy data file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the numpy data to.
- **results** (*ResultsTable*) – The results to convert to numpy save file.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

qnav.output.formats.**save_as_numpy_comp**(*file_path: Path, results: ResultsTable, down_sample: int = 1*)

Exports the given results table as a compressed numpy data file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the compressed numpy data to.
- **results** (*ResultsTable*) – The results to convert to compressed numpy save file.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

qnav.output.formats.**save_as_binary**(*file_path: Path, results: ResultsTable, down_sample: int = 1*)

Exports the given results table as a compressed numpy data file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the compressed numpy data to.
- **results** (*ResultsTable*) – The results to convert to compressed numpy save file.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

`qnav.output.formats.save_as_matlab(file_path: Path, results: ResultsTable, down_sample: int = 1)`

Exports the given results table as a (non-compressed) MATLAB data file.

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the MATLAB data to.
- **results** (*ResultsTable*) – The results to convert to MATLAB file.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

`qnav.output.formats.save_as_matlab_comp(file_path: Path, results: ResultsTable, down_sample: int = 1)`

Exports the given results table as a MATLAB data file (with compression).

Parameters

- **file_path** (*pathlib.Path*) – The file path to save the MATLAB data to.
- **results** (*ResultsTable*) – The results to convert to MATLAB file.
- **down_sample** (*int*) – Optional down-sampling factor to apply.

`qnav.output.formats.save_as(file_dir: Path, file_name: str, results: ResultsTable, down_sample: int = 1, *formats: SaveFormat)`

Saves the given results as one or format types.

Parameters

- **file_dir** (*pathlib.Path*) – The location to write results to.
- **file_name** (*str*) – The filename prefix to use.
- **results** (*ResultsTable*) – The results to convert and write.
- **down_sample** (*int*) – Optional down-sampling factor to apply.
- **formats** (*SaveFormat*) – The output formats to use.

qnav.output.figures

qnav.output.summary

1.9.9 Quantum

Modules relating to quantum sensor and fusion methods:

qnav.quantum.base**base.py****summary**

Provides basic concept quantum sensors and fusion methods. These serve as base models that more advanced and realistic implementations can extend from.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

```
class qnav.quantum.base.ConceptQuantumImu(full_frequency: float, imu_frequency: float,
                                           accelerometer_errors:
                                           ~qnav.measurement.error_properties.ErrorProperties,
                                           gyroscope_errors:
                                           ~qnav.measurement.error_properties.ErrorProperties,
                                           sensor_axis: ~qnav.measurement.platform.SensorAxis =
                                           <qnav.measurement.platform.SensorAxis object>, duty_cycle:
                                           float = 0.5, start_time: float = 0.0, rand_seed: int = None)
```

Concept Quantum IMU featuring use of quantum accelerometer and gyroscope. While not realistic in implementation, this base class produces measurements mimicking a quantum accelerometer and gyroscope, requiring multiple samples of the ground truth.

take_measurement (est_time: float, ground_truth: [GroundTruth](#))

Samples from the ground truth to collect data to produce measurements. Using the ground truth, captures the current acceleration and angles rates at the time. A series of these are required to produce a measurement before fusion.

Parameters

- **est_time** (float) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** ([GroundTruth](#)) – The corresponding ground truth data.

update (time_sec: float = None)

Called on demand to update the sensor's state. Typically called after measurements have been taken. When called this increases the next measurement time and propagates internal states, to the time difference.

Parameters

time_sec (float) – The current simulation time in seconds.

reset()

Resets the sensor, clearing all previous measurements. Normally used directly after fusion.

property last_measurement: **tuple**[ndarray, ndarray]

Returns the last acceleration and angle rate measurements captured by the quantum sensor, respectively.

Returns

The last measurements of the sensor.

Return type

tuple[np.ndarray, np.ndarray]

property num_active_steps: int

Returns the number of measurement steps required,

Returns

Number of measurements required.

Return type

int

property num_steps: int

Returns the number of total steps required,

Returns

Number of steps required for fusion.

Return type

int

property sensor_axis: *SensorAxis*

Returns the true sensor axis for the sensor.

Returns

The sensor axis used by the sensor.

Return type

SensorAxis

```
class qnav.quantum.base.ConceptQuantumFusion(qs_imu: ConceptQuantumImu, shared_accelerometer:
Accelerometer, shared_gyroscope: Gyroscope,
est_qs_imu_axis: SensorAxis = None, est_acc_axis:
SensorAxis = None, est_gyro_axis: SensorAxis = None)
```

A basic version of quantum fusion, namely used as a base class. This is the simplest implementation of position correction fusion for quantum IMU. Many of its methods can be extended and swapped out for more advanced implementations.

perform_fusion(*estimated_state: EstimatedState*) → None

Collects measurements and applies position fixing. Each cycle collects measurements from the typical IMU sensors and quantum sensor. After a set number of calls, this uses the single quantum measurement to calculate an approximate bias correction for the IMU. If successful, the position fixing is applied.

Parameters

estimated_state (*EstimatedState*) – The current estimated state to update.

reset()

Resets the state of the fusion, clearing internal temporary states. When call the fusion will return to expecting the first measurement, then continuing from there.

```
property sensors: (<class 'qnav.quantum.base.ConceptQuantumImu'>, <class
'qnav.measurement.accelerometer.Accelerometer'>, <class
'qnav.measurement.gyroscope.Gyroscope'>)
```

The provided sensors associated with the fusion method. This is a collection of all shared sensors whose measurements will be used by the fusion method.

Returns

The shared sensors used by the fusion method.

Return type

Iterable[*Sensor*]

qnav.quantum.realistic**realistic.py****summary**

Provides quantum sensors with realistic implementations. Extends from the base quantum sensor and includes realistic physics for capturing measurements.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

```
class qnav.quantum.realistic.ColdAtomInterferometer(num_of_atoms: int = 1000000.0, atom_mass:
                                                    float = 2.20693925e-25, sensor_length: float =
                                                    0.5, beam_width: float = 0.01, eta: float = 0.3,
                                                    time_pulse: float = 0.16, recoil_velocity: float =
                                                    0.007)
```

Parameters used for defining a Cold Atom Interferometer. These properties describe the qualities of an ultra-cold atom interferometer, which is used for capturing quantum accelerations and angle rates.

```
class qnav.quantum.realistic.QuantumIMU(full_frequency: float, imu_frequency: float,
                                         accelerometer_errors:
                                         ~qnav.measurement.error_properties.ErrorProperties,
                                         gyroscope_errors:
                                         ~qnav.measurement.error_properties.ErrorProperties,
                                         interferometer:
                                         ~qnav.quantum.realistic.ColdAtomInterferometer, sensor_axis:
                                         ~qnav.measurement.platform.SensorAxis =
                                         <qnav.measurement.platform.SensorAxis object>, duty_cycle:
                                         float = 0.5, start_time: float = 0.0, rand_seed: int = None)
```

Quantum IMU that uses an ultra-cold atom interferometer for measurements. This extends from the base concept quantum IMU by replacing the acquiring of measurement methods with realistic implementations. These feature

take_measurement (*est_time: float, ground_truth: [GroundTruth](#)*)

Samples from the ground truth to collect data to produce measurements. Using the ground truth, captures the current acceleration and angles rates at the time. A series of these are required to produce a measurement before fusion.

Parameters

- **est_time** (*float*) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** ([GroundTruth](#)) – The corresponding ground truth data.

reset()

Resets the sensor, clearing all previous measurements. Normally used directly after fusion.

qnav.quantum.realistic.**ang_diff**(*a: float, b: float*) → float

Calculate the difference between two angles (in radians).

Parameters

- **a** (*float*) – The first angle value.
- **b** (*float*) – The second angle value.

Returns

The difference between the given angles.

Return type

float

qnav.quantum.particle_filter**particle_filter.py****summary**

Particle filter implementations for quantum sensor fusion. This module contains fusion methods for quantum sensors that employ particle filter estimation.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

```
class qnav.quantum.particle_filter.ParticleFilterParams(num_particles: int = 200,
                                                         sigma_accelerometer: float = 0.01,
                                                         sigma_gyroscope: float = 0.01,
                                                         sigma_misalignment: float = 0.01,
                                                         rng_seed: int = None, low_noise_prob:
                                                         float = 0.9, low_noise: float = 0.1,
                                                         high_noise: float = 2.0)
```

Parameters for a particle filter fusion for quantum sensors. This contains the arguments used for applying particle filtering for position fixing with IMU quantum sensors. This namely contains the number of particles to use and the expected errors.

```
class qnav.quantum.particle_filter.QuantumPFFusion(qs_imu: ConceptQuantumImu,
                                                    shared_accelerometer: Accelerometer,
                                                    shared_gyroscope: Gyroscope, est_qs_imu_axis:
                                                    SensorAxis = None, est_acc_axis: SensorAxis =
                                                    None, est_gyro_axis: SensorAxis = None,
                                                    filter_params: ParticleFilterParams =
                                                    ParticleFilterParams(num_particles=200,
                                                    sigma_accelerometer=0.01,
                                                    sigma_gyroscope=0.01,
                                                    sigma_misalignment=0.01, rng_seed=None,
                                                    low_noise_prob=0.9, low_noise=0.1,
                                                    high_noise=2.0))
```

Quantum sensor fusion with particle filter estimation. This provides position correction by using a particle filter approach for estimating the bias errors within the class IMU sensors. This extends from the base class, overriding the relevant methods.

```
qnav.quantum.particle_filter.weighted_cov(y: ndarray, w: ndarray)
```

This function calculates a Weighted Covariance Matrix. Given a matrix of observations (rows) and variables (columns) along with an array of corresponding weights, this function returns a symmetric matrix of weighted covariances (a positive semi-definite matrix i.e. all its eigenvalues are non-negative).

Parameters

- **y** (*NumPy Array (T-by-N elements)*) – A T-by-N matrix of observations (rows) and variables (columns).

- **w** (*NumPy Array (T elements)*) – A T-element array of weights for the observations.

Returns

A symmetric matrix of weighted covariances.

Return type

NumPy Array

qnav.quantum.misalignment**misalignment.py****summary**

Extended implementations of Quantum Sensor Fusion for axis misalignment. This module contains fusion methods for quantum sensors that, in addition to position fixing, also correct axis misalignment, resulting in errors due to non-matching actual and estimated sensor axis.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

```
class qnav.quantum.misalignment.QuantumPFMFusion(qs_imu: ConceptQuantumImu,
                                                shared_accelerometer: Accelerometer,
                                                shared_gyroscope: Gyroscope, est_qs_imu_axis:
SensorAxis = None, est_acc_axis: SensorAxis =
None, est_gyro_axis: SensorAxis = None,
filter_params: ParticleFilterParams =
ParticleFilterParams(num_particles=200,
sigma_accelerometer=0.01, sigma_gyroscope=0.01,
sigma_misalignment=0.01, rng_seed=None,
low_noise_prob=0.9, low_noise=0.1,
high_noise=2.0))
```

Quantum sensor fusion with particle filter estimation for axis misalignment. This provides position and axis misalignment correction by using a particle filter approach for estimating the bias errors within the class IMU sensors. This extends from the base class, overriding the relevant methods.

qnav.quantum.gravity_gradient**gravity_gradient.py****summary**

Provides implementation for Quantum gravity gradient sensing and fusion. Implementation is mainly based on Peters et al, 'High precision gravity measurements using atom interferometry', Metrologia, Vol.38, pp.25-61 (2001).

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of class.

```
class qnav.quantum.gravity_gradient.GravityGradInterferometer(num_of_atoms: int = 200000,
                                                             atom_mass: float =
                                                             1.4431608262e-25, recoil_velocity:
                                                             float = 0.0058845,
                                                             stationary_velocity: float = 1.57,
                                                             sensor_z_top: float = 0.0,
                                                             sensor_z_bottom: float = -1.0, eta:
                                                             float = 0.25, time_pulse: float =
                                                             0.2, sigma_phi: float = 0.025,
                                                             sigma_n: float = 1.0, beam_width:
                                                             float = 0.01, sensor_length: float =
                                                             1.5, rand_failure_prob: float =
                                                             0.0)
```

Parameters used by the Cold Atom Gravity Gradient Interferometer. These properties describe the qualities of an ultra-cold atom interferometer, which is used for capturing gravity gradient estimations from the local environment.

property k_eff: float

Effective wave number, the number of wave cycles per unit distance. A property provided for a value commonly calculated.

Returns

A measure of the spatial frequency of a wave.

Return type

float

```
class qnav.quantum.gravity_gradient.GravityGradiometer(full_frequency: float, imu_frequency: float,
                                                       accelerometer_errors:
                                                       ~qnav.measurement.error_properties.ErrorProperties,
                                                       interferometer:
                                                       ~qnav.quantum.gravity_gradient.GravityGradInterferometer,
                                                       gravity_model:
                                                       ~qnav.gravity.base.GravityModel,
                                                       sensor_axis:
                                                       ~qnav.measurement.platform.SensorAxis =
                                                       <qnav.measurement.platform.SensorAxis
                                                       object>, duty_cycle: float = 0.5, start_time:
                                                       float = 0.0, rand_seed: int = None)
```

Quantum Gravity Gradiometer Sensor for estimating vertical gravity gradient. A Quantum sensor that features two ultra-cold atom accelerometers, measuring the change in the vertical gravity gradient.

take_measurement(est_time: float, ground_truth: [GroundTruth](#))

Samples from the ground truth to collect data to produce measurements. Using the current ground truth, captures the

Parameters

- **est_time** (float) – The estimated time the measurement of the sensor is expected to be captured at (in seconds).
- **ground_truth** ([GroundTruth](#)) – The corresponding ground truth data.

reset()

Resets the sensor, clearing all previous measurements. Normally used directly after fusion.

update(*time_sec: float = None*)

Called on demand to update the sensor's state. Typically called after measurements have been taken. When called this increases the next measurement time and propagates internal states, to the time difference.

Parameters

time_sec (*float*) – The current simulation time in seconds.

property last_measurement: **tuple**[float, float]

Returns the latest measurement produced from the sensor.

Returns

The last produced gravity gradient signal.

Return type

tuple[float, float]

property num_steps: **int**

Returns the total number of internal measurement steps required. Used namely for sharing array sizes.

Returns

The number of internal measurement steps used.

Return type

int

property interferometer: *GravityGradInterferometer*

Returns the interferometer properties used by the sensor.

Returns

Interferometer properties.

Return type

GravityGradInterferometer

property gravity_model: *GravityModel*

Returns the internal gravity model used by the sensor. Can be accessed for testing or producing similar measurements.

Returns

Gravity Model instance used by the sensor.

Return type

GravityModel

```
class qnav.quantum.gravity_gradient.ParticleFilterParams(num_particles: int = 1000,
                                                         init_position_sigma: float = 20.0,
                                                         init_velocity_sigma: float = 1e-05,
                                                         init_attitude_sigma: float = 1e-05,
                                                         weight_sigma: float = 0.00126, alpha:
                                                         float = 0.05, beta: float = 0.05, gamma:
                                                         float = 0.05, position_sigma: float = 2.0,
                                                         velocity_sigma: float = 0.005,
                                                         attitude_sigma: float = 0.001,
                                                         ellipse_intervals: int = 361,
                                                         ellipse_iterations: int = 5)
```

Particle Filter Properties for the gravity gradient fusion methods. A list of properties relating to the particle filter processing.

```
class qnav.quantum.gravity_gradient.GravityGradientPF(qs_gravity_grad: GravityGradiometer,  
                                                    filter_params: ParticleFilterParams =  
                                                    ParticleFilterParams(num_particles=1000,  
                                                                    init_position_sigma=20.0,  
                                                                    init_velocity_sigma=1e-05,  
                                                                    init_attitude_sigma=1e-05,  
                                                                    weight_sigma=0.00126, alpha=0.05,  
                                                                    beta=0.05, gamma=0.05,  
                                                                    position_sigma=2.0, velocity_sigma=0.005,  
                                                                    attitude_sigma=0.001, ellipse_intervals=361,  
                                                                    ellipse_iterations=5), est_interferometer:  
                                                    GravityGradInterferometer = None, rng_seed:  
                                                    int = None)
```

Gravity Gradient Particle Filter Fusion. Applies position correction using gravity gradient signals, via a particle filter approach.

perform_fusion(*estimated_state*: EstimatedState) → None

Performs the fusion with sensors and updates the estimated state. When called performs fusion when gravity gradient sensor measurements are ready. When so, the particle filter algorithms is employed and the estimated state is updated.

Parameters

estimated_state (EstimatedState) – The current estimated states of the platform. This is what the fusion method will update.

init_particle_filter(*estimated_state*: EstimatedState) → None

Initialises the particle filter using current estimate. This is called during the first iteration, setting the particles' estimated position, velocity and attitude values based around the current estimated state. The range of estimation is based on the initial sigma values set in the filter properties for each.

Parameters

estimated_state (EstimatedState) – The current estimated states of the platform.

reset()

Resets the state of the fusion, clearing internal temporary states. When call the fusion will return to expecting the first measurement, then continuing from there.

qnav.quantum.dummy

dummy_sensors.py

summary

Presents dummy sensors used for re-applying fusion algorithms. This module contains dummy sensor models used for re-applying previous fusion algorithms with corrected measurements. These dummy sensors allow internal states to be exposed and edited. These do not feature involvement of real ground truth data, errors states or “real” measurements.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

class qnav.quantum.dummy.DummySensor(*frequency: float, start_time: float = 0.0, rand_seed: int = None*)

Abstract class for fake sensor to handle synthetic measurements. Provides an abstract class for dummy sensor, which allows its measurements to be fabricated. This is solely used for solutions that require the re-use of fusion methods with updates/corrections applied to previous measurements.

property last_measurement: any

Returns the last measurement produced by the sensor.

Returns

Last measurement value produced.

Return type

any

property last_update: float

The time of the last measurement (in seconds).

Returns

Time of last measurement in seconds.

Return type

float

property next_update: float

The time of the next measurement (in seconds).

Returns

Time of next measurement in seconds.

Return type

float

class qnav.quantum.dummy.DummyAccelerometer(*accelerometer: Accelerometer*)

Dummy Accelerometer sensor used to mimic accelerometer measurements. This class is a dummy sensor that inherits functionalities for the Accelerometer class. Essentially supports synthetic accelerometer measurements to be used in fusion methods.

class qnav.quantum.dummy.DummyGyroscope(*gyroscope: Gyroscope*)

Dummy Gyroscope sensor used to mimic gyroscope measurements. This class is a dummy sensor that inherits functionalities for the Gyroscope class. Essentially supports synthetic gyroscope measurements to be used in fusion methods.

1.9.10 Simulation

Modules relating to simulation loop handling:

qnav.simulation.simulation

simulation.py

summary

The core harness responsible for coordinating simulations. This module contains the core functionalities for managing simulations. This mostly revolves around a core loop and the scheduling of sensor time intervals, sensor measurements and data fusion.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation.

```
qnav.simulation.simulation.run_simulation_loop(trajectory: ~qnav.waypoints.trajectory.Trajectory,
                                              estimated_state: ~qnav.estimation.state.EstimatedState,
                                              hardware: ~typing.Collection[~qnav.measurement.sensor.SensorFusion],
                                              clock: ~qnav.measurement.clock.Clock =
                                              <qnav.measurement.clock.DummyClock object>,
                                              results: ~qnav.simulation.results.ResultsCapture =
                                              <qnav.simulation.results.CaptureAll object>,
                                              display: ~qnav.simulation.display.Display =
                                              <qnav.simulation.display.ConsoleDisplay object>)
```

Executes the full simulation loop from start to finish. On succession returns states and results in requested format.

Parameters

- **trajectory** ([Trajectory](#)) – The trajectory to draw ground truth records from. This is what the virtual platform is to follow.
- **estimated_state** ([EstimatedState](#)) – The initial estimated state. This will be updated throughout the simulation via fusion.
- **hardware** ([Collection](#)[[SensorFusion](#)]) – A collection of [SensorFusion](#) instances. This represents the hardware and fusion algorithms attached to the virtual platform.
- **clock** – A virtual clock added to simulate clock errors. This is used when virtual clock drift is required.
- **results** – The results capture instance used to manage result data. This controls how and what data is collected throughout the simulation.
- **display** – The displayer instance used for displaying progress. This presents a display of the ongoing simulation allowing for immediate feedback of its progress.

qnav.simulation.scheduling**scheduling.py****summary**

Coordinates the scheduling of sensors and fusion methods. This module provides classes that are used for organising the timing of sensor measurements and fusion methods during simulations. Essentially this is used to efficiently get the next non-inform time increment along with what is to occur at that time.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
class qnav.simulation.scheduling.TimingHandler(hardware: Iterable[SensorFusion])
```

A handler for managing the timings for sensors and fusion methods. On creation this creates a mapping between shared sensors and fusion methods. Through this the sensor with the earliest due time can be obtained. Once this sensor is used (i.e a measurement has been produced) a flag can be set for the fusion methods associate with it. When sufficient sensor flags for a fusion method are set, the fusion method can then be also obtained and returned.

update_sensors_past_time(*delay_time: float*) → None

Repeatedly updates sensors until all due time are past the given time. Applies updates and skips measurements of all sensors until their due times for next measurements are past the given time. This is used to correctly induce a delay for measurements from the start of the simulation. Note this does not collect measurements or change any of the connected statuses used for fusion.

Parameters

delay_time (*float*) – The time to update all sensors past (in seconds). Essentially the delay before measurements are to be processed.

set_sensor_used(*sensor: Sensor*) → None

Flags that the given sensor has been used. Marks the given sensor instance as used for all fusion methods that requires this. If conditions are met, this trigger fusion methods that use this sensor to be ready.

Parameters

sensor (*Sensor*) – The sensor instance that has been used recently.

get_next_sensor() → *Sensor*

Returns the sensor with the earliest due time for measurement. Searches through shared sensors and returns the one with the earlier due time.

Returns

Sensor with the earliest due time.

Return type

Sensor

sensor_iterator(*max_time: float*) → Iterator[*Sensor*]

Generates an iterator for continually getting the next sensor. The produced iterator will be return the sensor with the earliest measurement due time. The iteration stops when the next earliest due time exceeds the given max time.

Parameters

max_time (*float*) – The maximum time due for measurements (in seconds).

Returns

An iterator returning the sensor with the earliest due time.

Return type

Iterator[*Sensor*]

get_next_fusion(*sensor: Sensor = None*) → list[*SensorFusion*]

Returns any fusion methods that are to be used. Searches through fusion methods and returns a list collection of those that are ready to be used. These are automatically marked as used on return. If no fusion methods are ready, then the list will be empty. For faster retrieval, the search of fusion methods can be filtered to only those assigned with a given sensor instance.

Parameters

sensor (*Sensor*) – Sensor instance to filter fusion methods by (Optional).

Returns

A list of fusion methods ready to be applied.

Return type

list[*SensorFusion*]

print_mappings()

Displays the internal scheduling mappings in the output console. When called, prints the fusion methods attached to each sensor, along with their ready status. Useful for debugging states.

qnav.simulation.holonomic**holonomic.py****summary**

Supplies means of applying configured holonomic constraints. Virtual simulations are vulnerable to numerical instabilities, causing unintended drift between the ground truth and estimated state over time. Unlike with artificial noise that is purposely added through configured sensor error properties, this is undesirable and can cause unintended behavior. To combat this, this holonomic module can be used to apply a variety of constraints to restrict the estimated state of the system to values within the expected behaviour. In other words, this is used to slightly nudge the estimated state toward that of the ground truth at a scale to overcome numerical instabilities (such as rounding errors) but not to affect simulated errors.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation.

class qnav.simulation.holonomic.**HolonomicConstraint**(*frequency: float, gt_filter: str, fixed_gain: float, axis_filter: ndarray = None*)

Obtains holonomic correction values during simulation. For a given ground truth record, gently shifts the estimated state towards the ground truth state at a rate to overcome unintended drift, but not affect simulation accuracy (i.e. intended behaviour from artificial/configured errors).

property filter_id: str

Displays the tag used for filtering the ground truth. Simply used for checking filter doesn't already exist.

Returns

The tag ID for the ground truth record to filter by.

Return type

str

property filter_gain: float | ndarray

Displays the gain amount (percentage) being applied. Simply used for checking the filter's value.

Returns

The gain amount to apply to each axis

Return type

float | np.ndarray

take_measurement(*est_time: float, ground_truth: GroundTruth*)

Extracts data from ground truth to obtain correction amount. When called during simulation loop, this checks the relevant data can be extracted from the ground truth and uses it to update the correction value to apply at that time.

Parameters

- **est_time** (*float*) – The estimate time of the measurement (not used).
- **ground_truth** (*GroundTruth*) – The ground truth record at the given time.

clear_measurement()

Clears the last measurement value obtained. Useful in preventing the accidental reuse of measurements when using multiple holonomic constraints asynchronously.

property last_measurement: any

Returns the last measurement captured (i.e. correction to apply). Upon error this will return None.

Returns

The holonomic correction to apply.

Return type

float | np.ndarray

class qnav.simulation.holonomic.HolonomicFusion(trigger: FusionTrigger, *holonomics: HolonomicConstraint)

Applies the holonomic constraint corrections during simulation. Manages a correction of holonomic constraints and applying them to correct the current estimated state.

perform_fusion(estimated_state: EstimatedState) → None

Apply the holonomic constraints and corrections during simulation. When called, performs fusion by overwriting current estimates.

Parameters

estimated_state (EstimatedState) – The current estimated states to update.

qnav.simulation.simulation

simulation.py

summary

The core harness responsible for coordinating simulations. This module contains the core functionalities for managing simulations. This mostly revolves around a core loop and the scheduling of sensor time intervals, sensor measurements and data fusion.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation.

qnav.simulation.simulation.run_simulation_loop(trajectory: ~qnav.waypoints.trajectory.Trajectory, estimated_state: ~qnav.estimation.state.EstimatedState, hardware: ~typing.Collection[~qnav.measurement.sensor.SensorFusion], clock: ~qnav.measurement.clock.Clock = <qnav.measurement.clock.DummyClock object>, results: ~qnav.simulation.results.ResultsCapture = <qnav.simulation.results.CaptureAll object>, display: ~qnav.simulation.display.Display = <qnav.simulation.display.ConsoleDisplay object>)

Executes the full simulation loop from start to finish. On succession returns states and results in requested format.

Parameters

- **trajectory** (Trajectory) – The trajectory to draw ground truth records from. This is what the virtual platform is to follow.
- **estimated_state** (EstimatedState) – The initial estimated state. This will be updated throughout the simulation via fusion.
- **hardware** (Collection[SensorFusion]) – A collection of SensorFusion instances. This represents the hardware and fusion algorithms attached to the virtual platform.

- **clock** – A virtual clock added to simulate clock errors. This is used when virtual clock drift is required.
- **results** – The results capture instance used to manage result data. This controls how and what data is collected throughout the simulation.
- **displayer** – The displayer instance used for displaying progress. This presents a display of the ongoing simulation allowing for immediate feedback of its progress.

qnav.simulation.display

display.py

summary

Allows means for displaying simulation progress in real-time. This module is responsible for providing an interface for allowing the displaying of simulation data so that progress can be presented to the user in real-time.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation.

`qnav.simulation.display.is_using_terminal()` → bool

Checks if the application is being executed via terminal. Used to determine if the toolbox is being run and displayed through a command-line terminal, and not some interactive shell (or IDE). Used to obtain an idea of how output data is to be displayed.

Returns

True if standard output is being displayed inside terminal.

Return type

bool

`qnav.simulation.display.clear_terminal_line(num_lines: int = 1)` → None

Clears a previously printed line in the console. Only works when executing application from command line.

Parameters

num_lines (*int*) – Number of lines to clear (default = 1).

`qnav.simulation.display.sec_to_str(time_sec: int)` → str

Converts seconds to printed hours, minutes and seconds. Used to make printed time more human readable.

Parameters

time_sec (*int*) – The time to display in seconds.

Returns

The given time in HH:MM:SS string format.

Return type

str

class `qnav.simulation.display.Display(update_interval: float)`

A template used to define classes for displaying simulation progress. This provides the definition to be followed for classes that can be used for displaying simulation data during runtime. This is useful for users who wish to see verbose information or plot data during simulations, without having to wait until the end.

abstract on_start(*trajectory*: [Trajectory](#))

Called at the start of the simulation before first iteration. Can be used to initialise any displaying components by extracting necessary data from the trajectory.

Parameters

trajectory ([Trajectory](#)) – The ground truth trajectory instance.

abstract on_update(*timestamp*: *float*, *estimated_state*: [EstimatedState](#), *ground_truth*: [GroundTruth](#))

Called during the simulation after each fusion update. Presents the current (true) timestep, estimated states and ground truth record.

Parameters

- **timestamp** (*float*) – The current (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The current estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The current ground truth state of the simulation.

abstract on_finish(*timestamp*: *float*, *estimated_state*: [EstimatedState](#), *ground_truth*: [GroundTruth](#))

Called at the end of the simulation after final iteration. Can be used to finish up any displaying of information. Presents the final (true) timestep, estimated states and ground truth record.

Parameters

- **timestamp** (*float*) – The final (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The final estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The final ground truth state of the simulation.

class `qnav.simulation.display.ConsoleDisplay`(*update_interval*: *float*)

A simple display for printing simulation progress in console. Used to display that periodically prints the simulation's runtime progress in the output console. Namely used as an example display class.

on_start(*trajectory*: [Trajectory](#))

Called at the start of the simulation before first iteration. Used to obtain the start and end times of the trajectory.

Parameters

trajectory ([Trajectory](#)) – The ground truth trajectory instance.

on_update(*timestamp*: *float*, *estimated_state*: [EstimatedState](#), *ground_truth*: [GroundTruth](#))

Called during the simulation after each fusion update. Updates the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The current (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The current estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The current ground truth state of the simulation.

on_finish(*timestamp*: *float*, *estimated_state*: [EstimatedState](#), *ground_truth*: [GroundTruth](#))

Called at the end of the simulation after final iteration. Finishes the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The final (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The final estimated state of the simulation.

- **ground_truth** ([GroundTruth](#)) – The final ground truth state of the simulation.

class qnav.simulation.display.**ConsoleDisplayPlus**(*update_interval: float, max_dist_error: float = None*)

Extension to the basic displaying class that includes estimation errors. This subclass also calculates the estimated position error at each interval and displays it in the console. In addition to this, an optional threshold can be set for the maximum error distance for prematurely stopping the simulation.

on_update(*timestamp: float, estimated_state: [EstimatedState](#), ground_truth: [GroundTruth](#)*)

Called during the simulation after each fusion update. Updates the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The current (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The current estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The current ground truth state of the simulation.

on_finish(*timestamp: float, estimated_state: [EstimatedState](#), ground_truth: [GroundTruth](#)*)

Called at the end of the simulation after final iteration. Finishes the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The final (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The final estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The final ground truth state of the simulation.

class qnav.simulation.display.**MultilineDisplay**(*update_interval: float, max_dist_error: float = None*)

on_start(*trajectory: [Trajectory](#)*)

Called at the start of the simulation before first iteration. Used to obtain the start and end times of the trajectory.

Parameters

- **trajectory** ([Trajectory](#)) – The ground truth trajectory instance.

on_update(*timestamp: float, estimated_state: [EstimatedState](#), ground_truth: [GroundTruth](#)*)

Called during the simulation after each fusion update. Updates the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The current (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The current estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The current ground truth state of the simulation.

on_finish(*timestamp: float, estimated_state: [EstimatedState](#), ground_truth: [GroundTruth](#)*)

Called at the end of the simulation after final iteration. Finishes the runtime progress display of the simulation.

Parameters

- **timestamp** (*float*) – The final (true) simulation time in seconds.
- **estimated_state** ([EstimatedState](#)) – The final estimated state of the simulation.
- **ground_truth** ([GroundTruth](#)) – The final ground truth state of the simulation.

1.9.11 Util

Modules relating to commonly used utility functions:

qnav.util.constants

constants.py

summary

A list of calculation constants that are across the toolbox. This shared module contains a collection of geological constants that are repeatedly used by multiple functions within the toolbox

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of this module.

qnav.util.constants.G = 9.81

g - The average gravitation acceleration.

qnav.util.constants.F = 0.0033528106647474805

f - Flattening measure of the WGS84 spheroid.

qnav.util.constants.POLAR_AXIS_A = 6378137.0

a - Semi-major axis equator radius (m).

qnav.util.constants.R_E = 6367444.65712259

R_e - The average radius of the Earth (m).

qnav.util.constants.ECC_SQ = 0.006694379990141316

e^2 - The squared first eccentricity value for the Earth.

Type

#

qnav.util.constants.ECC = 0.08181919084262149

e - The first eccentricity value for the Earth.

qnav.util.constants.ECC_PRIME = 0.08209443794969568

e' - The second eccentricity value for the Earth.

qnav.util.constants.G_E = 9.78032533590389

γ_e - Normal gravity at Equator (m/s^2).

qnav.util.constants.G_P = 9.832184937863401

γ_b - Normal gravity at Poles (m/s^2).

qnav.util.constants.OMEGA_E = 7.2921159e-05

ω_e - Approximate rotation rate of the Earth (m/s^2).

qnav.util.constants.GM = 398600441880000.0

GM - The Geocentric gravitational constant (m^3/s^2)

qnav.util.constants.M = 0.0034497873577006006

M - Somigliana constant, used for calculation normal gravity.

qnav.util.connections

`qnav.util.connections.download_file(url: str, dest: Path)`

Downloads a file from a URL and writes it to destination file. Commonly used by the toolbox for downloading database files from selected web-servers.

Parameters

- **url** (*str*) – The URL of the file to be downloaded.
- **dest** (*Path*) – The file to save the downloaded file as.

qnav.util.system

common.py

summary

Provides commonly used project functions. This module contains commonly used (non-calculation) functions that are called throughout the project. These are namely for providing information about the project and the current platform.

author

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module

`qnav.util.system.get_toolbox_version()` → str

A simple function to return the current version number of the toolbox. This is useful for reporting information that may dependent on the build of the software, such as generated results or reported errors.

Returns

The decimal version number for the toolbox.

Return type

str

`qnav.util.system.get_system_name(return_min: bool = False)` → str

Return a short string for identifying the host operating system. This is namely used for identifying the operating system in a human-readable format. This is useful for identifying the type of system used when running an experiment.

Parameters

return_min (*bool*) – If set to true, only the operating system name and major version number will be returned.

Returns

A short string used for identifying the host operating system.

Return type

str

`qnav.util.system.get_platform_details()` → str

A simple function that returns a description of the platform. This function returns a string including the python version, processor platform architecture of the host system.

Returns

A string describing the platform.

Return type

str

`qnav.util.system.is_window_platform() → bool`

A simple function which will return true if the system's operating system is Microsoft Windows based. Otherwise, this function will return false.

Returns

Returns true if the host operating system is Windows based.

Return type

bool

`qnav.util.system.is_mac_platform() → bool`

A simple function which will return true if the system's operating system is Mac-OS based. Otherwise, this function will return false.

Returns

Returns true if the host operating system is Mac-OS based.

Return type

bool

`qnav.util.system.is_linux_platform() → bool`

A simple function which will return true if the system's operating system is Linux/Unix based. Otherwise, this function will return false.

Returns

Returns true if the host operating system is Linux based.

Return type

bool

qnav.util.rng**rng.py****summary**

Provides alternative means for pseudo-random number generation. This module contains implementations for controlled random number generation, subject to realistic factors. Currently, this only supplies gaussian-markov random walk based number generation.

authors

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First implementation of module.

class `qnav.util.rng.GaussianMarkov`(*corr_time: float, psd: float, output_shape: int | tuple[int] = 1, start_time: float = 0, seed: int = None*)

Gaussian Markov First-Order Random Walk Model. Generates controlled random numbers following a Gaussian Markov random walk. Can be used to replace typical normal random number generation.

get_random(*clock_time: float*) → float | ndarray

Generates random number(s) for requested time and shape. Calling this method continues the random walk to the given time in seconds. Calling with a time earlier than that of the last measurement will result in a value error due to illegal state.

Parameters

clock_time (*float*) – The current simulation clock time.

Returns

Randomly generated number(s).

Return type

float | np.ndarray

reset(*clock_time: float = 0*) → None

Resets the internal state of the random walk, clearing previous history and information.

Parameters

clock_time (*float*) – Optional new initialisation clock time. By default this is 0 seconds.

property output_shape: int | tuple[int]

Returns the output shape numbers are being generated for. This is the dimensions of the output of the number generator.

Returns

Output shape for random number generation.

Return type

int | tuple[int]

property last_value: float | ndarray | None

Returns the last random value generated.

Returns

Last value returned by generator.

Return type

float | np.ndarray | None

property last_time: float

Returns time of last number(s) generated in seconds.

Returns

Time of last number(s) generated.

Return type

float

1.9.12 Waypoints

Modules related to trajectory generation:

qnav.waypoints.generation

generation.py

summary

Handles all interpolation and generation of waypoint trajectory data. This module contains a collection of functions for generating waypoint trajectories for various vehicle models. In addition to this, waypoint data can be imported from CSV files or binary numpy files.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.1.0 - Includes fixes and improvements.

`qnav.waypoints.generation.get_positions`(*lat_points: ndarray, lon_points: ndarray, alt_points: ndarray, vehicle_model: Vehicle, frequency: float = 200.0, avg_speed: float = 100.0, travel_times: ndarray = None*)

This function interpolates positions (latitude, longitude and altitude) between provided waypoints that are separated by either stated distance or time (not both). The generated route is subject to the limitations of the vehicle. The produced route can then be used for calculating the required trajectory for simulations.

Parameters

- **lat_points** (*numpy.ndarray*) – The decimal latitude points for the given waypoints. Use negative values for southern coordinates (in degrees).
- **lon_points** (*numpy.ndarray*) – The decimal longitude points for the given waypoints. Use negative values for western coordinates (in degrees).
- **alt_points** (*numpy.ndarray*) – The altitude for the given waypoints (in metres).
- **vehicle_model** (*Vehicle*) – An initialised vehicle model containing the manoeuvring limitations to consider when generating the waypoint positions.
- **frequency** (*float*) – The simulation's update frequency. This is used for calculating the number of intermediate points and total number of waypoints to return.
- **avg_speed** (*float*) – The average vehicle speed for the given waypoints. This is used to calculate the travel times between waypoints. This value is replaced by 'travel_times'. Therefore, a value for this is not required if the travel times are known.
- **travel_times** (*numpy.ndarray*) – The time (in seconds) when each waypoint is reached. This is required for interpolating the position between points.

Returns

An array containing column for latitude, longitude and altitude of the vehicle at each time step.

Return type

numpy.ndarray (n-by-4 elements)

`qnav.waypoints.generation.interpolate_positions`(*base_points: ndarray, frequency: float*) → *ndarray*

Interpolates the base waypoints at the new higher frequency. This function can be used to speed up and stabilise the calculation of the waypoint positions, by interpolating the intermediate steps.

Parameters

- **base_points** (*np.ndarray*) – The base waypoint data to interpolate.
- **frequency** (*float*) – The frequency to interpolate the waypoint data by.

Returns

The interpolated version of the given waypoint positions.

Return type

np.ndarray

`qnav.waypoints.generation.produce_noise`(*num_rows: int, first_val: float, noise_val: float, dt_vdp: float*)

This function generated random noise that can be added to various aspects. For instance, this function can be used to produce noise for position, attitude and oscillation displacement. This function is required so that the noise generated is done so in a self-balancing manner such that the displacements do not cause permanent or increasing drift.

Parameters

- **num_rows** (*int*) – The number of noise values to produce.
- **first_val** (*float or numpy.array*) – The first generated noise value(s).
- **noise_val** (*float*) – The scale of the noise values.
- **dt_vdp** – The dampening factor to apply. This is used to prevent the noise from causing permanent or excessive drift.

Returns

The generated noise subject to the given parameters.

Return type

numpy.array

`qnav.waypoints.generation.moving_average(input_array, window_size: int = 5)`

Obtains the “sliding window average” for values of an array. This function calculates the moving average of a given 1D array using the same approach as MATLAB. A sliding window is passed over the given array. For the centre (right) element the moving average is calculated using the values covered by the window. This means that fewer elements are covered at the start and end of the array. The returned values are the same size as the given array.

Parameters

- **input_array** (*NumPy array*) – The values which the moving average is going to be calculated for. This must be a 1D array.
- **window_size** (*int*) – The size of the sliding window. Must be an integer.

Returns

The calculated moving averages for the given array.

Return type

NumPy array

`qnav.waypoints.generation.rescale_values(values)`

A simple function that allows angles to be re-scaled to prevent inaccuracy calculations between angles differences.

Parameters

values (*numpy.array*) – The angle values to be rescaled.

Returns

The given values re-scaled between $-\pi/2$ and $\pi/2$.

Return type

numpy.array

`qnav.waypoints.generation.get_simple_trajectory_values(waypoints: ndarray, gravity_model: GravityModel, given_attitudes: ndarray = None, attitude_times: ndarray = None, smoothing_window_time: float = None, smoothing_passes: int = None, step_size: int = 2, virtual_mem_file: Path = None)`

Calculates the trajectory information and properties from given waypoints. This function calculates the trajectory properties for each position for a given set of waypoints. Only simple properties are generated, without any added noise, vibrations or oscillations of the platform. In previous releases this function was referred to as legacy waypoint generation.

Parameters

- **waypoints** (*NumPy Matrix (n-by-4 elements)*) – The interpolated waypoint array previously generated. It is important any changes/updates to the waypoints (such as altitude changes) are applied to the waypoints before using this function.
- **gravity_model** ([GravityModel](#)) – The gravity model used to calculate local gravity. This is required for calculating the downwards gravitation acceleration and its impact on the platform for each axis.
- **given_attitudes** (*numpy.array (n-elements)*) – (Optional) The requested attitude of the vehicle at given times. Intermediate attitude values will be interpolated.
- **attitude_times** (*numpy.array (n-elements)*) – (Optional) The requested corresponding times for the given attitudes. These timestamps are used to interpolate intermediate attitude values.
- **smoothing_window_time** (*float*) – (Optional) The size of the smoothing (moving average) window to use for smoothing the calculated waypoint velocities. This is useful for removing noise.
- **smoothing_passes** (*int*) – (Optional) The number of smoothing passes (iterations) to perform on the calculated waypoint velocities.
- **step_size** (*int*) – (Optional) The step size used for calculating the velocity between upcoming waypoints (default=1).
- **virtual_mem_file** (*bool*) – The on-disk file to use for virtual memory. Will be created or overwritten

Returns

A matrix containing the position, velocity, acceleration, attitude, angle rates and gravitational acceleration (and gradient) for each time step.

Return type

NumPy Matrix (n-by-16 elements)

```
qnav.waypoints.generation.get_trajectory_values(waypoints: ndarray, gravity_model: GravityModel,  
                                              vehicle_model: Vehicle, given_attitudes: ndarray =  
                                              None, attitude_times: ndarray = None,  
                                              smoothing_window_time: float = None,  
                                              smoothing_passes: int = None, step_size: int = 2,  
                                              virtual_mem_file: Path = None)
```

Using waypoints route information generated prior, this function will calculate the velocity, acceleration, attitude, angle rates and gravitational acceleration (and gradient) for each time step. This function calculates said values in an efficient vectorised manner while ensuring they abide by the given vehicle model's limitations.

Parameters

- **waypoints** (*NumPy Matrix (n-by-4 elements)*) – The interpolated waypoint array previously generated. It is important any changes/updates to the waypoints (such as altitude changes) are applied to the waypoints before using this function.
- **gravity_model** ([GravityModel](#)) – The gravity model used to calculate local gravity. This is required for calculating the downwards gravitation acceleration and its impact on the platform for each axis.
- **vehicle_model** ([Vehicle](#)) – An initialised vehicle model containing the manoeuvring limitations to consider when generating the waypoint positions.
- **given_attitudes** (*numpy.array (n-elements)*) – (Optional) The requested attitude of the vehicle at given times. Intermediate attitude values will be interpolated.

- **attitude_times** (*numpy.array (n-elements)*) – (Optional) The requested corresponding times for the given attitudes. These timestamps are used to interpolate intermediate attitude values.
- **smoothing_window_time** (*float*) – (Optional) The size of the smoothing (moving average) window to use for smoothing the calculated waypoint velocities. This is useful for removing noise.
- **smoothing_passes** (*int*) – (Optional) The number of smoothing passes (iterations) to perform on the calculated waypoint velocities.
- **step_size** (*int*) – (Optional) The step size used for calculating the velocity between upcoming waypoints (default=1).
- **virtual_mem_file** (*bool*) – The on-disk file to use for virtual memory. Will be created or overwritten

Returns

A matrix containing the position, velocity, acceleration, attitude, angle rates and gravitational acceleration (and gradient) for each time step.

Return type

NumPy Matrix (n-by-16 elements)

`qnav.waypoints.generation.plot_waypoints(data_values: ndarray, filename=None)`

A simple function to plot the generated waypoints.

Parameters

- **data_values** (*numpy.ndarray (n-by-3 elements)*) – The array of values returned by the generator function.
- **filename** (*str*) – (Optional) Saves the produced plot under the given file.

`qnav.waypoints.generation.plot_figures(data_values: ndarray, filename=None)`

A simple function to plot the change in variables during the waypoint generation. Each of these is presented within a sub-figure.

Parameters

- **data_values** (*numpy.ndarray (n-by-16 elements)*) – The array of values returned by the generator function.
- **filename** (*str*) – (Optional) Saves the produced plot under the given file.

`qnav.waypoints.generation.load_waypoint_csv(filepath: Path, down_sample_rate: int = 1)`

This function will extract the values from a given trajectory file, formatted as comma-separated-values. Once the relevant information is extracted, 7 rows will be returned containing information of the extracted timestamps, latitude points, longitude points, altitude points and three attitude angles. These rows are returned within a list.

Parameters

- **filepath** (*str or Path*) – The location of the CSV trajectory file to read.
- **down_sample_rate** (*int*) – The down sampling multiplier to apply. This can be used to skip rows for cases where CSV waypoints are given too frequently or spaced too closely together.

Returns

A list containing 7 arrays holding the extracted timestamps, latitude points, longitude points, altitude points and attitude angles from the given CSV trajectory file.

Return type

NumPy array

`qnav.waypoints.generation.crop_waypoints(waypoints: ndarray, max_time: float)`

Crops waypoint data between time ranges (in seconds).

Parameters

- **waypoints** (*numpy.ndarray*) – The waypoint data to crop.
- **max_time** (*float*) – The maximum time for the return waypoint data.

Returns

The cropped waypoint data

Return type

numpy.ndarray

`qnav.waypoints.generation.save_waypoints(waypoints: ndarray, file_path: Path, allow_overwrite: bool = False)`

Exports waypoint data to disk, allowing for reuse.

Parameters

- **waypoints** (*numpy.array*) – The waypoint data to save.
- **file_path** (*pathlib.Path*) – The file location to save the waypoint data to.
- **allow_overwrite** (*bool*) – Should file overwriting be permitted? This safeguards against accidental overwriting (default=False).

`qnav.waypoints.generation.load_waypoints(file_path: Path, as_virtual: bool = False)`

Imports waypoint data from disk, reusing saved values.

Parameters

- **file_path** (*pathlib.Path*) – The file location to load the waypoint data from.
- **as_virtual** (*bool*) – If set to True prevent all the data being loaded at once and use virtual memory. This reduces RAM usage, but impacts runtime (default=False).

Returns

The loaded waypoint data.

Return type

numpy.array (n-by-16 elements)

`qnav.waypoints.generation.get_trajectory(waypoints: ndarray, gravity_model: GravityModel)`

TODO: Add documentation :param waypoints: :param gravity_model: :return:

`qnav.waypoints.generation.read_csv_waypoints(file_path: Path, delimiter: str = ',') → dict[str, ndarray | None]`

Reads given CSV waypoint file and returns the records contained within it. For a given file path, attempts to read its content and return it's extracted records inside a dictionary.

Parameters

- **file_path** (*Path*) – The CSV file to attempt to read waypoint data from.
- **delimiter** (*str*) – The delimiter used to represent column breaks. By default, this is a comma, but other characters can be used.

Returns

A dictionary holding the extracted data.

Return type

dict[str, np.ndarray | None]

qnav.waypoints.trajectory**trajectory.py****summary**

Used for the core representation of ground truth trajectory information. This module is responsible for handling ground truth waypoint trajectories, including the managing of input data, position and value calculations and look-up interpolation of ground truth records.

authorMichael Wright - mjwright@liverpool.ac.uk**version**

1.0.0 - First full implementation of this module.

class qnav.waypoints.trajectory.**GroundTruth**(*timestamp: float, position: ndarray, velocity: ndarray, acceleration: ndarray, attitude: ndarray, angle_rates: ndarray*)

A record class for trajectory ground truth a given time. This class contains the ground truth (the true trajectory values) at a given time. The purpose of this is to supply the “real-world” data to sensor models, where simulated noise and errors will then be applied to produce measurement data.

timestamp: float

The time since initialisation (in seconds).

position: ndarray

The position as latitude-longitude-altitude (in deg-deg-metres).

velocity: ndarray

The velocity as along-across-down in body axis (in metres/second).

acceleration: ndarrayThe acceleration as along-across-down in body axis (in metres/second²).**attitude: ndarray**

The attitude as north-east-down in NED axis (in degrees).

angle_rates: ndarray

The angle rates as north-east-down in NED axis (in degrees/second).

as_dict() → dict

Returns class properties in dictionary form.

Returns

A dictionary containing ground truth values.

Return type

dict

as_numpy() → ndarray

Returns call properties in 1D numpy array form.

Returns

A numpy array containing ground truth values.

Return type
np.ndarray

class qnav.waypoints.trajectory.**Waypoints**(*timestamps: ndarray, lat_points: ndarray, lon_points: ndarray, alt_points: ndarray, attitudes: ndarray = None*)

A container for trajectory waypoint data used to define a course. This data class will contain the arrival time and positioning records for core points along a trajectory. These can either represent sparse base points that a trajectory is to cover, or a series of dense positions, a fixed frequency, for an already generated trajectory. Additionally, attitude values can be provided for each position.

as_array() → ndarray

Returns required records formatted as column arrays. Specifically returns an n-by-4 array holding the timestamps, latitude, longitude and altitude records as columns, respectively. Optional Attitude records are not included in this output.

Returns
Position data formatted as column array.

Return type
np.ndarray

class qnav.waypoints.trajectory.**Trajectory**(*waypoints: Waypoints, vehicle_model: Vehicle, gravity_model: GravityModel, base_freq: float, inter_method: str = 'pchip'*)

A fully-generated trajectory with ground truth look-up. On initialization, this generates the full trajectory from given base waypoints, vehicle model and gravity model. The core trajectory is generated with base points calculated at fixed frequency. Intermediate values are then interpolated.

get_record(*time_sec: float*) → *GroundTruth*

Returns values of the trajectory at given time (in seconds). Safely interpolates and returns ground truth at requested time.

Parameters
time_sec (*float*) – A requested time in seconds.

Returns
The corresponding ground truth record.

Return type
GroundTruth

get_records(*time_steps: ndarray*) → dict[str, ndarray]

Obtains full records of the trajectory for a given times (in seconds). Essentially performs vectorised interpolation for given time series.

Parameters
time_steps (*np.ndarray*) – The time steps (in seconds) to obtain records for.

Returns
A dictionary holding the records for each time step.

Return type
dict[str, np.ndarray]

property start_time: float

Returns the time of the first trajectory record (in seconds).

Returns
The start time of the trajectory

Return type

float

property end_time: float

Returns the time of the final trajectory record (in seconds).

Returns

The end time of the trajectory

Return type

float

property interp_method: str

The interpolation method used for ground truth generation.

Returns

Interpolation name.

Return type

str

`qnav.waypoints.trajectory.cumulative_distance(lat_points: ndarray, lon_points: ndarray, alt_points: ndarray) → ndarray`

Calculate the cumulative ‘great-circle’ distance between coordinate points. For a given series of latitude-longitude-altitude points (given in deg-deg-m), this calculates the cumulative distance between them using the standard WGS-84 reference ellipsoid.

Parameters

- **lat_points** (*np.ndarray (n-elements)*) – A series of latitude positions (in decimal degrees).
- **lon_points** – A series of longitude positions (in decimal degrees).
- **alt_points** – A series of altitude positions (in decimal degrees).

Returns

The cumulative distance all points, starting from first.

Return type

np.ndarray

qnav.waypoints.vehicle**vehicle.py****summary**

Represents vehicle properties for waypoint generation. This module contains a template Vehicle tuple listing all the properties applied during waypoint generation. In addition to this a handful of integrated vehicle profiles are include for testing. It is recommended that a custom Vehicle instance is created for finer control.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk
Michael Wright - mjwright@liverpool.ac.uk

version

1.1.0 - Added integrated vehicle types for testing.

```

class qnav.waypoints.vehicle.Vehicle(description: str = 'Unnamed', turn_rate_max: float = 5.0,
                                     acceleration_max: float = 2.0, deceleration_max: float = 1.0,
                                     time_delay_acc: float = 1.0, time_delay_turn: float = 5.0,
                                     pitch_constant_offset: float = 0, pitch_osc_period: float = 0,
                                     pitch_osc_magnitude: float = 0, pitch_max: float = 0,
                                     roll_constant_offset: float = 0, roll_osc_period: float = 0,
                                     roll_osc_magnitude: float = 0, roll_max: float = 0,
                                     d_roll_d_turn_rate: float = 0, vibration_noise_acceleration: float =
                                     0, vibration_noise_angle_rates: float = 0,
                                     vibration_damping_period: float = 0, variation_per_second1: float
                                     = 0.005, variation_per_second2: float = 0.01, rand_seed: int = 0)

```

A structure used for holding vehicle model information. This is used to describe limitations, offsets and expected noise/oscillations. Unlike a normal class, the variables held within this structure are immutable and cannot be changed. This is done to enforce restrictions without them being accidentally overwritten later.

description: str

Alias for field number 0

turn_rate_max: float

Alias for field number 1

acceleration_max: float

Alias for field number 2

deceleration_max: float

Alias for field number 3

time_delay_acc: float

Alias for field number 4

time_delay_turn: float

Alias for field number 5

pitch_constant_offset: float

Alias for field number 6

pitch_osc_period: float

Alias for field number 7

pitch_osc_magnitude: float

Alias for field number 8

pitch_max: float

Alias for field number 9

roll_constant_offset: float

Alias for field number 10

roll_osc_period: float

Alias for field number 11

roll_osc_magnitude: float

Alias for field number 12

roll_max: float

Alias for field number 13

d_roll_d_turn_rate: float

Alias for field number 14

vibration_noise_acceleration: float

Alias for field number 15

vibration_noise_angle_rates: float

Alias for field number 16

vibration_damping_period: float

Alias for field number 17

variation_per_second1: float

Alias for field number 18

variation_per_second2: float

Alias for field number 19

rand_seed: int

Alias for field number 20

`qnav.waypoints.vehicle.get_default_profile(rand_seed: int = 0) → Vehicle`

Returns a [Vehicle](#) instance representing the default profile. This profile is for a typical aircraft with no noise/vibrations.

Parameters

rand_seed (*int*) – The seed to use for random number generation.

Returns

The default [Vehicle](#) profile.

Return type

[Vehicle](#)

`qnav.waypoints.vehicle.get_large_aircraft_profile(rand_seed: int = 0) → Vehicle`

Returns a [Vehicle](#) instance representing that of a large aircraft. This profile supplies the parameters for a 737 type of air-vehicle, stating slow acceleration and deceleration with wide turning.

Parameters

rand_seed (*int*) – The seed to use for random number generation.

Returns

The [Vehicle](#) profile for a large aircraft.

Return type

[Vehicle](#)

`qnav.waypoints.vehicle.get_large_ship_profile(rand_seed: int = 0) → Vehicle`

Returns a [Vehicle](#) instance representing that of a large aircraft. This profile supplies the parameters for a large ship, such as a cargo vessel.

Parameters

rand_seed (*int*) – The seed to use for random number generation.

Returns

The [Vehicle](#) profile for a large ship.

Return type

[Vehicle](#)

`qnav.waypoints.vehicle.get_large_van_profile(rand_seed: int = 0) → Vehicle`

Returns a *Vehicle* instance representing that of a large van. This profile supplies the parameters for a large ground vehicle, such as a heavy van vessel, slow acceleration/deceleration and little turning limitations.

Parameters

rand_seed (*int*) – The seed to use for random number generation.

Returns

The *Vehicle* profile for a large van.

Return type

Vehicle

`qnav.waypoints.vehicle.get_submarine_profile(rand_seed: int = 0) → Vehicle`

Returns a *Vehicle* instance representing that of a submarine. This profile supplies the parameters for a submarine vessel.

Parameters

rand_seed (*int*) – The seed to use for random number generation.

Returns

The *Vehicle* profile for a submarine vessel.

Return type

Vehicle

`qnav.waypoints.vehicle.get_unrestricted_profile() → Vehicle`

Returns a *Vehicle* instance with no movement restrictions. This profile is solely used for following tightly placed waypoints in situations where placing vehicle limitations would be impractical. No vehicle noise or vibrations are contained in this profile.

Returns

The *Vehicle* profile with no limitations or noise.

Return type

Vehicle

qnav.waypoints.tracks

tracks.py

summary

Provides default waypoint generation for testing tracks. This module contains various functions for generating different default waypoint tracks. This is an alternative instead of using user supplied trajectories. This is namely used for testing, but can also be used for rapid experimentation when a custom trajectory is unavailable.

authors

Prof. Jason Ralph - jfralph@liverpool.ac.uk

Michael Wright - mjwright@liverpool.ac.uk

version

1.0.0 - First full implementation of module.

```
qnav.waypoints.tracks.generate_racetrack(frequency: float = 1000.0, max_time: float = 500, lap_time:  
                                         float = 500, radius_x: float = 1000.0, radius_y: float = 1000.0,  
                                         radius_z: float = 5.0, theta_x: float = 0.0, theta_y: float = 0.0,  
                                         theta_z: float = 0.0, ref_lat: float = 0.0, ref_lon: float = 0.0,  
                                         ref_alt: float = 0.0, num_var_y: float = 2, num_var_z: float =  
                                         3) → ndarray
```

Generates waypoints for a racetrack of requested dimensions. This function produces waypoints at a requested frequency that represent a simple looping path. The produced waypoints can be then used with previously implemented waypoint functions, essentially (efficiently) pre-calculating the values at each position along the racetrack.

Parameters

- **frequency** (*float*) – The simulation's update frequency (in Hz). This is used for calculating the number of intermediate points and total number of waypoints to return.
- **max_time** (*float*) – The time at which the simulation is to end (in seconds). No waypoints will be generated for points after this time
- **lap_time** (*float*) – The time period (in seconds) for one circuit of the track. This is used to calculate how fast the vehicle is moving around the generated racetrack.
- **radius_x** (*float*) – The size of the racetrack along the x-direction (in metres).
- **radius_y** (*float*) – The size of the racetrack along the y-direction (in metres).
- **radius_z** (*float*) – The height variations of the racetrack (in metres).
- **theta_x** (*float*) – The starting X position on the circuit.
- **theta_y** (*float*) – The starting Y position on the circuit.
- **theta_z** (*float*) – The starting Z position on the circuit.
- **ref_lat** (*float*) – The reference latitude of the centre position of the circuit (in degrees).
- **ref_lon** (*float*) – The reference longitude of the centre position of the circuit (in degrees).
- **ref_alt** (*float*) – The reference altitude of the centre position of the circuit (in degrees).
- **num_var_y** (*float*) – Number of periods of y variations for each circuit in x.
- **num_var_z** (*float*) – Number of periods of height variations for each circuit in x.

Returns

The calculated latitude, longitude and altitude for each time step from 0 to the maximum time requested.

Return type

numpy.ndarray (n-by-4 elements)

1.10 System Requirements

The finalised toolbox requires various hardware and software requirements to be efficiently used. The minimum and suggested requirements have been acquired based on experimentation and auditing of the toolbox.

1.10.1 Hardware Requirements

The hardware requirements are subject to the requested input. The resources needed can be reduced by decreasing the requested frequency/resolution and increasing the output down-sampling via the configuration file.

Minimum Specifications

The minimum requirements to use the toolbox are outlined in the table below. The host system must at a minimum have these hardware specifications to use the toolbox at a basic (limited) level. Updates and potential additions to the toolbox may increase these minimum requirements.

Hardware	Minimum Resources Required
Processor (CPU)	x86 32-bit processor 1.00 GHz (or better)
Physical Memory (RAM)	2.00 GB of memory (or larger)
Disk Storage Space (HDD)	20.00 GB available (or more)

Recommended Specifications

The recommended requirements to fully use the toolbox are outlined in the table below. If the host system has these hardware specifications or higher it is guaranteed to use the software to its full extent.

Hardware	Recommended Resources Required
Processor (CPU)	x86 64-bit processor 2.00 GHz (or better)
Physical Memory (RAM)	8.00 GB of memory (or larger)
Disk Storage Space (HDD)	100.00 GB available (or more)

Note About Memory and Storage

Executing simulations can demand a large amount of RAM. This is especially true if high frequencies and/or a large amount of waypoint data is used. For example, a single simulation at 1,000 Hz is expected to require around 1 GB of RAM for each simulated hour due to the amount of data that needs to be produced and recorded. To significantly reduce this resource requirement various settings can be changed.

Firstly, virtual memory can be used for offloading waypoint and estimation results to disk. This is enabled under the **Waypoint** and **Estimation** sections in the configuration file via the `useVirtualMemory` options. In addition, an `outputDownSampleRate` can be specified under the **Output** section, meaning that only a sub-sampled amount of the estimation results will be collected and saved. Lastly, at a worst case scenario the waypoint and measurement frequencies can be reduced.

Warning

The use of virtual memory on 32-bit systems is limited to at most 2 GB. This means that waypoints and simulation results cannot use more than 2 GB if the use of virtual memory is enabled on 32-bit systems. If this limitation causes significant problems requested updates to the toolbox can be made to overcome it.

In the event that the output data needs to be saved in multiple formats, but there is insufficient storage space, a single format (such as simple binary) can be used which can later be converted. Using the read and write modules inside the

output sub-package, the single generated file can be read and its content can then be written into the other needed formats.

1.10.2 Software Requirements

Operating System

The toolbox has been developed with platform independence constantly in mind, taking fully advantage of Python's flexibility.

The handling of file paths and managing local resources has been done without replaying on any operating system specific features/formats. Likewise, the implementation of multi-processing functions has been done in a way that acknowledges adapts to the different ways operating system handle communication between multiple threads/processes. As a result, the same project build can be used on any major platform without any alteration to its source code.

The toolbox has been tested and proved to function flawlessly on Windows, Mac and Linux systems. Specifically:

- Windows 10 - Enterprise
- Linux - Ubuntu LTS 22.04
- MacOS - Sonoma 14.6.1
- Raspberry Pi OS - 2022-09-22 (32-bit)

Python

The toolbox was developed using the [latest stable release of Python](#). To benefit from this, the latest Python features have been employed. Therefore, the toolbox requires the installation of **Python version 3.11 or higher**. During development, the use of using future safe functions/packages was strongly considered. While older versions of Python *may* be able to run the toolbox, the correct behaviour/output cannot be guaranteed.

Required Packages

Where possible, the use of dependencies has been eliminated by implementing the majority of the software manually. However, this could not be achieved for instances where the required functionality is not available in base Python. As a result, the toolbox requires the use of the following packages.

- [Mypy](#) (version 3.12.1+)
- [NumPy](#) (version 2.1.1+)
- [SciPy](#) (version 1.14.1+)
- [Pandas](#) (version 2.2.3+)
- [Matplotlib](#) (version 3.9.2+)
- [Plotly](#) (version 5.24.1+)
- [H5py](#) (version 1.11.2+)

If [PIP](#) is installed on the host machine, all the above required packages for their target version can be installed automatically by utilising the included `requirements.txt` file by using the following command:

Listing 3: Automatically install required packages (via PIP)

```
pip install -r requirements.txt
```

Sometimes PIP may not be installed as a standalone application, or it may be associated with another version of Python that is installed (i.e. an older version). In this case PIP can be use through Python.

Listing 4: Automatically install required packages

```
python -m pip install -r requirements.txt
```

Note

If the toolbox is installed as a package via PIP, all packages and dependencies will also be installed automatically along side the software. You can check which Python packages are installed using the `pip list` command.

For installing and handling Python packages in general, please see [this guide](#).

1.11 Example Usage

This section covers and describes the examples included with the project. Examples are group into categories, with a handful of cases being given for each.

1.11.1 Changing INS Grades

The sensor grades for the INS can be configured under the **Measurement** section. Specifically, by changing the path values of `accelerometerConfig` and `gyroscopeConfig`, or by changing the contents of the sub-configuration files. The examples included cover all of the typical sensors grades ranging from theoretically perfect down to basic consumer level grade.

```
python3 run.py 'examples/example_1/perfect.ini'
python3 run.py 'examples/example_1/marine.ini'
python3 run.py 'examples/example_1/aviation.ini'
python3 run.py 'examples/example_1/intermediate.ini'
python3 run.py 'examples/example_1/tactical.ini'
python3 run.py 'examples/example_1/consumer.ini'
```

1.11.2 Changing INS Integration

The INS integration method can be configured under the **Estimation** section. Specifically, by changing the value of `integrationMethod` to either *numerical*, *runge_kutta*, *adams_bashforth* or *kalman*. If the Kalman Filter is selected, the variable `estimatedStateModel` must also be changed to *kalman* as well. The example include covering these four integration methods against perfect and aviation grade sensors.

```
python3 run.py 'examples/example_2/perfect_numerical.ini'
python3 run.py 'examples/example_2/perfect_runge_kutta.ini'
python3 run.py 'examples/example_2/perfect_adams_bashforth.ini'
python3 run.py 'examples/example_2/perfect_kalman.ini'
```

(continues on next page)

(continued from previous page)

```
python3 run.py 'examples/example_2/aviation_numerical.ini'
python3 run.py 'examples/example_2/aviation_runge_kutta.ini'
python3 run.py 'examples/example_2/aviation_adams_bashforth.ini'
python3 run.py 'examples/example_2/aviation_kalman.ini'
```

1.11.3 Holonomic Constraints

Holonomic constraints can be enabled to combat the impact of unwanted numerical sensitivities during simulations. To do this, the option `useHolonomics` under the `Holonomics` section should be enabled. These examples includes repeating the previously used trajectory with perfect sensor and other very long trajectories, all of which show varying degrees of round errors. More examples are then used to repeat these with holonomic constraints, removing the effect.

```
python3 run.py 'examples/example_3/perfect_numerical.ini'
python3 run.py 'examples/example_3/20deg_perfect_without_hc.ini'
python3 run.py 'examples/example_3/20deg_perfect_with_hc.ini'
python3 run.py 'examples/example_3/5deg_perfect_without_hc.ini'
python3 run.py 'examples/example_3/5deg_perfect_with_hc.ini'
python3 run.py 'examples/example_3/60deg_perfect_without_hc.ini'
python3 run.py 'examples/example_3/60deg_perfect_with_hc.ini'
python3 run.py 'examples/example_3/lwpl_perfect_without_hc.ini'
python3 run.py 'examples/example_3/lwpl_perfect_with_hc.ini'
```

1.11.4 Vehicle Models

The selected vehicle model for forming the trajectory can be configured under the `Vehicle` section. Specifically, a sub-configuration for the desired vehicle type can be selected using `vehicleConfig`, or a custom one can be generated. Each vehicle model describes the expected speeds, turning rates, acceleration profiles and drift delays expected. These configurations also can contain additional noise, vibration and oscillation profiles. The examples included cover various land, sea and aerial based vehicles.

```
python3 run.py 'examples/example_4/perfect_custom.ini'
python3 run.py 'examples/example_4/perfect_large_plane.ini'
python3 run.py 'examples/example_4/perfect_large_ship.ini'
python3 run.py 'examples/example_4/perfect_large_van.ini'
python3 run.py 'examples/example_4/perfect_submarine.ini'
```

1.11.5 Configuring Trajectories

The shape of the trajectory can be described in a handful of ways. Firstly, by setting `waypointMode` to `config` under the `Waypoints` section, a trajectory can be defined by setting a series of latitude, longitude and altitude points with the variables `waypointLat`, `waypointLon` and `waypointAlt`. Alternatively, the waypoint mode can be set to `csv` to read these values from a CSV file. Lastly, by settings the waypoint to `racetrack`, an arithmetically generated looping trajectory will be used, as configured under the `Racetrack` section. The examples included cover a few cases for each of these:

```
python3 run.py 'examples/example_5/perfect_lwpl_csv.ini'
python3 run.py 'examples/example_5/perfect_lwpl_csv_2.ini'
python3 run.py 'examples/example_5/perfect_racetrack_1.ini'
```

(continues on next page)

(continued from previous page)

```
python3 run.py 'examples/example_5/'perfect_racetrack_2.ini'
python3 run.py 'examples/example_5/'perfect_s3k119200_qt_csv.ini'
python3 run.py 'examples/example_5/'perfect_trajeuler_csv.ini'
```

1.11.6 Altimeter Sensors and Fusion

An altimeter can be enabled to provide corrections to the estimated vertical position, configured under the **Altimeter** section. This allows not only the selection of sensor grade, but also whether the fusion method *fixed_gain* or *alpha_beta* should be used, selected using the `altimeterFusion` option. The examples covers a few scenarios:

```
python3 run.py 'examples/example_6/'aviation_alpha_beta.ini'
python3 run.py 'examples/example_6/'aviation_fixed_gain.ini'
python3 run.py 'examples/example_6/'aviation_only.ini'
python3 run.py 'examples/example_6/'perfect_alpha_beta.ini'
python3 run.py 'examples/example_6/'perfect_fixed_gain.ini'
python3 run.py 'examples/example_6/'perfect_only.ini'
```

1.11.7 GPS Sensors and Fusion

The involvement of GPS can be configured under the **GPS** section. This allows the enabling of the sensor, stating if GPS data can be automatically downloaded, the configuration of noise and the involvement of active and inactive zones. These examples cover the three fusion methods *fixed_gain*, *loose* and *tight* that can be selected using `gpsFusionMethod`.

```
python3 run.py 'examples/example_7/'aviation_gps_fixed.ini'
python3 run.py 'examples/example_7/'aviation_gps_fixed_with_zones.ini'
python3 run.py 'examples/example_7/'aviation_gps_loose.ini'
python3 run.py 'examples/example_7/'aviation_gps_loose_with_zones.ini'
python3 run.py 'examples/example_7/'aviation_gps_tight.ini'
python3 run.py 'examples/example_7/'aviation_gps_tight_with_zones.ini'
python3 run.py 'examples/example_7/'aviation_only.ini'
python3 run.py 'examples/example_7/'perfect_only.ini'
python3 run.py 'examples/example_7/'tactical_only.ini'
```

1.11.8 Inertial Quantum Sensors

The use of cold-atom quantum sensor can be configured under the **Quantum** section. This covers which sensors are to be enabled (inertia and/or gravity), their implementation, their configured error profiles and their selected fusion methods. These examples explore the options available for `quantumImu`:

```
python3 run.py 'examples/example_8/'aviation_only.ini'
python3 run.py 'examples/example_8/'aviation_quantum_concept.ini'
python3 run.py 'examples/example_8/'aviation_quantum_realistic.ini'
python3 run.py 'examples/example_8/'perfect_only.ini'
python3 run.py 'examples/example_8/'perfect_quantum_concept.ini'
python3 run.py 'examples/example_8/'perfect_quantum_realistic.ini'
python3 run.py 'examples/example_8/'tactical_only.ini'
python3 run.py 'examples/example_8/'tactical_quantum_concept.ini'
python3 run.py 'examples/example_8/'tactical_quantum_realistic.ini'
```

1.12 Frequently Asked Questions

This section lists all the commonly asked questions about the toolbox. Each block lists a question followed by a short but descriptive answer.

1.12.1 Questions and Answers

Can I extend the functionality of the toolbox?

Yes. All Python classes implemented in the toolbox can be extended/inherited by custom children classes. These custom children classes should be supported by all modules in the toolbox. For example, you may choose to implement a custom Kalman filter than extends from the toolbox's.

Can I run the toolbox on a Raspberry Pi?

Yes. Many modifications have been made to the toolbox to allow it to be used efficiently on systems with limited resources. It is recommended that waypoints are generated a head of time on a more powerful platform, then exported so that they can be imported by other devices. The use of virtual memory is strongly recommended for systems with limited RAM.

Warning

While 32-bit system are supported, it is recommended that a 64-bit OS is used due to the amount of memory (virtual or real) required by the toolbox.

The toolbox is using too much memory. Can I reduce this?

Yes. The easiest way the reduce memory usage is to enable the `useVirtualMemory` options in the configuration file. This will use temporary disk space to offload large arrays from memory.

Can the software be install in an isolated environment?

Yes. The software, Python and the required packages can be installed in an isolated environment without Internet access. This will require the above to be downloaded on a machine with internet access of similar platform, then copied onto the isolated machine (i.e. through the use of write-only media).

For example, the Python installer must be downloaded on a machine with the same operating system as the isolated environment. Using the same machine, the required packages (listed in `requirements.txt`) must be downloaded to a local folder (not installed) via PIP. Once performed the Python installer, downloaded packages and toolbox source code must then be moved to the isolated environment. Once Python is installed on the isolated environment, the python packages can be installed locally. The toolbox can then be used on the isolated machine.

There are many guides online for installing Python packages offline for various platforms. If you need assistance, please email us and we will be happy to help.

Can multi-threading be used to speed up a simulation?

No. Unfortunately the calculations in one iteration are always required by the next. Because of this multi-threading cannot be used efficiently to improve runtime. However, vectorisation has been used where possible to improve processing times for large arrays, namely during waypoint generation.

1.13 Potential Future Work

The produced PNT toolbox is very capable of providing facilities for conducting a wide range of navigation based experiments. However, it can always be improved and additional features can be added anytime. For this reason, this section lists some of the possible ways the toolbox can be extended given a potential project extension.

1.13.1 Possible Extensions

More Sensors and Estimation Algorithms

If needed additional virtual hardware classes could be added, including non-classical sensors. On top of this, more advanced estimation algorithms can be implemented. For example, cold-atom quantum sensors could be introduced to the toolbox along with new algorithms to fuse their measurements with the current estimated states.

Concurrent Simulations

In some cases it is required than experiments are executed to cover a pre-set range of parameters. If needed, the toolbox could be adjusted so that multiple simulations are executed to cover a requested range of configuration values or cover all permutations automatically.

Moreover, since the simulation execution cannot efficiently benefit from multithreading, the independent simulations could be conducted in parallel with each using a separate CPU core. In theory, this would allow the range of configuration settings to be covers in a fraction of the time.

Multi-Vehicle Cooperation

The toolbox is built for simulating a single platform following a set trajectory, taking measurements and using them to update its estimated states. For more advanced experiments, multiple platforms may need to cooperate with one another and collaborate to share estimates. While the toolbox is very capable of doing this (when used as a Python library) there is no set module, function or configuration to conduct it immediately. Nevertheless, the resources to conduct such experiments could be implemented if requested.

Graphical User Interface

If requested, a custom Graphical User Interface (GUI) could be implemented. This would aid the user with configuring experiments, showing produced trajectories and displaying simulation results in real-time. In principle, this would allow the software to be utilised much more easily as a standalone application.

The user interface could be implemented as desktop application using a standard Python framework such as [Tkinter](#). Alternatively, a web-browser based interface could be made, allowing the software to be interfaced with across a local network. This would be more beneficial if the software is to be executed on system that does not feature a display or operating system with a desktop environment (i.e. command-line based interface). By using a web-browser, the interfacing system will not required any software or resources to be installed (including Python).

Numba Optimisation

If faster processing times are required, the toolbox can utilise [Numba](#). This would employ a JIT (Just-In-Time) compiler to convert the Python code to low-level code, obtaining performance comparable to that produced by C-code. In addition, this would utilise hardware such as multiple CPU cores and GPU devices to further accelerate calculations.

1.14 Software Licence

1.14.1 MIT License

Quantum Navigation Toolbox

Copyright (c) 2019 University of Liverpool

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the “Software”), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED “AS IS”, WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

INDICES AND TABLES

- `genindex`
- `modindex`
- `search`

PYTHON MODULE INDEX

q

- qnav.earth.dted, 36
- qnav.earth.geoid, 40
- qnav.earth.utm, 44
- qnav.estimation.state, 45
- qnav.fusion.altimeter, 53
- qnav.fusion.ins, 49
- qnav.fusion.kalman, 51
- qnav.gps.ephemeris, 54
- qnav.gps.fusion, 56
- qnav.gps.satellite, 57
- qnav.gps.sensor, 61
- qnav.gps.zones, 62
- qnav.gravity.base, 65
- qnav.gravity.egm, 77
- qnav.gravity.geosat, 79
- qnav.gravity.ggm_plus, 85
- qnav.gravity.irish, 91
- qnav.gravity.map, 73
- qnav.gravity.marine, 82
- qnav.gravity.nima, 72
- qnav.gravity.simple, 68
- qnav.gravity.somigliana, 70
- qnav.gravity.srtm2gravity, 90
- qnav.input.config, 94
- qnav.input.config_handler, 97
- qnav.measurement.accelerometer, 108
- qnav.measurement.altimeter, 110
- qnav.measurement.clock, 111
- qnav.measurement.error_properties, 106
- qnav.measurement.gyroscope, 109
- qnav.measurement.platform, 105
- qnav.measurement.sensor, 103
- qnav.output.data, 112
- qnav.output.figures, 114
- qnav.output.formats, 113
- qnav.output.summary, 114
- qnav.quantum.base, 115
- qnav.quantum.dummy, 122
- qnav.quantum.gravity_gradient, 119
- qnav.quantum.misalignment, 119
- qnav.quantum.particle_filter, 118
- qnav.quantum.realistic, 117
- qnav.simulation.display, 128
- qnav.simulation.holonomic, 126
- qnav.simulation.scheduling, 124
- qnav.simulation.simulation, 123
- qnav.util.connections, 132
- qnav.util.constants, 131
- qnav.util.rng, 133
- qnav.util.system, 132
- qnav.util.transformations, 24
- qnav.waypoints.generation, 134
- qnav.waypoints.tracks, 145
- qnav.waypoints.trajectory, 140
- qnav.waypoints.vehicle, 142

INDEX

A

acceleration (*qnav.estimation.state.EstimatedState* property), 45
acceleration (*qnav.waypoints.trajectory.GroundTruth* attribute), 140
acceleration_max (*qnav.waypoints.vehicle.Vehicle* attribute), 143
Accelerometer (class in *qnav.measurement.accelerometer*), 108
AdamsBashforth (class in *qnav.fusion.ins*), 50
alpha (*qnav.fusion.altimeter.AlphaBetaAltimeter* property), 53
AlphaBetaAltimeter (class in *qnav.fusion.altimeter*), 53
Altimeter (class in *qnav.measurement.altimeter*), 111
ang_diff() (in module *qnav.quantum.realistic*), 117
angle_rates (*qnav.estimation.state.EstimatedState* property), 46
angle_rates (*qnav.waypoints.trajectory.GroundTruth* attribute), 140
append_config() (*qnav.input.config_handler.ConfigHandler* method), 97
as_array() (*qnav.waypoints.trajectory.Waypoints* method), 141
as_dict() (*qnav.estimation.state.EstimatedState* method), 46
as_dict() (*qnav.waypoints.trajectory.GroundTruth* method), 140
as_numpy() (*qnav.estimation.state.EstimatedState* method), 47
as_numpy() (*qnav.waypoints.trajectory.GroundTruth* method), 140
attitude (*qnav.estimation.state.EstimatedState* property), 46
attitude (*qnav.waypoints.trajectory.GroundTruth* attribute), 140
avg_meas_noise (*qnav.measurement.error_properties.ErrorProperties* property), 108

B

base_model (*qnav.gravity.base.GravityModel* property), 68

base_model (*qnav.gravity.map.GravityMap* property), 76
beta (*qnav.fusion.altimeter.AlphaBetaAltimeter* property), 53
bias_drift_rate (*qnav.measurement.error_properties.ErrorProperties* property), 107
bias_error (*qnav.measurement.error_properties.ErrorProperties* property), 107
body2sensor_mat (*qnav.measurement.platform.SensorAxis* property), 106

C

c_ic (*qnav.gps.satellite.SatelliteParams* attribute), 58
c_is (*qnav.gps.satellite.SatelliteParams* attribute), 58
c_rc (*qnav.gps.satellite.SatelliteParams* attribute), 58
c_rs (*qnav.gps.satellite.SatelliteParams* attribute), 58
c_uc (*qnav.gps.satellite.SatelliteParams* attribute), 58
c_us (*qnav.gps.satellite.SatelliteParams* attribute), 58
calc_gravity_xyz() (*qnav.gravity.base.GravityModel* method), 66
calc_gravity_xyz() (*qnav.gravity.map.GravityMap* method), 75
calc_gravity_xyz() (*qnav.gravity.nima.WGS84Gravity* method), 72
calc_gravity_xyz() (*qnav.gravity.somigliana.Somigliana* method), 70
calc_gravity_xyz_vec() (*qnav.gravity.base.GravityModel* method), 67
calc_gravity_xyz_vec() (*qnav.gravity.map.GravityMap* method), 75
calc_gravity_xyz_vec() (*qnav.gravity.somigliana.Somigliana* method), 71
calc_gravity_z() (*qnav.gravity.base.GravityModel* method), 65
calc_gravity_z() (*qnav.gravity.egm.GeoidCorrection* method), 77
calc_gravity_z() (*qnav.gravity.geosat.Geosat44* method), 79
calc_gravity_z() (*qnav.gravity.ggm_plus.GGMPlusAcc*

- method), 85
- calc_gravity_z() (qnav.gravity.ggm_plus.GGMPlusDist method), 89
- calc_gravity_z() (qnav.gravity.irish.IrishSeaGravity method), 91
- calc_gravity_z() (qnav.gravity.marine.MarineGravity method), 82
- calc_gravity_z() (qnav.gravity.nima.WGS84Gravity method), 72
- calc_gravity_z() (qnav.gravity.simple.FixedValue method), 68
- calc_gravity_z() (qnav.gravity.simple.SimpleUniform method), 69
- calc_gravity_z() (qnav.gravity.somigliana.Somigliana method), 70
- calc_gravity_z_vec() (qnav.gravity.base.GravityModel method), 67
- calc_gravity_z_vec() (qnav.gravity.egm.GeoidCorrection method), 77
- calc_gravity_z_vec() (qnav.gravity.geosat.Geosat44 method), 79
- calc_gravity_z_vec() (qnav.gravity.ggm_plus.GGMPlusAcc method), 86
- calc_gravity_z_vec() (qnav.gravity.ggm_plus.GGMPlusDist method), 89
- calc_gravity_z_vec() (qnav.gravity.irish.IrishSeaGravity method), 92
- calc_gravity_z_vec() (qnav.gravity.marine.MarineGravity method), 83
- calc_gravity_z_vec() (qnav.gravity.simple.FixedValue method), 69
- calc_gravity_z_vec() (qnav.gravity.simple.SimpleUniform method), 69
- calc_gravity_z_vec() (qnav.gravity.somigliana.Somigliana method), 71
- calc_vertical_grad() (qnav.gravity.base.GravityModel method), 66
- calc_vertical_grad() (qnav.gravity.map.GravityMap method), 75
- calc_vertical_grad_vec() (qnav.gravity.base.GravityModel method), 67
- calc_vertical_grad_vec() (qnav.gravity.map.GravityMap method), 76
- calculate_cov_mat() (in module qnav.gps.fusion), 57
- calculate_curvature() (in module qnav.gravity.base), 68
- CircularGrid (class in qnav.gravity.map), 73
- clear_cache() (qnav.earth.dted.DtedHandler method), 39
- clear_measurement() (qnav.simulation.holonomic.HolonomicConstraint method), 126
- clear_terminal_line() (in module qnav.simulation.display), 128
- Clock (class in qnav.measurement.clock), 111
- clock_bias (qnav.gps.satellite.SatelliteParams attribute), 58
- clone() (qnav.estimation.state.EstimatedState method), 47
- ColdAtomInterferometer (class in qnav.quantum.realistic), 117
- ConceptQuantumFusion (class in qnav.quantum.base), 116
- ConceptQuantumImu (class in qnav.quantum.base), 115
- ConfigHandler (class in qnav.input.config_handler), 97
- ConsoleDisplay (class in qnav.simulation.display), 129
- ConsoleDisplayPlus (class in qnav.simulation.display), 130
- copy_file() (qnav.input.config_handler.ConfigHandler method), 102
- create_interp_grid() (in module qnav.gravity.geosat), 82
- crop_waypoints() (in module qnav.waypoints.generation), 139
- cross_prod_xy() (in module qnav.util.transformations), 35
- cumulative_distance() (in module qnav.waypoints.trajectory), 142
- ## D
- d_roll_d_turn_rate (qnav.waypoints.vehicle.Vehicle attribute), 143
- day (qnav.gps.satellite.SatelliteParams attribute), 58
- deg2deg() (in module qnav.util.transformations), 25
- deceleration_max (qnav.waypoints.vehicle.Vehicle attribute), 143
- deg2dec() (in module qnav.util.transformations), 25
- deg2rad() (in module qnav.util.transformations), 24
- delta_n (qnav.gps.satellite.SatelliteParams attribute), 58
- description (qnav.waypoints.vehicle.Vehicle attribute), 143
- Display (class in qnav.simulation.display), 128
- distance_to_centre() (qnav.earth.dted.DtedTile method), 38
- download_data() (qnav.gravity.geosat.Geosat44 method), 81

- download_data() (*qnav.gravity.ggm_plus.GGMPlusAcc method*), 88
- download_data() (*qnav.gravity.marine.MarineGravity method*), 85
- download_file() (*in module qnav.util.connections*), 132
- download_rinex_file() (*in module qnav.gps.ephemeris*), 54
- download_rinex_files() (*in module qnav.gps.ephemeris*), 55
- drift_freq_bias (*qnav.gps.satellite.SatelliteParams attribute*), 58
- drift_rate_message_time (*qnav.gps.satellite.SatelliteParams attribute*), 58
- DtedHandler (*class in qnav.earth.dted*), 38
- DtedTile (*class in qnav.earth.dted*), 36
- DummyAccelerometer (*class in qnav.quantum.dummy*), 123
- DummyClock (*class in qnav.measurement.clock*), 112
- DummyGyroscope (*class in qnav.quantum.dummy*), 123
- DummySensor (*class in qnav.quantum.dummy*), 122
- ## E
- e (*qnav.gps.satellite.SatelliteParams attribute*), 58
- ECC (*in module qnav.util.constants*), 131
- ECC_PRIME (*in module qnav.util.constants*), 131
- ECC_SQ (*in module qnav.util.constants*), 131
- ecef2eci() (*in module qnav.util.transformations*), 32
- ecef2lla() (*in module qnav.util.transformations*), 32
- ecef2lla_vec() (*in module qnav.util.transformations*), 32
- ecef2ned() (*in module qnav.util.transformations*), 33
- ecef2ned_transform() (*in module qnav.util.transformations*), 33
- ecef2ned_transform_vec() (*in module qnav.util.transformations*), 33
- eci2ecef() (*in module qnav.util.transformations*), 32
- end_time (*qnav.waypoints.trajectory.Trajectory property*), 142
- error_matrix (*qnav.measurement.error_properties.ErrorProperties method*), 108
- ErrorProperties (*class in qnav.measurement.error_properties*), 106
- EstimatedState (*class in qnav.estimation.state*), 45
- euler_to_quaternion() (*in module qnav.util.transformations*), 29
- euler_to_quaternion_vec() (*in module qnav.util.transformations*), 29
- expand_section() (*qnav.input.config_handler.ConfigHandler method*), 98
- ## F
- F (*in module qnav.util.constants*), 131
- FileFormat (*class in qnav.output.formats*), 113
- filter_gain (*qnav.simulation.holonomic.HolonomicConstraint property*), 126
- filter_id (*qnav.simulation.holonomic.HolonomicConstraint property*), 126
- fixed_gain (*qnav.fusion.altimeter.FixedGainAltimeter property*), 53
- FixedGainAltimeter (*class in qnav.fusion.altimeter*), 53
- FixedValue (*class in qnav.gravity.simple*), 68
- FlaggedPolygon (*class in qnav.gps.zones*), 63
- FlaggedPolygonGroup (*class in qnav.gps.zones*), 63
- frequency (*qnav.measurement.sensor.Sensor property*), 104
- FusionTrigger (*class in qnav.measurement.sensor*), 104
- ## G
- G (*in module qnav.util.constants*), 131
- G_E (*in module qnav.util.constants*), 131
- G_P (*in module qnav.util.constants*), 131
- GaussianMarkov (*class in qnav.util.rng*), 133
- generate_grid() (*qnav.gravity.map.CircularGrid method*), 73
- generate_racetrack() (*in module qnav.waypoints.tracks*), 145
- GeoidCorrection (*class in qnav.gravity.egm*), 77
- GeoidDummy (*class in qnav.earth.geoid*), 42
- GeoidModel (*class in qnav.earth.geoid*), 40
- GeoidPGM (*class in qnav.earth.geoid*), 41
- Geosat44 (*class in qnav.gravity.geosat*), 79
- get_all_values() (*qnav.input.config_handler.ConfigHandler method*), 98
- get_anomaly() (*qnav.gravity.egm.GeoidCorrection method*), 77
- get_anomaly() (*qnav.gravity.geosat.Geosat44 method*), 80
- get_anomaly() (*qnav.gravity.ggm_plus.GGMPlusAcc method*), 86
- get_anomaly() (*qnav.gravity.irish.IrishSeaGravity method*), 92
- get_anomaly() (*qnav.gravity.map.GravityMap method*), 74
- get_anomaly() (*qnav.gravity.marine.MarineGravity method*), 83
- get_anomaly_vec() (*qnav.gravity.egm.GeoidCorrection method*), 78
- get_anomaly_vec() (*qnav.gravity.geosat.Geosat44 method*), 80
- get_anomaly_vec() (*qnav.gravity.ggm_plus.GGMPlusAcc method*), 86
- get_anomaly_vec() (*qnav.gravity.irish.IrishSeaGravity method*), 92

<code>get_anomaly_vec()</code> (<i>qnav.gravity.map.GravityMap method</i>), 74	<code>get_dov_vec()</code> (<i>qnav.earth.geoid.GeoidDummy method</i>), 43
<code>get_anomaly_vec()</code> (<i>qnav.gravity.marine.MarineGravity method</i>), 84	<code>get_dov_vec()</code> (<i>qnav.earth.geoid.GeoidModel method</i>), 41
<code>get_bool()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 100	<code>get_dov_vec()</code> (<i>qnav.earth.geoid.GeoidPGM method</i>), 42
<code>get_collector()</code> (<i>in module qnav.input.config</i>), 95	<code>get_dted_file_path()</code> (<i>in module qnav.earth.dted</i>), 39
<code>get_config_name()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 97	<code>get_elevation()</code> (<i>qnav.earth.dted.DtedHandler method</i>), 38
<code>get_configuration()</code> (<i>in module qnav.input.config</i>), 94	<code>get_elevation()</code> (<i>qnav.earth.dted.DtedTile method</i>), 36
<code>get_csv_numeric()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 100	<code>get_elevation_vec()</code> (<i>qnav.earth.dted.DtedHandler method</i>), 38
<code>get_csv_str()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 101	<code>get_elevation_vec()</code> (<i>qnav.earth.dted.DtedTile method</i>), 36
<code>get_default_profile()</code> (<i>in module qnav.waypoints.vehicle</i>), 144	<code>get_empty_ggm_tile()</code> (<i>in module qnav.gravity.ggm_plus</i>), 90
<code>get_displayer()</code> (<i>in module qnav.input.config</i>), 96	<code>get_empty_srtm_tile()</code> (<i>in module qnav.gravity.srtm2gravity</i>), 91
<code>get_disturbance()</code> (<i>qnav.gravity.egm.GeoidCorrection method</i>), 78	<code>get_estimation()</code> (<i>in module qnav.input.config</i>), 95
<code>get_disturbance()</code> (<i>qnav.gravity.geosat.Geosat44 method</i>), 81	<code>get_figure_settings()</code> (<i>in module qnav.input.config</i>), 96
<code>get_disturbance()</code> (<i>qnav.gravity.ggm_plus.GGMPlusAcc method</i>), 87	<code>get_float()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 100
<code>get_disturbance()</code> (<i>qnav.gravity.ggm_plus.GGMPlusDist method</i>), 89	<code>get_height()</code> (<i>qnav.earth.geoid.GeoidDummy method</i>), 42
<code>get_disturbance()</code> (<i>qnav.gravity.irish.IrishSeaGravity method</i>), 93	<code>get_height()</code> (<i>qnav.earth.geoid.GeoidModel method</i>), 40
<code>get_disturbance()</code> (<i>qnav.gravity.map.GravityMap method</i>), 74	<code>get_height()</code> (<i>qnav.earth.geoid.GeoidPGM method</i>), 41
<code>get_disturbance()</code> (<i>qnav.gravity.marine.MarineGravity method</i>), 84	<code>get_height_vec()</code> (<i>qnav.earth.geoid.GeoidDummy method</i>), 42
<code>get_disturbance_vec()</code> (<i>qnav.gravity.egm.GeoidCorrection method</i>), 78	<code>get_height_vec()</code> (<i>qnav.earth.geoid.GeoidModel method</i>), 40
<code>get_disturbance_vec()</code> (<i>qnav.gravity.geosat.Geosat44 method</i>), 81	<code>get_height_vec()</code> (<i>qnav.earth.geoid.GeoidPGM method</i>), 41
<code>get_disturbance_vec()</code> (<i>qnav.gravity.ggm_plus.GGMPlusAcc method</i>), 87	<code>get_int()</code> (<i>qnav.input.config_handler.ConfigHandler method</i>), 99
<code>get_disturbance_vec()</code> (<i>qnav.gravity.ggm_plus.GGMPlusDist method</i>), 90	<code>get_large_aircraft_profile()</code> (<i>in module qnav.waypoints.vehicle</i>), 144
<code>get_disturbance_vec()</code> (<i>qnav.gravity.irish.IrishSeaGravity method</i>), 93	<code>get_large_ship_profile()</code> (<i>in module qnav.waypoints.vehicle</i>), 144
<code>get_disturbance_vec()</code> (<i>qnav.gravity.map.GravityMap method</i>), 74	<code>get_large_van_profile()</code> (<i>in module qnav.waypoints.vehicle</i>), 144
<code>get_disturbance_vec()</code> (<i>qnav.gravity.marine.MarineGravity method</i>), 84	<code>get_location_azimuth_elevation()</code> (<i>qnav.gps.satellite.Satellite method</i>), 59
<code>get_dov()</code> (<i>qnav.earth.geoid.GeoidDummy method</i>), 43	<code>get_map_value()</code> (<i>qnav.gravity.geosat.Geosat44 method</i>), 80
<code>get_dov()</code> (<i>qnav.earth.geoid.GeoidModel method</i>), 40	<code>get_map_value()</code> (<i>qnav.gravity.ggm_plus.GGMPlusAcc method</i>), 87
<code>get_dov()</code> (<i>qnav.earth.geoid.GeoidPGM method</i>), 41	<code>get_map_value()</code> (<i>qnav.gravity.ggm_plus.GGMPlusDist method</i>), 88
	<code>get_map_value()</code> (<i>qnav.gravity.irish.IrishSeaGravity</i>

method), 91
 get_map_value() (qnav.gravity.marine.MarineGravity method), 83
 get_map_values() (qnav.gravity.ggm_plus.GGMPlusAcc method), 88
 get_map_values() (qnav.gravity.ggm_plus.GGMPlusDist method), 88
 get_measurements() (qnav.gps.satellite.Satellite method), 59
 get_next_fusion() (qnav.simulation.scheduling.TimingHandler method), 125
 get_next_sensor() (qnav.simulation.scheduling.TimingHandler method), 125
 get_platform_clock() (in module qnav.input.config), 95
 get_platform_details() (in module qnav.util.system), 132
 get_platform_hardware() (in module qnav.input.config), 95
 get_point_flag() (qnav.gps.zones.FlaggedPolygonGroup method), 63
 get_point_flags() (qnav.gps.zones.FlaggedPolygonGroup method), 64
 get_positions() (in module qnav.waypoints.generation), 135
 get_random() (qnav.util.rng.GaussianMarkov method), 133
 get_record() (qnav.waypoints.trajectory.Trajectory method), 141
 get_records() (qnav.waypoints.trajectory.Trajectory method), 141
 get_results_settings() (in module qnav.input.config), 96
 get_rinex_path() (in module qnav.gps.ephemeris), 54
 get_sections() (qnav.input.config_handler.ConfigHandler method), 98
 get_simple_trajectory_values() (in module qnav.waypoints.generation), 136
 get_str() (qnav.input.config_handler.ConfigHandler method), 99
 get_str_alpha() (qnav.input.config_handler.ConfigHandler method), 99
 get_submarine_profile() (in module qnav.waypoints.vehicle), 145
 get_system_name() (in module qnav.util.system), 132
 get_template_dted_data() (in module qnav.earth.dted), 39
 get_time() (qnav.measurement.clock.Clock method), 112
 get_time() (qnav.measurement.clock.DummyClock method), 112
 get_time_pos_vel() (in module qnav.util.transformations), 31
 get_toolbox_version() (in module qnav.util.system), 132
 get_trajectory() (in module qnav.input.config), 94
 get_trajectory() (in module qnav.waypoints.generation), 139
 get_trajectory_values() (in module qnav.waypoints.generation), 137
 get_transport_rate() (in module qnav.util.transformations), 35
 get_transport_rate_vec() (in module qnav.util.transformations), 35
 get_unrestricted_profile() (in module qnav.waypoints.vehicle), 145
 get_value_filepath() (qnav.input.config_handler.ConfigHandler method), 101
 get_value_filepath_str() (qnav.input.config_handler.ConfigHandler method), 101
 GGMPlusAcc (class in qnav.gravity.ggm_plus), 85
 GGMPlusDist (class in qnav.gravity.ggm_plus), 88
 GM (in module qnav.util.constants), 131
 GpsFixedGainFusion (class in qnav.gps.fusion), 56
 GpsFusion (class in qnav.gps.fusion), 56
 GpsLooseFusion (class in qnav.gps.fusion), 56
 GpsSensor (class in qnav.gps.sensor), 61
 GpsTightFusion (class in qnav.gps.fusion), 56
 gravity_model (qnav.estimation.state.EstimatedState property), 46
 gravity_model (qnav.quantum.gravity_gradient.GravityGradiometer property), 121
 GravityGradientPF (class in qnav.quantum.gravity_gradient), 121
 GravityGradInterferometer (class in qnav.quantum.gravity_gradient), 119
 GravityGradiometer (class in qnav.quantum.gravity_gradient), 120
 GravityMap (class in qnav.gravity.map), 74
 GravityModel (class in qnav.gravity.base), 65
 GroundTruth (class in qnav.waypoints.trajectory), 140
 Gyroscope (class in qnav.measurement.gyroscope), 109

H

haversine() (in module qnav.util.transformations), 30
 haversine_vec() (in module qnav.util.transformations), 30
 HolonomicConstraint (class in qnav.simulation.holonomic), 126
 HolonomicFusion (class in qnav.simulation.holonomic), 127
 hour (qnav.gps.satellite.SatelliteParams attribute), 58

I

i_0 (qnav.gps.satellite.SatelliteParams attribute), 58
 i_dot (qnav.gps.satellite.SatelliteParams attribute), 58

- [id](#) (*qnav.gps.satellite.SatelliteParams* attribute), 58
[init_particle_filter\(\)](#) (*qnav.quantum.gravity_gradient.GravityGradientInterferometer* property), 122
[interferometer](#) (*qnav.quantum.gravity_gradient.GravityGradientInterferometer* property), 121
[interp_method](#) (*qnav.waypoints.trajectory.Trajectory* property), 142
[interpolate_positions\(\)](#) (in module *qnav.waypoints.generation*), 135
[IrishSeaGravity](#) (class in *qnav.gravity.irish*), 91
[is_active_area](#) (*qnav.gps.zones.FlaggedPolygon* property), 63
[is_data_loaded](#) (*qnav.output.data.ResultsTable* property), 112
[is_inside_polygon\(\)](#) (in module *qnav.gps.zones*), 64
[is_inside_polygon_vec\(\)](#) (in module *qnav.gps.zones*), 64
[is_linux_platform\(\)](#) (in module *qnav.util.system*), 133
[is_mac_platform\(\)](#) (in module *qnav.util.system*), 133
[is_point_inside\(\)](#) (*qnav.gps.zones.Polygon* method), 62
[is_points_inside\(\)](#) (*qnav.gps.zones.Polygon* method), 62
[is_position_covered\(\)](#) (*qnav.earth.dted.DtedTile* method), 37
[is_position_covered_vec\(\)](#) (*qnav.earth.dted.DtedTile* method), 37
[is_using_terminal\(\)](#) (in module *qnav.simulation.display*), 128
[is_value_set\(\)](#) (*qnav.input.config_handler.ConfigHandler* method), 102
[is_window_platform\(\)](#) (in module *qnav.util.system*), 133
- ## K
- [k_eff](#) (*qnav.quantum.gravity_gradient.GravityGradientInterferometer* property), 120
[KalmanEstimatedState](#) (class in *qnav.estimation.state*), 47
[KalmanINS](#) (class in *qnav.fusion.kalman*), 52
- ## L
- [last_measurement](#) (*qnav.gps.sensor.GpsSensor* property), 61
[last_measurement](#) (*qnav.measurement.accelerometer.Accelerometer* property), 109
[last_measurement](#) (*qnav.measurement.altimeter.Altimeter* property), 111
[last_measurement](#) (*qnav.measurement.gyroscope.Gyroscope* property), 110
[last_measurement](#) (*qnav.measurement.sensor.Sensor* property), 103
[last_measurement](#) (*qnav.quantum.base.ConceptQuantumImu* property), 115
[last_measurement](#) (*qnav.quantum.dummy.DummySensor* property), 123
[last_measurement](#) (*qnav.quantum.gravity_gradient.GravityGradientInterferometer* property), 121
[last_measurement](#) (*qnav.simulation.holonomic.HolonomicConstraint* property), 127
[last_time](#) (*qnav.util.rng.GaussianMarkov* property), 134
[last_update](#) (*qnav.measurement.sensor.Sensor* property), 104
[last_update](#) (*qnav.quantum.dummy.DummySensor* property), 123
[last_value](#) (*qnav.util.rng.GaussianMarkov* property), 134
[lat_points](#) (*qnav.gps.zones.Polygon* property), 62
[lever_arm](#) (*qnav.measurement.platform.SensorAxis* property), 106
[lla2cecf\(\)](#) (in module *qnav.util.transformations*), 28
[lla2cecf_vec\(\)](#) (in module *qnav.util.transformations*), 29
[lla2ned\(\)](#) (in module *qnav.util.transformations*), 27
[lla2ned_vec\(\)](#) (in module *qnav.util.transformations*), 27
[load_configuration\(\)](#) (*qnav.input.config_handler.ConfigHandler* method), 97
[load_data\(\)](#) (*qnav.gravity.geosat.Geosat44* method), 81
[load_data\(\)](#) (*qnav.gravity.irish.IrishSeaGravity* method), 91
[load_data\(\)](#) (*qnav.gravity.marine.MarineGravity* method), 85
[load_waypoint_csv\(\)](#) (in module *qnav.waypoints.generation*), 138
[load_waypoints\(\)](#) (in module *qnav.waypoints.generation*), 139
[lon_points](#) (*qnav.gps.zones.Polygon* property), 62
- ## M
- [M](#) (in module *qnav.util.constants*), 131
[m_bar_0](#) (*qnav.gps.satellite.SatelliteParams* attribute), 58
[MarineGravity](#) (class in *qnav.gravity.marine*), 82
[minute](#) (*qnav.gps.satellite.SatelliteParams* attribute), 58
[module](#) *qnav.earth.dted*, 36
[module](#) *qnav.earth.geoid*, 40
[module](#) *qnav.earth.utm*, 44
[module](#) *qnav.estimation.state*, 45
[module](#) *qnav.fusion.altimeter*, 53
[module](#) *qnav.fusion.ins*, 49
[module](#) *qnav.fusion.kalman*, 51
[module](#) *qnav.gps.ephemeris*, 54

qnav.gps.fusion, 56
 qnav.gps.satellite, 57
 qnav.gps.sensor, 61
 qnav.gps.zones, 62
 qnav.gravity.base, 65
 qnav.gravity.egm, 77
 qnav.gravity.geosat, 79
 qnav.gravity.ggm_plus, 85
 qnav.gravity.irish, 91
 qnav.gravity.map, 73
 qnav.gravity.marine, 82
 qnav.gravity.nima, 72
 qnav.gravity.simple, 68
 qnav.gravity.somigliana, 70
 qnav.gravity.srtm2gravity, 90
 qnav.input.config, 94
 qnav.input.config_handler, 97
 qnav.measurement.accelerometer, 108
 qnav.measurement.altimeter, 110
 qnav.measurement.clock, 111
 qnav.measurement.error_properties, 106
 qnav.measurement.gyroscope, 109
 qnav.measurement.platform, 105
 qnav.measurement.sensor, 103
 qnav.output.data, 112
 qnav.output.figures, 114
 qnav.output.formats, 113
 qnav.output.summary, 114
 qnav.quantum.base, 115
 qnav.quantum.dummy, 122
 qnav.quantum.gravity_gradient, 119
 qnav.quantum.misalignment, 119
 qnav.quantum.particle_filter, 118
 qnav.quantum.realistic, 117
 qnav.simulation.display, 128
 qnav.simulation.holonomic, 126
 qnav.simulation.scheduling, 124
 qnav.simulation.simulation, 123, 127
 qnav.util.connections, 132
 qnav.util.constants, 131
 qnav.util.rng, 133
 qnav.util.system, 132
 qnav.util.transformations, 24
 qnav.waypoints.generation, 134
 qnav.waypoints.tracks, 145
 qnav.waypoints.trajectory, 140
 qnav.waypoints.vehicle, 142
 month (*qnav.gps.satellite.SatelliteParams* attribute), 58
 moving_average() (in module *qnav.waypoints.generation*), 136
 MultilineDisplay (class in *qnav.simulation.display*), 130

N

NavConfigError, 103
 NavConfigWarning, 102
 ned2azel() (in module *qnav.util.transformations*), 34
 ned2ecef() (in module *qnav.util.transformations*), 33
 ned2ecef_vec() (in module *qnav.util.transformations*), 34
 ned2lla() (in module *qnav.util.transformations*), 28
 ned2lla_vec() (in module *qnav.util.transformations*), 28
 ned2state_vector() (in module *qnav.fusion.kalman*), 52
 next_update (*qnav.measurement.sensor.Sensor* property), 104
 next_update (*qnav.quantum.dummy.DummySensor* property), 123
 non_orth_error (*qnav.measurement.error_properties.ErrorProperties* property), 108
 num_active_steps (*qnav.quantum.base.ConceptQuantumImu* property), 115
 num_polygons (*qnav.gps.zones.FlaggedPolygonGroup* property), 64
 num_steps (*qnav.quantum.base.ConceptQuantumImu* property), 116
 num_steps (*qnav.quantum.gravity_gradient.GravityGradiometer* property), 121
 NumericalINS (class in *qnav.fusion.ins*), 49

O

omega_0 (*qnav.gps.satellite.SatelliteParams* attribute), 58
 omega_dot (*qnav.gps.satellite.SatelliteParams* attribute), 59
 OMEGA_E (in module *qnav.util.constants*), 131
 on_finish() (*qnav.simulation.display.ConsoleDisplay* method), 129
 on_finish() (*qnav.simulation.display.ConsoleDisplayPlus* method), 130
 on_finish() (*qnav.simulation.display.Display* method), 129
 on_finish() (*qnav.simulation.display.MultilineDisplay* method), 130
 on_start() (*qnav.simulation.display.ConsoleDisplay* method), 129
 on_start() (*qnav.simulation.display.Display* method), 128
 on_start() (*qnav.simulation.display.MultilineDisplay* method), 130
 on_update() (*qnav.simulation.display.ConsoleDisplay* method), 129
 on_update() (*qnav.simulation.display.ConsoleDisplayPlus* method), 130
 on_update() (*qnav.simulation.display.Display* method), 129

`on_update()` (*qnav.simulation.display.MultilineDisplay* method), 130

`output_shape` (*qnav.util.rng.GaussianMarkov* property), 134

`overwrite_value()` (*qnav.input.config_handler.ConfigHandler* method), 102

P

`ParticleFilterParams` (class in *qnav.quantum.gravity_gradient*), 121

`ParticleFilterParams` (class in *qnav.quantum.particle_filter*), 118

`perform_fusion()` (*qnav.fusion.altimeter.AlphaBetaAltimeter* method), 53

`perform_fusion()` (*qnav.fusion.altimeter.FixedGainAltimeter* method), 53

`perform_fusion()` (*qnav.fusion.ins.AdamsBashforth* method), 51

`perform_fusion()` (*qnav.fusion.ins.NumericalINS* method), 50

`perform_fusion()` (*qnav.fusion.ins.RungeKutta* method), 50

`perform_fusion()` (*qnav.fusion.kalman.KalmanINS* method), 52

`perform_fusion()` (*qnav.gps.fusion.GpsFixedGainFusion* method), 56

`perform_fusion()` (*qnav.gps.fusion.GpsLooseFusion* method), 56

`perform_fusion()` (*qnav.gps.fusion.GpsTightFusion* method), 57

`perform_fusion()` (*qnav.measurement.sensor.SensorFusion* method), 105

`perform_fusion()` (*qnav.quantum.base.ConceptQuantumFusion* method), 116

`perform_fusion()` (*qnav.quantum.gravity_gradient.GravityGradient* method), 122

`perform_fusion()` (*qnav.simulation.holonomic.HolonomicFusion* method), 127

`pitch_constant_offset` (*qnav.waypoints.vehicle.Vehicle* attribute), 143

`pitch_max` (*qnav.waypoints.vehicle.Vehicle* attribute), 143

`pitch_osc_magnitude` (*qnav.waypoints.vehicle.Vehicle* attribute), 143

`pitch_osc_period` (*qnav.waypoints.vehicle.Vehicle* attribute), 143

`plot_figures()` (in module *qnav.waypoints.generation*), 138

`plot_waypoints()` (in module *qnav.waypoints.generation*), 138

`POLAR_AXIS_A` (in module *qnav.util.constants*), 131

`Polygon` (class in *qnav.gps.zones*), 62

`polygons` (*qnav.gps.zones.FlaggedPolygonGroup* property), 63

`position` (*qnav.estimation.state.EstimatedState* property), 45

`position` (*qnav.gps.satellite.Satellite* property), 60

`position` (*qnav.waypoints.trajectory.GroundTruth* attribute), 140

`print_banner()` (in module *qnav.input.config*), 96

`print_line()` (in module *qnav.input.config*), 97

`print_mappings()` (*qnav.simulation.scheduling.TimingHandler* method), 125

`priority` (*qnav.gps.zones.Polygon* property), 63

`prop` (*qnav.gps.satellite.SatelliteParams* attribute), 59

`produce_noise()` (in module *qnav.waypoints.generation*), 135

`pseudo_range_noise` (*qnav.gps.satellite.Satellite* property), 60

`pseudo_range_rate_noise` (*qnav.gps.satellite.Satellite* property), 60

Q

`qnav.earth.dted` module, 36

`qnav.earth.geoid` module, 40

`qnav.earth.utm` module, 44

`qnav.estimation.state` module, 45

`qnav.fusion.altimeter` module, 53

`qnav.fusion.ins` module, 49

`qnav.fusion.kalman` module, 51

`qnav.gps.ephemeris` module, 54

`qnav.gps.fusion` module, 56

`qnav.gps.satellite` module, 57

`qnav.gps.sensor` module, 61

`qnav.gps.zones` module, 62

`qnav.gravity.base` module, 65

`qnav.gravity.egm` module, 77

`qnav.gravity.geosat` module, 79

`qnav.gravity.ggm_plus` module, 85

`qnav.gravity.irish`

- module, 91
- qnav.gravity.map
 - module, 73
- qnav.gravity.marine
 - module, 82
- qnav.gravity.nima
 - module, 72
- qnav.gravity.simple
 - module, 68
- qnav.gravity.somigliana
 - module, 70
- qnav.gravity.srtm2gravity
 - module, 90
- qnav.input.config
 - module, 94
- qnav.input.config_handler
 - module, 97
- qnav.measurement.accelerometer
 - module, 108
- qnav.measurement.altimeter
 - module, 110
- qnav.measurement.clock
 - module, 111
- qnav.measurement.error_properties
 - module, 106
- qnav.measurement.gyroscope
 - module, 109
- qnav.measurement.platform
 - module, 105
- qnav.measurement.sensor
 - module, 103
- qnav.output.data
 - module, 112
- qnav.output.figures
 - module, 114
- qnav.output.formats
 - module, 113
- qnav.output.summary
 - module, 114
- qnav.quantum.base
 - module, 115
- qnav.quantum.dummy
 - module, 122
- qnav.quantum.gravity_gradient
 - module, 119
- qnav.quantum.misalignment
 - module, 119
- qnav.quantum.particle_filter
 - module, 118
- qnav.quantum.realistic
 - module, 117
- qnav.simulation.display
 - module, 128
- qnav.simulation.holonomic

- module, 126
- qnav.simulation.scheduling
 - module, 124
- qnav.simulation.simulation
 - module, 123, 127
- qnav.util.connections
 - module, 132
- qnav.util.constants
 - module, 131
- qnav.util.rng
 - module, 133
- qnav.util.system
 - module, 132
- qnav.util.transformations
 - module, 24
- qnav.waypoints.generation
 - module, 134
- qnav.waypoints.tracks
 - module, 145
- qnav.waypoints.trajectory
 - module, 140
- qnav.waypoints.vehicle
 - module, 142
- QuantumIMU (*class in qnav.quantum.realistic*), 117
- QuantumPFFusion (*class in qnav.quantum.particle_filter*), 118
- QuantumPFMFusion (*class in qnav.quantum.misalignment*), 119
- quaternion_to_euler() (*in module qnav.util.transformations*), 29
- quaternion_to_euler_vec() (*in module qnav.util.transformations*), 30
- quick_append_config() (*qnav.input.config_handler.ConfigHandler method*), 98

R

- R_E (*in module qnav.util.constants*), 131
- rad2deg() (*in module qnav.util.transformations*), 25
- radius() (*in module qnav.util.transformations*), 25
- radius84() (*in module qnav.util.transformations*), 26
- radius84_vec() (*in module qnav.util.transformations*), 26
- radius_vec() (*in module qnav.util.transformations*), 25
- rand_seed (*qnav.waypoints.vehicle.Vehicle attribute*), 144
- read_csv_waypoints() (*in module qnav.waypoints.generation*), 139
- read_dted_file() (*in module qnav.earth.dted*), 39
- read_geosat_file() (*in module qnav.gravity.geosat*), 81
- read_ggm_file() (*in module qnav.gravity.ggm_plus*), 90

read_irish_ers_file() (in module *qnav.gravity.irish*), 94
 read_irish_gravity_file() (in module *qnav.gravity.irish*), 93
 read_marine_file() (in module *qnav.gravity.marine*), 85
 read_pgm_data() (in module *qnav.earth.geoid*), 43
 read_pgm_header() (in module *qnav.earth.geoid*), 44
 read_rinex_file() (in module *qnav.gps.ephemeris*), 55
 read_srtm_file() (in module *qnav.gravity.srtm2gravity*), 91
 rescale_values() (in module *qnav.waypoints.generation*), 136
 reset() (*qnav.quantum.base.ConceptQuantumFusion* method), 116
 reset() (*qnav.quantum.base.ConceptQuantumImu* method), 115
 reset() (*qnav.quantum.gravity_gradient.GravityGradient* method), 122
 reset() (*qnav.quantum.gravity_gradient.GravityGradient* method), 120
 reset() (*qnav.quantum.realistic.QuantumIMU* method), 117
 reset() (*qnav.util.rng.GaussianMarkov* method), 134
 resolution (*qnav.earth.geoid.GeoidPGM* property), 42
 ResultsTable (class in *qnav.output.data*), 112
 rinex_to_satellite_params() (in module *qnav.gps.ephemeris*), 55
 roll_constant_offset (*qnav.waypoints.vehicle.Vehicle* attribute), 143
 roll_max (*qnav.waypoints.vehicle.Vehicle* attribute), 143
 roll_osc_magnitude (*qnav.waypoints.vehicle.Vehicle* attribute), 143
 roll_osc_period (*qnav.waypoints.vehicle.Vehicle* attribute), 143
 rotate_3d() (in module *qnav.util.transformations*), 26
 rotate_3d_vec() (in module *qnav.util.transformations*), 27
 rotation_rate() (in module *qnav.util.transformations*), 26
 rotation_rate_vec() (in module *qnav.util.transformations*), 26
 run_simulation_loop() (in module *qnav.simulation.simulation*), 124, 127
 RungeKutta (class in *qnav.fusion.ins*), 50

S

sat_id (*qnav.gps.satellite.Satellite* property), 60
 sat_time (*qnav.gps.satellite.Satellite* property), 60
 Satellite (class in *qnav.gps.satellite*), 59
 satellite_param_fields (in module *qnav.gps.satellite*), 57
 SatelliteParams (class in *qnav.gps.satellite*), 57
 SatelliteType (class in *qnav.gps.ephemeris*), 54
 save_as() (in module *qnav.output.formats*), 114
 save_as_binary() (in module *qnav.output.formats*), 113
 save_as_csv() (in module *qnav.output.formats*), 113
 save_as_csv_comp() (in module *qnav.output.formats*), 113
 save_as_matlab() (in module *qnav.output.formats*), 114
 save_as_matlab_comp() (in module *qnav.output.formats*), 114
 save_as_numpy() (in module *qnav.output.formats*), 113
 save_as_numpy_comp() (in module *qnav.output.formats*), 113
 save_waypoints() (in module *qnav.waypoints.generation*), 139
 SaveFormat (class in *qnav.output.formats*), 113
 scale_error (*qnav.measurement.error_properties.ErrorProperties* property), 107
 sec_to_str() (in module *qnav.simulation.display*), 128
 second (*qnav.gps.satellite.SatelliteParams* attribute), 59
 Sensor (class in *qnav.measurement.sensor*), 103
 sensor_angles_deg (*qnav.measurement.platform.SensorAxis* property), 105
 sensor_angles_rad (*qnav.measurement.platform.SensorAxis* property), 105
 sensor_axis (*qnav.measurement.accelerometer.Accelerometer* property), 109
 sensor_axis (*qnav.measurement.gyroscope.Gyroscope* property), 110
 sensor_axis (*qnav.quantum.base.ConceptQuantumImu* property), 116
 sensor_iterator() (*qnav.simulation.scheduling.TimingHandler* method), 125
 SensorAxis (class in *qnav.measurement.platform*), 105
 SensorFusion (class in *qnav.measurement.sensor*), 104
 sensors (*qnav.measurement.sensor.SensorFusion* property), 104
 sensors (*qnav.quantum.base.ConceptQuantumFusion* property), 116
 set_default_grid() (in module *qnav.gravity.map*), 73
 set_max_tiles() (*qnav.earth.dted.DtedHandler* method), 39
 set_sensor_used() (*qnav.simulation.scheduling.TimingHandler* method), 125
 SimpleUniform (class in *qnav.gravity.simple*), 69
 solve() (*qnav.gps.fusion.GpsFusion* method), 56
 Somigliana (class in *qnav.gravity.somigliana*), 70
 sqrt_a (*qnav.gps.satellite.SatelliteParams* attribute), 59
 SRTM2GravityFull (class in *qnav.gravity.srtm2gravity*), 90
 SRTM2GravityRes (class in *qnav.gravity.srtm2gravity*), 91

- [start_time](#) (*qnav.waypoints.trajectory.Trajectory* property), 141
[state_vector](#) (*qnav.estimation.state.KalmanEstimatedState* property), 48
[state_vector2ned\(\)](#) (in module *qnav.fusion.kalman*), 52
- ## T
- [t_0e](#) (*qnav.gps.satellite.SatelliteParams* attribute), 59
[take_measurement\(\)](#) (*qnav.gps.sensor.GpsSensor* method), 61
[take_measurement\(\)](#) (*qnav.measurement.accelerometer.Accelerometer* method), 109
[take_measurement\(\)](#) (*qnav.measurement.altimeter.Altimeter* method), 111
[take_measurement\(\)](#) (*qnav.measurement.gyroscope.Gyroscope* method), 110
[take_measurement\(\)](#) (*qnav.measurement.sensor.Sensor* method), 103
[take_measurement\(\)](#) (*qnav.quantum.base.ConceptQuantumImu* method), 115
[take_measurement\(\)](#) (*qnav.quantum.gravity_gradient.GravityGradiometer* method), 120
[take_measurement\(\)](#) (*qnav.quantum.realistic.QuantumIMU* method), 117
[take_measurement\(\)](#) (*qnav.simulation.holonomic.HolonomicConstraint* method), 126
[time_delay_acc](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 143
[time_delay_turn](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 143
[time_step](#) (*qnav.measurement.sensor.Sensor* property), 104
[timestamp](#) (*qnav.estimation.state.EstimatedState* property), 45
[timestamp](#) (*qnav.waypoints.trajectory.GroundTruth* attribute), 140
[TimingHandler](#) (class in *qnav.simulation.scheduling*), 124
[Trajectory](#) (class in *qnav.waypoints.trajectory*), 141
[trigger](#) (*qnav.measurement.sensor.SensorFusion* property), 104
[turn_rate_max](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 143
[type](#) (*qnav.gps.satellite.SatelliteParams* attribute), 59
- ## U
- [update\(\)](#) (*qnav.gps.satellite.Satellite* method), 59
[update\(\)](#) (*qnav.gps.sensor.GpsSensor* method), 61
[update\(\)](#) (*qnav.measurement.accelerometer.Accelerometer* method), 109
[update\(\)](#) (*qnav.measurement.altimeter.Altimeter* method), 111
[update\(\)](#) (*qnav.measurement.clock.Clock* method), 111
[update\(\)](#) (*qnav.measurement.gyroscope.Gyroscope* method), 110
[update\(\)](#) (*qnav.measurement.sensor.Sensor* method), 103
[update\(\)](#) (*qnav.quantum.base.ConceptQuantumImu* method), 115
[update\(\)](#) (*qnav.quantum.gravity_gradient.GravityGradiometer* method), 120
[update_estimates\(\)](#) (*qnav.estimation.state.EstimatedState* method), 46
[update_estimates\(\)](#) (*qnav.estimation.state.KalmanEstimatedState* method), 49
[update_sensors_past_time\(\)](#) (*qnav.simulation.scheduling.TimingHandler* method), 124
[utc2gps\(\)](#) (in module *qnav.util.transformations*), 34
[utm2lla\(\)](#) (in module *qnav.earth.utm*), 44
- ## V
- [variation_per_second1](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 144
[variation_per_second2](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 144
[Vehicle](#) (class in *qnav.waypoints.vehicle*), 142
[vel_ecef2vel_ned\(\)](#) (in module *qnav.util.transformations*), 35
[velocity](#) (*qnav.estimation.state.EstimatedState* property), 45
[velocity](#) (*qnav.gps.satellite.Satellite* property), 60
[velocity](#) (*qnav.waypoints.trajectory.GroundTruth* attribute), 140
[vibration_damping_period](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 144
[vibration_noise_acceleration](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 144
[vibration_noise_angle_rates](#) (*qnav.waypoints.vehicle.Vehicle* attribute), 144
[vned2vecef\(\)](#) (in module *qnav.util.transformations*), 34
- ## W
- [w](#) (*qnav.gps.satellite.SatelliteParams* attribute), 59
[Waypoints](#) (class in *qnav.waypoints.trajectory*), 141
[weighted_cov\(\)](#) (in module *qnav.quantum.particle_filter*), 118
[WGS84Gravity](#) (class in *qnav.gravity.nima*), 72
[write_file\(\)](#) (*qnav.input.config_handler.ConfigHandler* method), 102

Y

`year` (*qnav.gps.satellite.SatelliteParams* attribute), [59](#)